Contract #: 693JJ6-22-C-000037

RCVW Installation Standard Operation Procedures and Checklist

U.S. Department of Transportation
Federal Railroad Administration
Office of Railroad Policy and Development
Office of Research, Development and Technology
Washington, DC 20590

March 7, 2025

BATTELLE

Produced by The U.S. Department of Transportation

with Support from Battelle under Contract 693JJ6 22 C000037

Federal Railroad Administration

Notice

Table of Contents

Page

Table of Contents i
Chapter 1. Introduction 1
Chapter 2. Computing Platform Installation Standard Operating Procedures 3
2.1 Hardware Overview 3
2.1.1 Neousys Technology POC-451VTC 4
2.1.2 Neousys Technology Nuvo-7200VTC 5
2.1.3 Connecting to the Computing Platform 5
2.2 Operating System Overview 9
2.2.1 Setup Ubuntu 22.04.5 LTS Release 10
2.2.2 Setup Docker Community Edition 15
2.2.3 Setup Network Time Synchronization 17
2.2.4 Setup System Logging 19
2.3 Application Software Overview 19
2.3.1 Setup the RCVW Application 20
2.3.2 Building the RCVW Application 25
Chapter 3. Roadside Installation Standard Operating Procedures 30
3.1 Computing Platform Overview 30
3.1.1 Setup the Neousys Technology POC-451VTC 31
3.1.2 Setup the RBS Application 34
3.2 Roadside Unit Overview 37
3.2.1 Setup the Danlaw Technology RouteLink Roadside Unit 37
3.3 Optional Base-Station Overview 44
3.3.1 Setup the u-Blox ZED-F9P Base Station 44
Chapter 4. Vehicle Installation Standard Operating Procedures 47
4.1 Computing Platform Overview 47
4.1.1 Setup the Neousys Technology Nuvo-7200VTC 48
4.1.2 Setup the VBS Application 50
4.2 On-Board Unit Overview 53
4.2.1 Setup Danlaw Technology AutoLink Aftermarket Safety Device (ASD) 54
4.3 Global Navigation Satellite System Overview 60
4.3.1 Setup u-Blox ZED-F9R 61
4.3.2 Setup the VBS Application 62
4.4 Driver-Vehicle Interface Overview 65

4.4.1 Setup the Tru-Vu HDMI Monitor 65
4.4.2 Setup the VBS Application 68
Chapter 5. Microsoft Azure Cloud Installation Standard Operating Procedures 72
5.1 Azure Structured Query Language (SQL) Overview 72
5.1.1 Setup the CBS Database 72
5.2 Azure Service Bus Overview 72
5.3 Azure Container Environment Overview 73
5.3.1 Setup the Function Apps Environment 74
5.3.2 Setup the Container Apps Environment 74
Chapter 6. MAP Messaging Standard Operating Procedures 76
6.1 HRI MAP Messaging Overview 76
6.1.1 Building a MAP Message 77
6.1.2 Configuring the MAP Plugin 82
Chapter 7. SPaT Messaging Standard Operating Procedures 84
7.1 HRI SPaT Messaging Overview 84
7.1.1 Configuring the Preemption Signal 84
7.1.2 Configuring the HRI Status Plugin 85
Chapter 8. RTCM Correction Messaging Standard Operating Procedures 88
8.1 HRI RTCM Messaging Overview 88
8.1.1 Configuring the RTCM Plugin 88

List of Tables

Page

Table 1: Computing Platform Minimum Hardware Specification Checklist....3
Table 2: Computing Platform Minimum Operating System Specification Checklist....9
Table 3: Environment File Minimum Requirement Checklist....20
Table 4: RCVW Messaging Topic Checklist....23
Table 5: RCVW Minimum Build Dependency Checklist....26
Table 6: RBS Environment File Configuration Checklist....35
Table 7: RSU Immediate Forward Messages Checklist....41
Table 8: RSU Immediate Forward Messages Checklist....51
Table 9: VBS GNSS Minimum Hardware Specification Checklist....60
Table 10: DVI Monitor Minimum Hardware Specification Checklist....65
Table 11: CBS Container Environment Configuration Checklist....73
Table 12: CBS RTCM Proxy Container Environment Configuration Checklist....75

List of Figures

Page

Figure 1. RCVW System Architecture Overview....2
Figure 2: Neousys POC-451VTC....5
Figure 3: Neuosys Nuvo-7200VTC....5
Figure 4. Direct Computer Processing Unit (CPU) Connection....6
Figure 5. LAN Connection....6
Figure 6: NetworkManager main screen....11
Figure 7: NetworkManager connection selection screen....12
Figure 8: NetworkManager connection editor screen....13
Figure 9: RBS Software Design....30
Figure 10: RBS Prototype Equipment....31
Figure 11: DIO Port of the Neousys Intel CPU....32
Figure 12. COM ports of the Neousys Intel CPU....32
Figure 13: COM Port BIOS Configuration....33
Figure 14: RouteLink Configuration Tool Login Screen....37
Figure 15: RouteLink Configuration Tool Main Dashboard....38
Figure 16: PoE Injector....39
Figure 17: Neousys Computing Platform Universal Serial Bus (USB) and Ethernet Ports....39

Figure 18: Danlaw Technology RouteLink C-V2X Radio and GPS Antennas 39
Figure 19: RouteLink Configuration Tool Network Screen 40
Figure 20: RouteLink Configuration Tool Messages Screen 41
Figure 21: RouteLink Configuration Tool Logging Screen 43
Figure 22: RouteLink Configuration Tool Security Screen 44
Figure 23: u-Blox C099-F9P USB connection 45
Figure 24: u-Blox C099-F9P antenna connection 45
Figure 25: VBS Software Design 47
Figure 26: VBS Prototype Device-Mounting Plate 48
Figure 27: Nuvo-7200VTC Rotary Ignition Switch on the MezIO™ Module 49
Figure 28: Nuvo-7200VTC Ethernet and PoE ports 50
Figure 29: Danlaw Technology AutoLink C-V2X ASD 54
Figure 30: OBU Mounting Bracket 54
Figure 31: GNSS/C-V2X Radio Antenna 55
Figure 32: Example vehicle antenna positioning 55
Figure 33: u-Blox C102-F9R 61
Figure 34: u-Blox mult-channel GNSS antenna 62
Figure 35: GNSS antenna location 62
Figure 36: RAM Ball mount attached to DVI 66
Figure 37. RAM Pod HD vehicle mount 66
Figure 38. RAM Pod vehicle mount and DVI installation 67
Figure 39. DVI connectors 67
Figure 40. Neousys Intel CPU ports 68
Figure 41. RCVW VBS Application Screen 70
Figure 42. Ubuntu Power Settings Menu 71
Figure 43. Two approaches for two lane nodes 78
Figure 44. Basic Approach Numbering 79
Figure 45. A simple HRI with approach lanes and tracks 80
Figure 46. Approach lane connection over the tracks 81
Figure 47. Railroad track lane connection 81

Chapter 1. Introduction

This document describes the standard operating procedure (SOP) for setup and installation of the Rail-Crossing Violation Warning (RCVW) prototype equipment. The RCVW is a connected vehicle (CV) application designed to distribute state information from a signalized highway-rail intersection (HRI) directly to approaching vehicles as well as indirectly to mobile devices and traffic management centers (TMC). As depicted in

Figure 1, the system contains three major components:

- Roadside-Based Subsystem (RBS)
- Vehicle-Based Subsystem (VBS)
- Cloud-Based Subsystem (CBS)

The RBS captures the signal state of the HRI and distributes that information simultaneously through a cellular vehicle-to-everything (C-V2X) radio to the VBS and over some traditional TCP/IP network to the CBS. This document covers both hardware and software installation and configuration for each of these components, as well as companion checklists for reference.

Whereas the US Department of Transportation (US DOT) Federal Railroad Administration (FRA) and its sub-contractors have provided four pre-configured prototype units of the RBS and VBS, this document details how these may be customized for specific deployment. Additionally, this document can be used to construct additional prototypes, given procurement of the proper equipment. This SOP document supersedes previous iterations of both the RCVW SOP Hardware and Software Configuration (Baumgardner, et. al., 2021) and the RCVW Standard Operating Procedures – Signal Phase and Timing (SPaT), Roadway Geometry (MAP) and Radio Technical Commission for Maritime Services (RTCM) Messaging (Baumgardner, 2021), explicitly covering all of the following procedures:

- Installation and Setup of an RCVW Computing Platform
- Installation and Setup of the RBS component at an HRI Roadside
- Installation and Setup of the VBS component in a vehicle
- Installation and Setup of the CBS component in Microsoft Azure
- Messaging for MAP data via the RBS
- Messaging for SPaT data via the RBS
- Messaging for RTCM corrections via the RBS

Technical Note:

Using this SOP may require a minimum technical skill level in order to install radio equipment, instrument the vehicle and roadside cabinet, and administer and operate Linux computers. It is recommended that performing any complicated tasks be restricted to

certified professionals only in order to reduce the likelihood of damage to equipment or infrastructure.

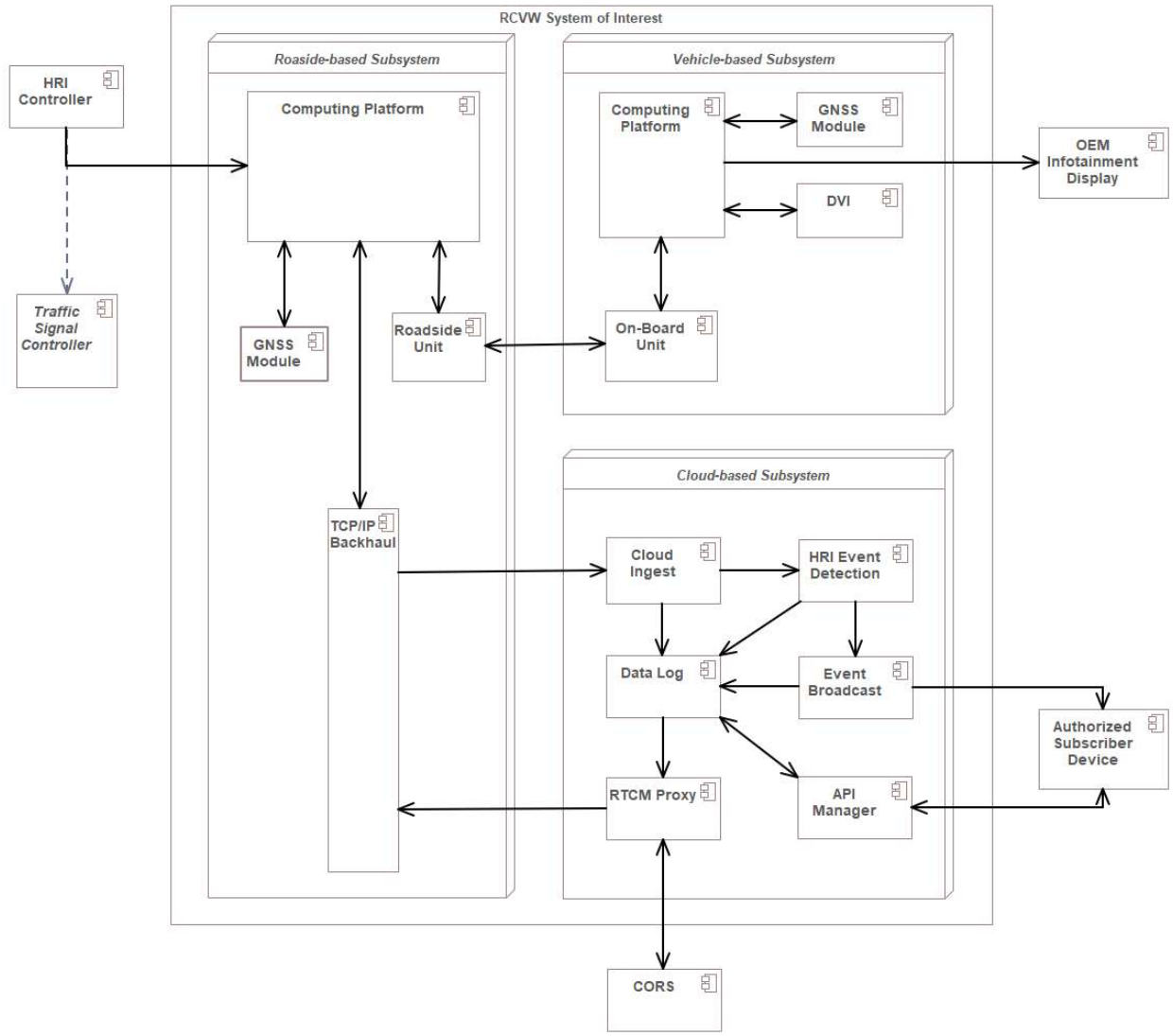


Figure 1. RCVW System Architecture Overview[1]

[1] Taken directly from the RCVW Architecture and Design document (Sanchez-Badillo & Baumgardner, 2024).

Chapter 2. Computing Platform Installation Standard Operating Procedures

The computing platform (CP) is the physical hardware used to employ the V2X Hub software developed under RCVW and previous US DOT projects. The V2X Hub software allows for *plugins* to share information amongst one another through the use of a common messaging system and external broadcast C-V2X radios when necessary. Utilizing V2X Hub facilitates a reduction in deployment time and effort through the reuse of plugins already developed from other projects. The RBS uses V2X Hub on its CP to distribute the SAE J2735 MAP, SPaT and RTCM correction messages for the given HRI to the C-V2X roadside unit (RSU) radio and to the CBS. The VBS also uses V2X Hub on its CP to receive and distribute the incoming J2735 messages from its C-V2X on-board unit (OBU). The CBS does not require a CP because each sub-component runs on virtualized hardware within a cloud service provider instance.

2.1 Hardware Overview

The CP for the RBS and VBS were specifically selected for the demands of the RCVW application. Ideally, both systems would be identical, but due to the high-performance demands of the RCVW application algorithm, a VBS CP with a more advanced processor was chosen for the delivered prototypes. As a result, the VBS CP is more expensive and has a significantly larger form factor than the RBS CP.

Integrated C-V2X radios in the VBS CP may be the desirable architecture. In fact, previous versions of RCVW utilized a single CP that combined Ethernet, Wi-Fi, Long Term Evolution (LTE) and V2X functions. However, this CP supported only Dedicated Short-Range Communication (DSRC) radios, and which they needed to be replaced by C-V2X radios. As a result, the prototype systems have independent CPs and C-V2X radios.

Table 1 shows the minimum specifications required for the RBS and VBS CPs. Check your hardware specifications or use the following Linux commands when applicable to verify.

```
$ sudo lshw
```

Table 1: Computing Platform Minimum Hardware Specification Checklist

Component	Measure	Minimum	Applies To	Check?
Processor	# Cores	>= 4	RBS	
	Speed	>= 2.0 GHz	VBS	
Memory	Total	16 GB	RBS	
			VBS	

Hard-Drive	Size	256 GB	RBS VBS
Network Interface	Ethernet	1x	RBS VBS
Serial Interface	USB	2x	RBS VBS
Digital I/O Interface	DB-9 or DB-15	1x	RBS
***AND/OR* Serial COM Interface**	RS-485	1x	RBS
Video Interface with Audio Input	HDMI or DVI-D	1900x1200 resolution	VBS
Optional Vehicle Interface	CAN 2.0	1x	VBS
Vehicle Power Supply	+8-35 V	3-pin terminal (IGN/GND/V+)	VBS
Environmental	Operating Temperature	-25º – 70ºC	RBS VBS

2.1.1 Neousys Technology POC-451VTC

The RBS prototypes come equipped with the POC-451VTC from Neousys Technology[2] pre-configured for use as the roadside equipment for a local HRI. Refer to the datasheet for more detailed information.

[2] http://neousys-tech.com

Figure 2: Neousys POC-451VTC

2.1.2 Neousys Technology Nuvo-7200VTC

The VBS prototype contain the Nuvo-7200VTC computing platform from Neousys mounted and wired to the deployment plate. Refer to the datasheet for more detailed information.

Figure 3: Neuosys Nuvo-7200VTC

2.1.3 Connecting to the Computing Platform

In order to configure and manage the RCVW system, an external computer is required. The external device should be cabled directly to the RCVW CP using a standard Ethernet cable, or through an Ethernet-based Local Area Network (LAN). See Figure 4 and Figure 5Figure 5, respectively:

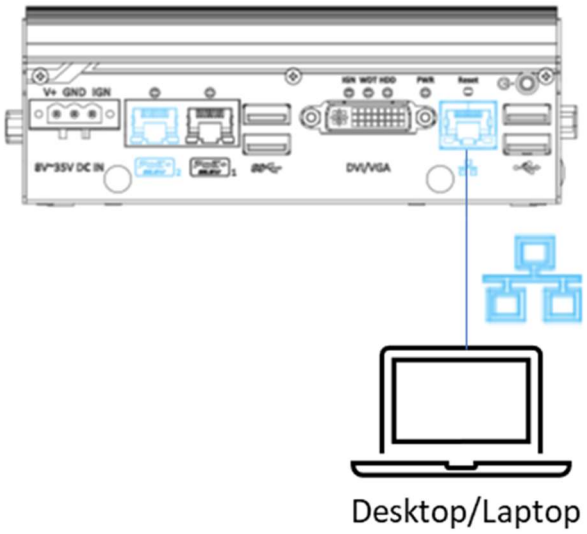


Figure 4. Direct Computer Processing Unit (CPU) Connection

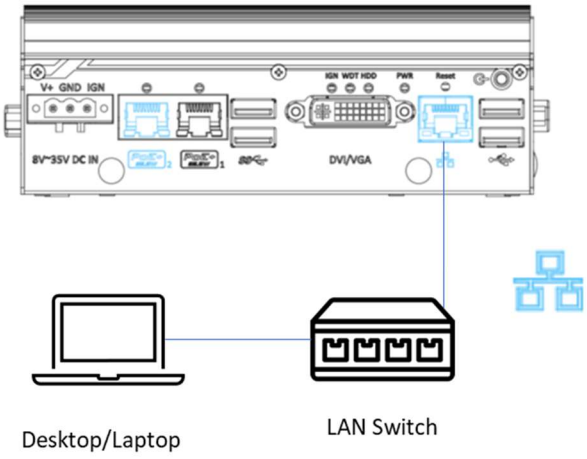
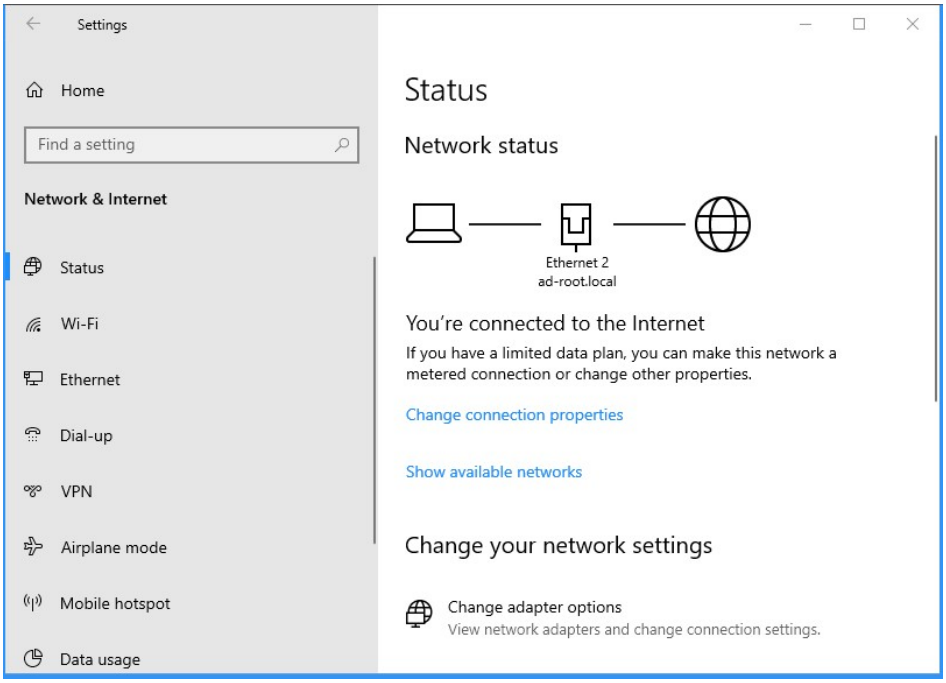
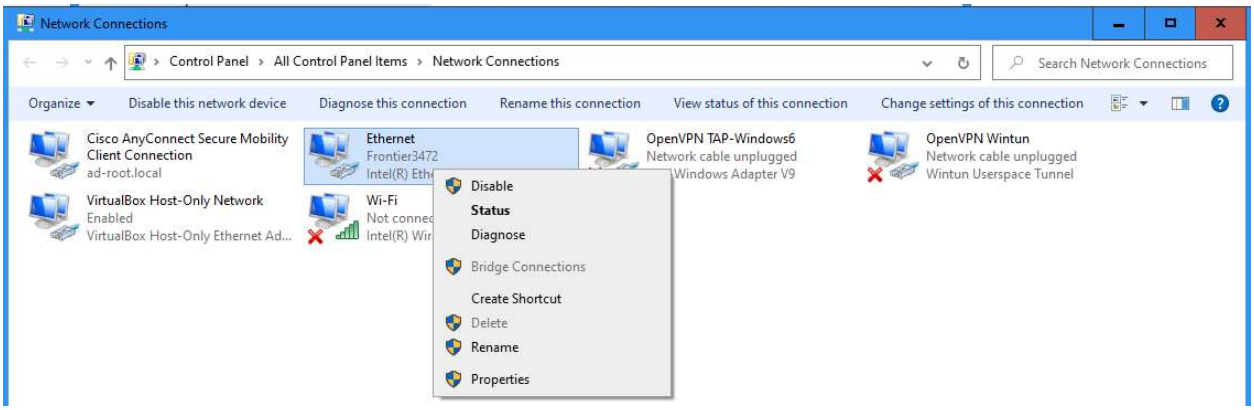


Figure 5. LAN Connection

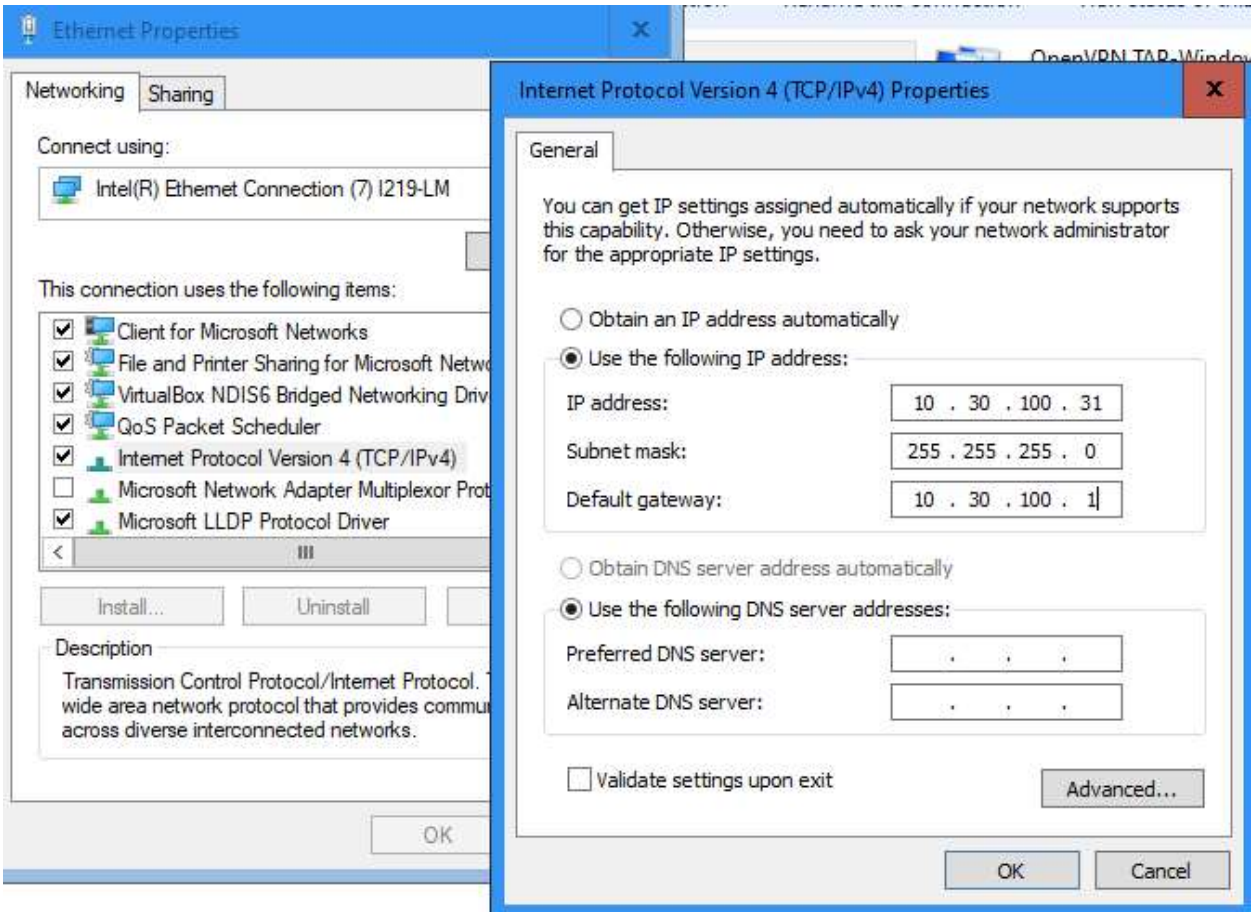
In either setup, the desktop or laptop Ethernet settings must use the same Internet Protocol (IP) subnet as the RCVW CPU, which is 10.30.100.0/24. For a Windows-based system, go to the Network and Internet settings:



Click on **Change adapter options** and bring up the Ethernet **Properties** menu.



You may have to enter an Administrator password. Set the **Internet Protocol Version 4 (TCP/IPv4)** properties to use some IP address within 10.30.100.10 – 10.30.100.50, plus a 255.255.255.0 Subnet mask. The Gateway can optionally be 10.30.100.1.



After saving the settings ping the IP address labeled on the RCVW Neousys CPU. For example, open up a Command Prompt (CMD) window and type:

```
C:\Users>ping 10.30.100.230

Pinging 10.30.100.230 with 32 bytes of data:
Reply from 10.30.100.230: bytes=32 time=59ms TTL=63
Reply from 10.30.100.230: bytes=32 time=57ms TTL=63
Reply from 10.30.100.230: bytes=32 time=66ms TTL=63

Ping statistics for 10.30.100.230:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 57ms, Maximum = 66ms, Average = 60ms
```

2.2 Operating System Overview

The RCVW application runs on the Linux Ubuntu long-term support (LTS) distribution[3] for the CP. The prototype units come pre-installed with version 22.04.5 LTS, also known as *Jammy Jellyfish*. The VBS requires the Desktop edition of Ubuntu in order to work with the supplied Driver-Vehicle Interface (DVI).

The RCVW application deployment is segmented into a set of inter-dependent Docker[4] containers for ease of use. This eliminates the majority of operating system software dependencies and libraries. Instead, the primary software that must be installed should be the Docker program itself. The version of Docker installed must minimally support the `docker config` commands. The RCVW prototypes come pre-configured with all necessary dependencies. The VBS system does require a web browser to use with the DVI. The current software, however, only works with Mozilla FireFox.

Table 2 shows the minimum specifications required for the operating system and dependencies. Follow the subsequent procedures in this section to remedy any missing dependencies.

Table 2: Computing Platform Minimum Operating System Specification Checklist

Dependency	Package	Minimum Version	Applies To	Check?
Operating System Release	`NAME`	Ubuntu	RBS	
`cat /etc/os-release`	`VERSION`	22.04.5	VBS	

[3] https://ubuntu.com/

[4] https://www.docker.com/

Docker `docker --version`	`docker-ce`	>= 27.0	RBS VBS
Docker Compose `docker compose --help`	`docker-compose-plugin`	>= 2.28	RBS VBS
Network Time Protocol `chronyc --version`	`chrony`	>= 4.2	RBS VBS
SSH Server `sshd -?`	`openssh-server`	>= 8.9	RBS VBS

2.2.1 Setup Ubuntu 22.04.5 LTS Release

2.2.1.1 Installing the Operating System

To install a fresh copy of Ubuntu, refer to https://ubuntu.com/tutorials/install-ubuntu-desktop#1-overview for the Desktop edition or https://ubuntu.com/tutorials/install-ubuntu-server#1-overview for the server edition.

If you have a previous patch release of Ubuntu 22.04, such as 22.04.1, then run the following commands to update your system and the verify the version again:

```
$ sudo apt update
$ sudo apt upgrade
$ sudo reboot
```

If you have a previous major version of Ubuntu less than 22.04, then refer to the separate documentation on how to upgrade that system to Ubuntu 22.04 release. For example, https://jumpcloud.com/blog/how-to-upgrade-ubuntu-20-04-to-ubuntu-22-04 can be used to upgrade from 20.04 to 22.04. Note that this process generally takes a very long time, does not always take you to the most recent version available, and it therefore is recommended to just install a fresh copy of Ubuntu instead.

2.2.1.2 Configuring the Network

When using Ubuntu Desktop, the system comes with the NetworkManager package in order to help setup the network connectivity. There is a user interface accessible on the top right of the user desktop, or the following command can be executed in a terminal:

```
$ sudo nmtui
```

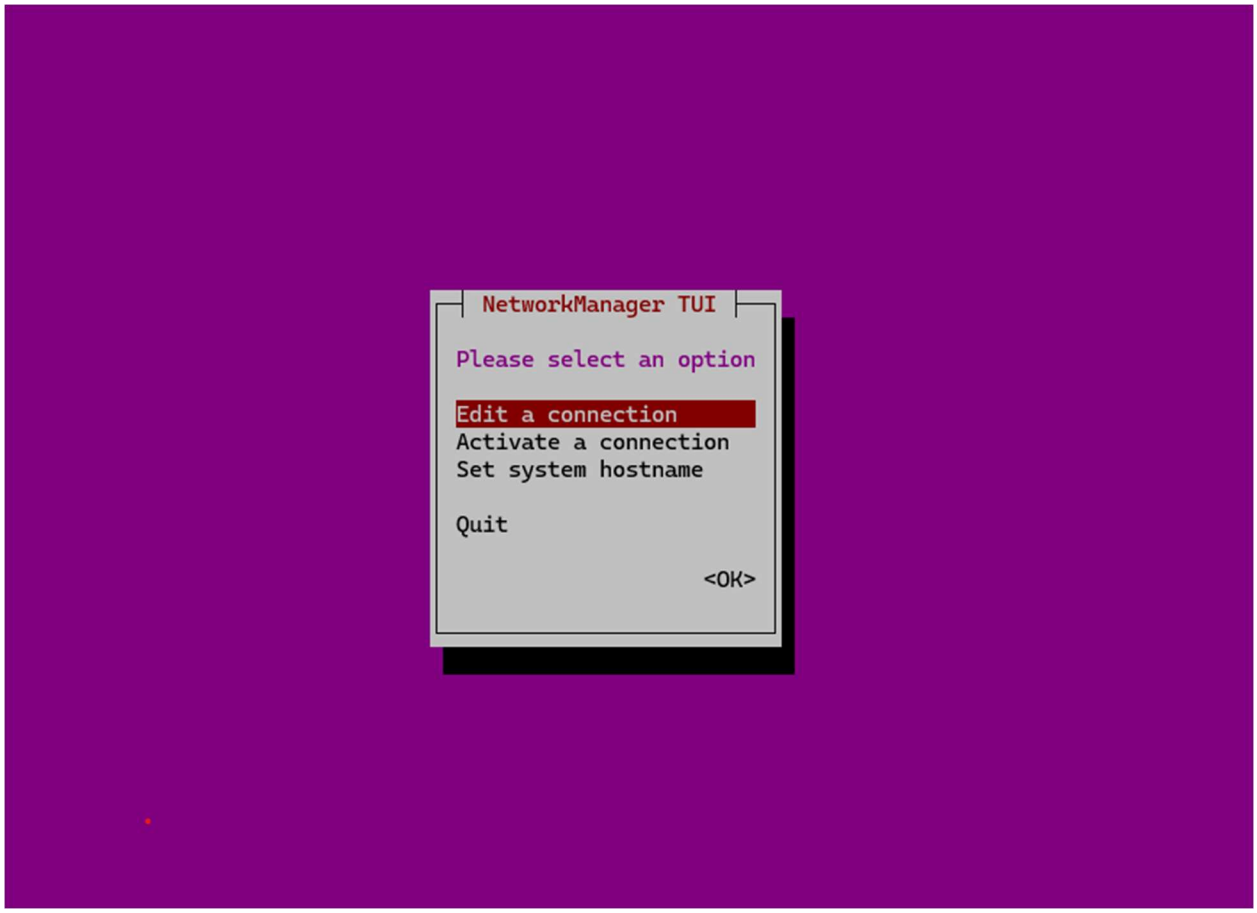


Figure 6: NetworkManager main screen

Assuming the hardware connection are already in place, select ***`Edit a connection`***. The list of Ethernet interfaces comes up. On the VBS prototype, for example, there will be six options, one for each of the Ethernet ports available on the front side of the device. Presuming the other interfaces are not in use, the top one in the list should be the connected device. Select the proper connection from the Ethernet list and hit **`Enter`**.

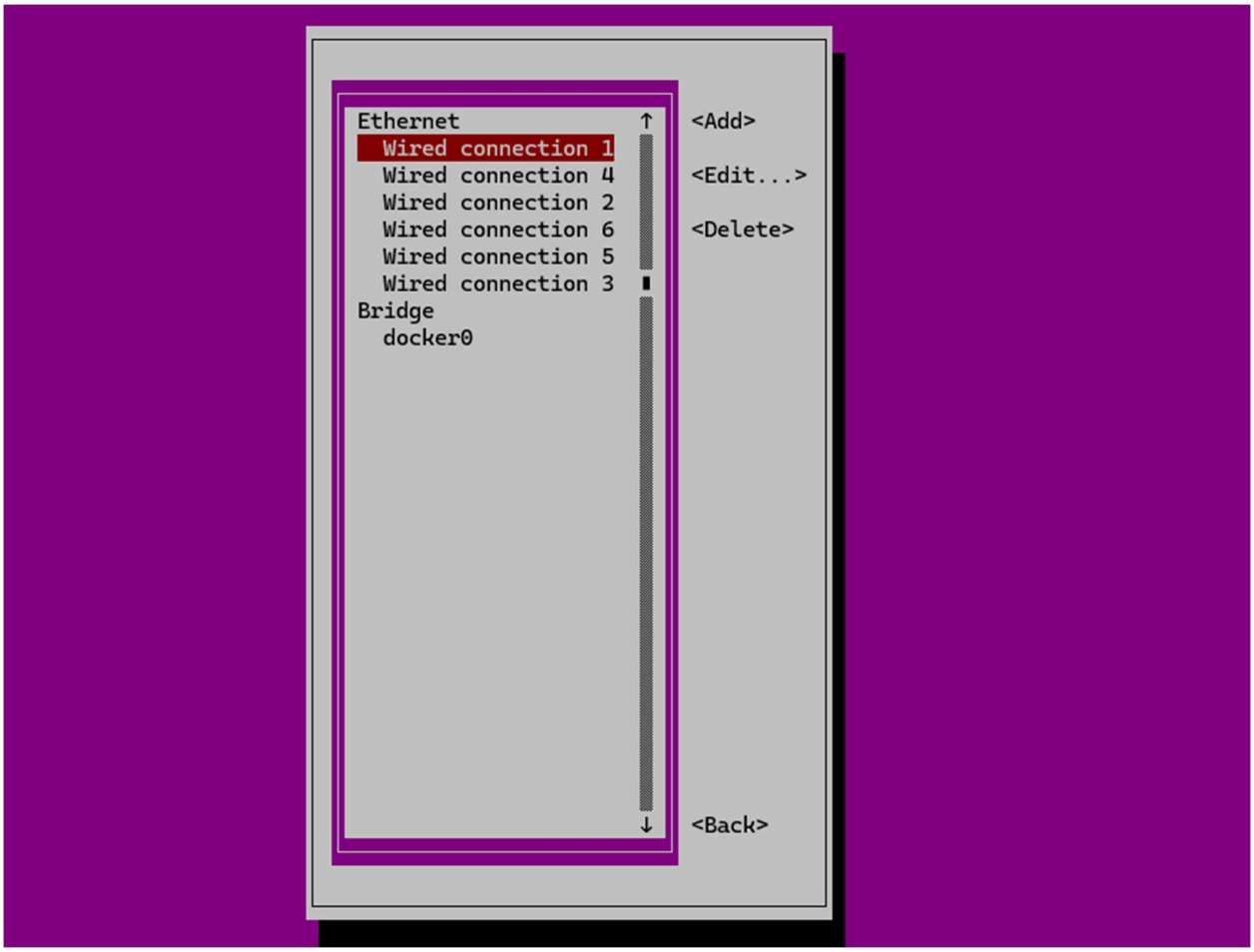


Figure 7: NetworkManager connection selection screen

The connection editor screen for the select connection appears. At minimum, fill in the **`Addresses`**, **`Gateway`** and **`DNS Servers`** fields. The example in Figure 8 sets the IP address to `192.168.100.24` and the default gateway of `192.168.100.1`. The default values for the rest should suffice. Then, navigate with the **`Tab`** key to the **`<Ok>`** button and hit **`Enter`**.

The connection list screen reappears. Navigate to the **`<Back>`** button and hit **`Enter`**.

The main screen reappears. Navigate to **`Quit`** and hit **`Enter`**.

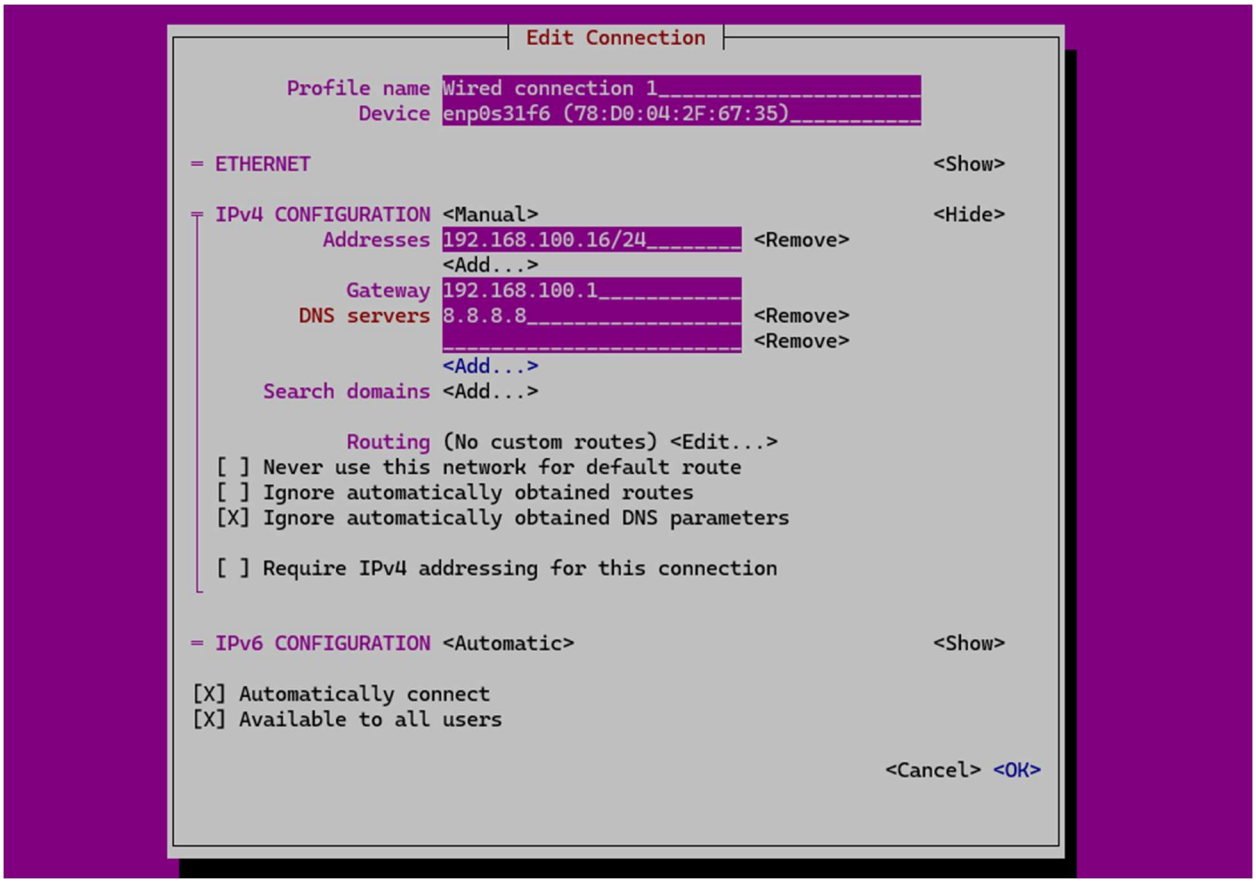


Figure 8: NetworkManager connection editor screen

After rebooting, verify that the network address has been set and that the Ubuntu update servers can now be reached.

```
$ sudo systemctl enable systemd-resolved.service
$ sudo reboot
$ ip address
$ sudo apt update
```

Next, install the OpenSSH server to allow for remote login to this device.

```
$ sudo apt install -y openssh-server
```

2.2.1.3 Configuring the Operating System

A non-root admin user should be created on each CP in order to setup and run the RCVW application software. This is best accomplished during the installation of Ubuntu, but the following steps can be used to create a new user. Note that the prototype systems all contain a

admin user called `user` with the password of `RCVW@pplication!`, including the punctuation. This user must also be part of the docker group in order to run the application.

```
$ sudo useradd -c "Rail-Crossing Violation Warning" -d /home/user -m -s /bin/bash -G
adm,sudo user
$ sudo passwd user
New password: RCVW@pplication!
Retype new password: RCVW@pplication!
passwd: password updated successfully
```

Additionally, it is helpful to set the `hostname` parameters for the CP based on the hardware identification and serial number, if known, as well as the deployment component, e.g., RBS or VBS.

For example, the RBS prototype on the POC 451VTC with serial number N1900078 to be deployed at the HRI near Phillips and Sunbeam, this can be done with the following commands:

```
$ sudo hostnamectl hostname rcvw-rbs-N1900078
$ sudo hostnamectl deployment rcvw-rbs
$ sudo hostnamectl location 'Phillips & Sunbeam'
$ hostnamectl status
Static hostname: rcvw-rbs-N1900078
       Icon name: computer-laptop
         Chassis: laptop
      Deployment: rcvw-rbs
        Location: Phillips & Sunbeam
      Machine ID: 0a066371e27c46ed8468a322897a050d
         Boot ID: 8a41adafce9b4e529e4f57d1d40ba529
Operating System: Ubuntu 22.04.5 LTS
          Kernel: Linux 6.2.0-37-generic
    Architecture: x86-64
 Hardware Vendor: Neousys Technology Inc.
  Hardware Model: POC-451VTC
```

Or, for the VBS prototype on the Nuvo 7204VTC with serial number N0800032 install in a 2009 Mazda 3, run the following:

```
$ sudo hostnamectl hostname rcvw-vbs-N0800032
$ sudo hostnamectl deployment rcvw-vbs
```

```
$ sudo hostnamectl location '2009 Mazda 3'
$ hostnamectl status
 Static hostname: rcvw-vbs-N0800032
       Icon name: computer-laptop
         Chassis: laptop
      Deployment: rcvw-vbs
        Location: 2009 Mazda 3
      Machine ID: 994c6cbad8424dddbfe9cb95d934b36c
         Boot ID: 31e0c6182c4045fab89ac492042c51b7
Operating System: Ubuntu 22.04.5 LTS
          Kernel: Linux 6.5.0-41-generic
    Architecture: x86-64
 Hardware Vendor: Neousys Technology Inc.
  Hardware Model: Nuvo-7100VTC Series
```

By default, Ubuntu comes with an automatic package update that may accidentally break the operation of the HRI detection system on the RBS. Likewise, the update process also periodically pops up a notification window that may interfere with the RCVW alerting system on the VBS. Therefore, the automatic package update should be completely removed.

```
$ sudo apt purge -y unattended-upgrades
$ sudo apt purge -y update-notifier update-notifier-common
$ sudo apt purge -y update-manager update-manager-core
```

Similarly, the AppArmor access control security framework should also be removed from the Ubuntu installation since it occasionally interferes with normal RCVW operations. Note that this may attempt to remove `snapd` as well, but it is not critical for that removal to be successful.

```
$ sudo apt -y purge apparmor
$ sudo apt autoremove -y
```

2.2.2 Setup Docker Community Edition

The base Ubuntu system may come with Docker pre-installed. Unfortunately, this version of Docker does not include the necessary features. Therefore, it is recommended to install the latest version of the Docker Community Edition by following the following instructions[5]. Note that the prototype systems each have Docker Community Edition version 27.5.1 pre-installed.

[5] Adapted from https://docs.docker.com/engine/install/ubuntu/

```
$ sudo apt remove -y docker.io docker-doc docker-compose docker-compose-v2 podman-
docker containerd runc
$ sudo apt update
$ sudo apt -y install ca-certificates curl
$ sudo install -m 0755 -d /etc/apt/keyrings
$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
$ sudo chmod a+r /etc/apt/keyrings/docker.asc
$ echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
$ sudo apt update
$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

The default configuration of Docker only allows the local `root` login to access the commands. However, new administrative users may also be allowed to run Docker if they are added to the local `docker` group. This must be done for the RCVW login `user`.

```
$ sudo usermod -G docker -a user
```

Login back into the CP with the RCVW login `user` to verify Docker.

```
$ docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:d715f14f9eca81473d9112df50457893aa4d099adeb4729f679006bf5ea12407
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
```

```
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

2.2.3 Setup Network Time Synchronization

It is crucial to CV applications to have accurate timing. Network connected systems tend to synchronize their clocks to central Network Time Protocol (NTP) servers, while disconnected systems might instead utilize timing from a Global Navigation Satellite System (GNSS) signal such as Global Positioning System (GPS). The Linux `chrony` server is designed to receive inputs from these multiple timing sources and apply gradual adjustments to the local clock. Both C-V2X OBU and RSU radios already come with `chrony` pre-configured to receive the GPS time. However, the RBS and VBS computing platforms should be set up to poll the local GPS, if one is attached, or otherwise the attached radio or some other network time source. Note that the prototype systems are already set up this way.

First, make sure that the `chrony` package is installed.

```
$ sudo apt install -y chrony
```

Secondly, ensure that the local C-V2X radio is configured within the `hosts` file. At this stage, it is not important that the radio is connected to the same network as the CP, but only that the IP address of the device is known. These steps assume that the IP address of the radio devices are specified in IP version 4 format, i.e., `aa.bb.cc.dd`.

For the RBS only, add the IP address of the RSU to the `hosts` file and verify. For example, if the RSU address is `192.168.100.22`:

```
$ echo 192.168.100.22 local-rsu | sudo tee -a /etc/hosts
$ host -t A local-rsu
```

```
local-rsu has address 192.168.100.22
```

For the VBS only, add the IP address of the OBU to the `hosts` file and verify. For example, if the OBU address is `192.168.100.23`:

```
$ echo 192.168.100.23 local-obu | sudo tee -a /etc/hosts
$ host -t A local-obu
local-obu has address 192.168.100.23
```

Make sure to add the -a option to the tee command to append to the file instead of overwriting it completely.

Finally, to add the sequence of time sources to use with `chrony`. By default, some well-known NTP servers are pre-configured. However, since those may be unavailable unless the CP is connected to an external network, the system must also try to utilize the existing radio:

For the RBS only:

```
$ echo peer local-rsu iburst | sudo tee -a /etc/chrony/chrony.conf
peer local-rsu iburst
```

For the VBS only:

```
$ echo peer local-obu iburst | sudo tee -a /etc/chrony/chrony.conf
peer local-obu iburst
```

Then, the local GPS time and pulse per second (PPS) from any attached GNSS receiver.

```
$ echo refclock SHM 0 refid GPS precision 1e-1 offset 0.9999 delay 0.2 | sudo tee -a
/etc/chrony/chrony.conf
refclock SHM 0 refid GPS precision 1e-1 offset 0.9999 delay 0.2
$ echo refclock SHM 1 refid PPS precision 1e-7 | sudo tee -a /etc/chrony/chrony.conf
refclock SHM 1 refid PPS precision 1e-7
```

Restart the `chrony` server and verify the sources. Note that if the `LastRx` field is denoted as a dash (`-`), then the associated source is not connected and therefore not used in the synchronization process. This would be the case if the local radio is not reachable, or if no local GNSS receiver is configured. Both of these cases are as shown below for an example VBS.

```
$ sudo systemctl restart chronyd
```

```
$ chronyc sources
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
#? GPS                           0   4     0     -     +0ns[   +0ns] +/-    0ns
#? PPS                           0   4     0     -     +0ns[   +0ns] +/-    0ns
^? prod-ntp-4.ntp4.ps5.cano>     2   6     3     1  +2606us[+2606us] +/-   50ms
^? alphyn.canonical.com          2   6     3     1  -1915us[-1915us] +/-   39ms
^? prod-ntp-5.ntp1.ps5.cano>     2   6     3     2  +1933us[+1933us] +/-   50ms
^? prod-ntp-3.ntp1.ps5.cano>     2   6     3     2  -2244us[-2244us] +/-   46ms
^? ntpool1.603.newcontinuum>     2   6     3     2   -412us[ -412us] +/-   27ms
^? 208.67.72.43                  3   6     3     2  -6161us[-6161us] +/-   90ms
^- tools.ninjaneering.net        2   6    17     8   +160us[ +160us] +/-   85ms
^+ us-west-1.clearnet.pw         2   6    17     8   +551us[  +59us] +/-   58ms
=? local-obu                     0   6     0     -     +0ns[   +0ns] +/-    0ns
```

2.2.4 Setup System Logging

The RCVW application by default logs to the system `journald` logging component. This system must be configured properly to ensure no application logging messages are missed:

```
$ sudo mkdir -p /var/log/journal
$ echo RateLimitIntervalSec=0 | sudo tee -a /etc/systemd/journald.conf
$ echo RateLimitBurst=0 | sudo tee -a /etc/systemd/journald.conf
```

2.3 Application Software Overview

The RCVW application is an open-source project sponsored by the US DOT and made available for public use. See section 2.3.2 for details on how to attain and compile the source code. However, the only binary distribution of the RBS and VBS application software offered by the government is included with the four pre-built prototypes. It is possible to extract the Docker containers directly from the prototypes, if available.

```
$ docker image ls --format '{{.Repository}}:{{.Tag}}' | grep "^rcvw-" | \
        xargs docker save | gzip > rcvw-images.tgz
```

The resulting `rcvw-images.tgz` file can be transferred to a newly configured CP and installed:

```
$ docker load --input rcvw-images.tgz
```

This, however, does not copy over the Docker compose configuration. Instead, archive the docker compose files:

```
$ find /usr/local/share/rcvw/ -type f | tar -cvf rcvw-docker-compose.tgz -T -
```

Then, copy over the resulting `rcvw-docker-compose.tgz` file in the new CP and install:

```
$ sudo tar -xvf rcvw-docker-compose.tgz -C /
```

Unfortunately, there is currently no binary distribution image for any of the CBS software directly available within the prototype kits. Therefore, in order to ensure completeness of a new deployment, it is recommended to build the images from source code, as described in the section 2.3.1.2.

2.3.1 Setup the RCVW Application

As previously stated, the RCVW application runs using the Docker `compose` functionality, which is configurable through environment variables. Each computing platform comes pre-configured with an environment stored in the `~user/.env` file that allows for a functional test operation of the system. However, live deployments of RCVW require modifications to this environment file in order to perform optimally.

The RCVW source code contains a base version of the environment file that is functional, but the prototype implementations modify that version slightly on the VBS to capture the unique hardware serial number of the device.

Subsequent sections of this document describe each configuration variable for the given component. However, Table 3 below lists the default supplied parameters that are specified by default in the prototype environment in order to provide a plug-and-play operational environment. Note, however, that this configuration was not necessarily intended to function in live deployments and must be updated for specific use.

```
$ cat ~user/.env
```

Table 3: Environment File Minimum Requirement Checklist

Variable	Recommended Value	Default Value	Applies To	Check?
RCVW_HRI	HRI Identifier number specified in the MAP	`27812`	RBS CBS	
RCVW_MAP	Hexadecimal encoded J2735 MapData bytes	An RCVW compliant map used in the	RBS	

		RCVW Jacksonville deployment[6]	
RCVW_RSU_AUTHPASS	The authentication password for NTCIP (SNMP v3) connection to the local RSU	Private	RBS
RCVW_RSU_PRIVPASS	The privacy password for NTCIP (SNMP v3) connection to the local RSU	Private	RBS
RCVW_CBS_HOST	The hostname for the CBS messaging service, e.g., Azure Service Bus	Private	RBS CBS
RCVW_CBS_USER	The user or access key name for the CBS messaging service	Private	RBS CBS
RCVW_CBS_PASS	The password or access key for the CBS messaging service	Private	RBS CBS
RCVW_DB_HOST	The hostname for the CBS database storage component, e.g., Azure SQL	Private	CBS
RCVW_DB_USER	The user or access key name for the CBS database storage component	Private	CBS
RCVW_DB_PASS	The password or access key name for the CBS database storage component	Private	CBS
RCVW_CBS_API	The hostname for the CBS API service	Private	CBS

[6] See Appendix B

2.3.1.1 Running the application

After the environment file is set up properly, the configuration can be verified and then, if no error exists, the application may be launched.

For the RBS only, this can be done by executing the following commands:

```
$ docker compose --env-file ~user/.env \
        --project-directory /usr/local/share/rcvw/deploy/ --profile rbs config
$ docker compose --env-file ~user/.env \
        --project-directory /usr/local/share/rcvw/deploy/ \
        up --force-recreate -d rcvw-rbs
```

For the VBS only, run the following:

```
$ docker compose --env-file ~user/.env \
        --project-directory /usr/local/share/rcvw/deploy/ --profile vbs config
$ docker compose --env-file ~user/.env \
        --project-directory /usr/local/share/rcvw/deploy/ \
        up --force-recreate -d rcvw-vbs
```

There are also services for each of the four CBS components in order to run those on the local CP for testing purposes. For convenience, a top-level Docker `compose` project file can be added to the `user` home directory to avoid having to repeat the `--env-file` and the `--project-directory` arguments to the previous commands.

```
$ printf 'include:\n    - ${RCVW_HOME:-/usr/local/share/rcvw/deploy}/docker-
compose.yaml\n' > ~user/docker-compose.yaml
```

For the RBS:

```
$ cd ~user
$ docker compose --profile rbs config
$ docker compose up --force-recreate -d rcvw-rbs
```

For the VBS:

```
$ cd ~user
$ docker compose --profile vbs config
```

```
$ docker compose up --force-recreate -d rcvw-vbs
```

Note that this also allows for relocation of the provide Docker deployment files to a different location for further customization of the deployment. This can be done by setting an additional environment variable as seen below. Note, however, that this is only to be done if customization of the deployment is required beyond the scope of what is provided in the environment file.

```
$ export RCVW_HOME=/path/to/deployment/files
```

2.3.1.2 Verifying the Application

All processes can be verified as running by executing the following Docker command:

```
$ docker compose ps –all --json
```

Any service comes with a log that can be read from Docker. For example, the MAP plugin:

```
$ docker compose logs -f rbs-map
```

Additionally, any RCVW plugin service can be used to detect the status of all connected plugins.

```
$ docker compose exec rbs-map tmxctl --plugins --context kafka://localhost:29092
```

Or, to check for open Apache Kafka topics:

```
$ docker compose exec rbs-map tmxctl --info --context kafka://localhost:29092
```

Or even to receive the next incoming messages at one of those topics:

```
$ docker compose exec rbs-map tmxctl --topic J2735.MAP --context
kafka://localhost:29092
```

Additionally, there is a tool available on Ubuntu that can provide more detailed information about Apache Kafka instance and the messages flowing through it.

```
$ sudo apt install -y kafkacat
$ kafkacat -b :29092 -L -J
$ kafkacat -b :29092 -t J2735.MAP -o end -J
```

There should be the following topics in Apache Kafka. Additionally, much of the RBS topics will also show up in the CBS Service Bus.

Table 4: RCVW Messaging Topic Checklist

Name	Description	Applies To	Check?
J2735.MAP	The HRI MAP message, coming from the MAP plugin once per second.	RBS VBS CBS	
J2735.SPAT	The HRI SPAT message, coming from the HRI Status plugin ten times per second.	RBS VBS CBS	
J2735.RTCM	The RTCM corrections for the HRI, coming from the RTCM plugin as corrections are received.	RBS VBS	
J2735.RSA	The road-side alert message for a vehicle to detect weather information, not generally used.	VBS	
tmx.plugin.v2x.HRIStatusPlugin .HRIStatusPlugin.error	The errors generated by the HRI Status plugin.	RBS CBS	
tmx.plugin.v2x.HRIStatusPlugin .HRIStatusPlugin.status	The diagnostic status information generated by the HRI Status plugin.	RBS CBS	
tmx.plugin.v2x.Map .MapPlugin.error	The errors generated by the MAP plugin.	RBS CBS	
tmx.plugin.v2x.Map .MapPlugin.status	The diagnostic status information generated by the MAP plugin.	RBS CBS	
tmx.plugin.v2x.RSU .RSUImmediateForwardPlugin .error	The errors generated by the C-V2X Immediate Forward plugin.	RBS CBS	

tmx.plugin.v2x.RSU .RSUImmediateForwardPlugin .status	The diagnostic status information generated by the C-V2X Immediate Forward plugin.	RBS CBS
tmx.plugin.v2x.rtcm .RtcmPlugin.error	The errors generated by the RTCM plugin.	RBS CBS
tmx.plugin.rtcm .RtcmPlugin.status	The diagnostic status information generated by the RTCM plugin.	RBS CBS
tmx.plugin.v2x.MessageReceiver .MessageReceiverPlugin.error	The errors generated by the Message Receiver plugin.	VBS
tmx.plugin.v2x.MessageReceiver .MessageReceiverPlugin.status	The diagnostic status information generated by the Message Receiver plugin.	VBS
tmx.plugin.v2x.differentialGPS .DifferentialGPSPlugin.error	The errors generated by the Differential GPS plugin.	VBS
tmx.plugin.v2x.differentialGPS .DifferentialGPSPlugin.status	The diagnostic status information generated by the Differential GPS plugin.	VBS
tmx.plugin.rcvw.app .RCVWPlugin.error	The errors generated by the RCVW application plugin.	VBS
tmx.plugin.rcvw.app .RCVWPlugin.error	The diagnostic status information generated by the RCVW application plugin.	VBS
V2X.Application	HMI notification messages, including faults and alerts, coming from the RCVW application as generated.	VBS
V2X.VBM	Basic telemetry information for the vehicle, not generally used.	VBS
V2X.RTCM3	RTCM v3 correction data, coming from the RTCM plugin when a local base station is used.	RBS CBS

2.3.2 Building the RCVW Application

Alternatively, new RCVW images for the RBS, the VBS or the CBS can be constructed directly from the source code, which is available on GitHub:

https://github.com/RCVW-PHASEIII/RCVW

The minimum software requirements list in Table 5 must be resolved to correctly compile the RCVW deployments. Note that this may require installation of a pre-compiled binary package or a new compile of the minimum version on your build machine. Much of these dependencies are required to successfully execute the `cmake` setup. However, those explicitly listed as applying only to the RBS component means that one could successfully build functional VBS and CBS components without these dependencies.

Table 5: RCVW Minimum Build Dependency Checklist

Dependency	Package	Minimum Version	Applies To	Check?
Git `git --version`	`git`	>= 2.34.1	RBS VBS CBS	
CMake `cmake --version`	`cmake`	>= 3.18	RBS VBS CBS	
GNU C++ Compiler `g++ --version`	`gcc` `g++`	>= 9.4.0 >= 9.4.0	RBS VBS CBS	
Package Config `pkg-config --version`	`pkg-config`	>= 0.29	RBS VBS CBS	
Boost `dpkg -s libboost-dev`	`libboost-all-dev`	>= 1.71	RBS VBS CBS	
Apache Kafka `dpkg -s librdkafka-dev`	`librdkafka-dev`	>= 1.8.0	RBS VBS CBS	

GPS `dpkg -s libgps-dev`	`libgps-dev`	>= 3.22	RBS VBS CBS
UUID `dpkg -s uuid-dev`	`uuid-dev`	>= 2.34	RBS VBS CBS
USB `dpkg -s libusb-1.0-0-dev`	`libusb-1.0-0-dev`	>= 1.0.23	RBS VBS CBS
Apache Qpid Proton `dpkg -s libqpid-proton-cpp12`	`libqpid-proton-cpp12`	>= 0.39[7]	RBS
SNMP `dpkg -s libsnmp-dev`	`libsnmp-dev`	>= 5.8	RBS

2.3.2.1 Building and Installing Core Components

Clone the repository and setup a build directory to create the `make` environment. The `TMX_PROJECT` variable is key to ensuring that all the RCVW code is compiled and packaged.

```
$ git clone https://github.com/RCVW-PHASEIII/RCVW.git
$ mkdir RCVW/build/$(dpkg --print-architecture)-Debug
$ cd RCVW/build/$(dpkg --print-architecture)-Debug
$ cmake -DTMX_PROJECT=rcvw ..
$ make package-all
```

After successfully running these commands, the build directory you create should now have the following artifacts:

`bin` – The program binaries, generally just `tmxctl` and a few test programs. These will be installed under `/usr/local` by default.

`lib` – The shared library binaries. These will be installed under `/usr/local` by default.

[7] This version is not the default for Ubuntu 22.04, thus most likely requires a manual compilation and install on the build system. The RCVW packaging includes the Qpid Proton libraries required, so this dependency is only for the build environment

`rcvw` – The RCVW specific RBS and VBS plugin files. These will be installed under `/var/lib/plugin` by default.

`v2x-hub` – The V2X Hub specific RBS and VBS plugin files. These will be installed under `/var/lib/plugin` by default.

These compiled components can be installed directly on the machine with the following command. Note, however, that additional dependencies need to be installed, and much configuration of the plugins would still be required.

```
$ sudo make install
```

Additionally, the individual components come with the base binary files already packaged into a TAR or ZIP archive, or into the Debian/Ubuntu package file. The former archives can be copied over and extracted directly on the target host machine, while the latter can be installed on a Debian/Ubuntu target as shown below. The Debian/Ubuntu native distributions currently have no dependency management nor special configuration options, making them no more useful than the archives.

```
$ sudo dpkg -i rcvw-4.0.0-*.deb
```

2.3.2.2 Building Deployment Images

The most convenient distribution for RCVW would be to create and use the Docker images that are available within the `cmake` environment. This makes deployment easier since all the dependencies required to run the component are already built into the image, as is the configuration. One should note that the CBS components can only be built into a Docker deployment image anyway. However, due the way that the build is structured, the Docker image creation requires that Docker is installed on the build system, as described in section 2.2.2. Additionally, the minimum environment setup described in Table 3 must be completed in order to ensure the container are built, since the build directly uses the Docker `compose` deployment file to build images. This environment can either be setup into the final `~user/.env` file and copied to the source folder or edited directly within the source folder.

```
$ cp ~user/.env RCVW/build/Docker/deploy/rcvw-all/
$ cmake -DTMX_PROJECT=rcvw ..
$ make package-all
```

Note that the package-all target is a pre-requisite for any RCVW image. Therefore, it must always be run before attempting to create a Docker image.

The Roadside-based Subsystem compiles into a single Docker image.

```
$ make docker-image-rcvw-rbs
```

The Vehicle-based Subsystem compiles into a single Docker image.

```
$ make docker-image-rcvw-vbs
```

The Cloud-based Subsystem has multiple Docker images.

```
$ make docker-image-cbs-api
$ make docker-image-cbs-event-mgr
$ make docker-image-cbs-ingest
$ make docker-image-cbs-rtcm
```

Refer to section 2.3.12.3.1.1 for further instructions on running the Docker-based RCVW application.

Chapter 3. Roadside Installation Standard Operating Procedures

3.1 Computing Platform Overview

As denoted in Figure 25, the VBS software architecture includes multiple components running on the computing platform, including:

- HRI Status plugin
- MAP plugin
- RTCM plugin
- C-V2X Immediate Forward plugin

These software components operate as one or more Docker *services*, each running inside a separate *container*.

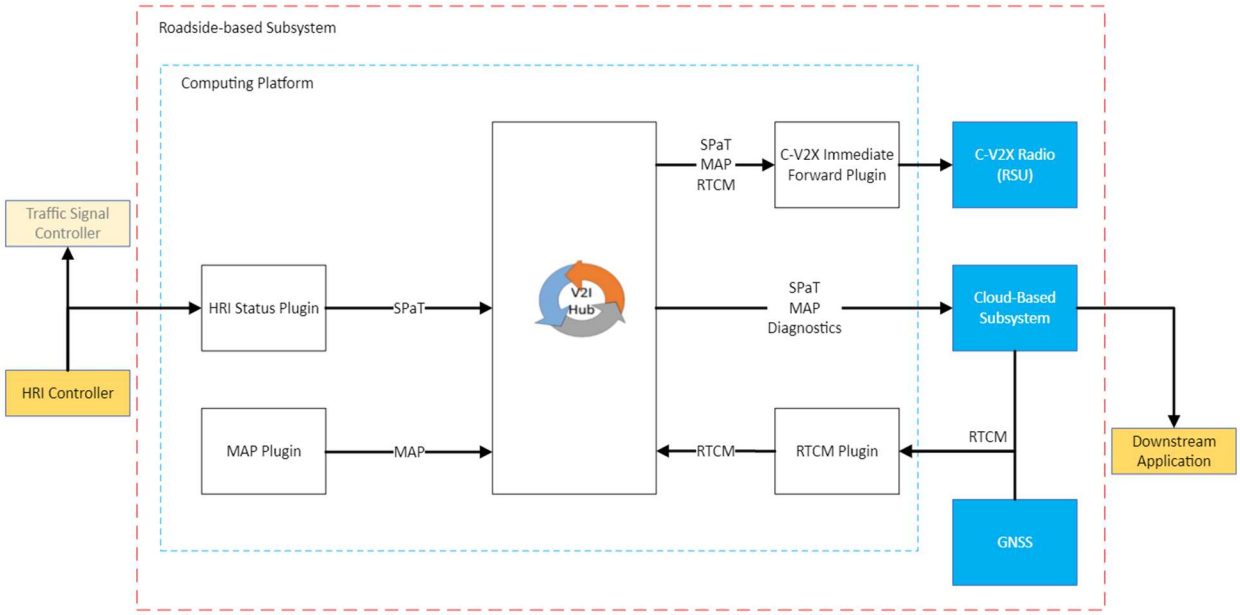


Figure 9: RBS Software Design[8]

Additionally, the RBS has hardwired connectivity to receive preemption signal status, either via the HRI controller provided by the railroad or the traffic signal controller (TSC) provided by the DOT locality. The following should be considered before installing the Roadside Subsystem:

[8] Taken directly from the RCVW Architecture and Design document (Sanchez-Badillo & Baumgardner, 2024)

- A National Electrical Manufacturers Association (NMEA) rated enclosure is required to house the Roadside CP, the Power Over Ethernet (PoE) injector, and the optional U-blox C099-F9P base station to protect them from the environment.
- The Roadside subsystem requires 110 Volts Alternating Current (VAC) power.
- The GNSS Base Station Antenna's precise longitude, latitude and elevation is needed for the base station to produce accurate RTCM corrections. For this purpose, a survey grade GPS device is needed to acquire precise antenna coordinates.
- If interfacing with an operating railroad, consult with authorized railroad personnel for connectivity.

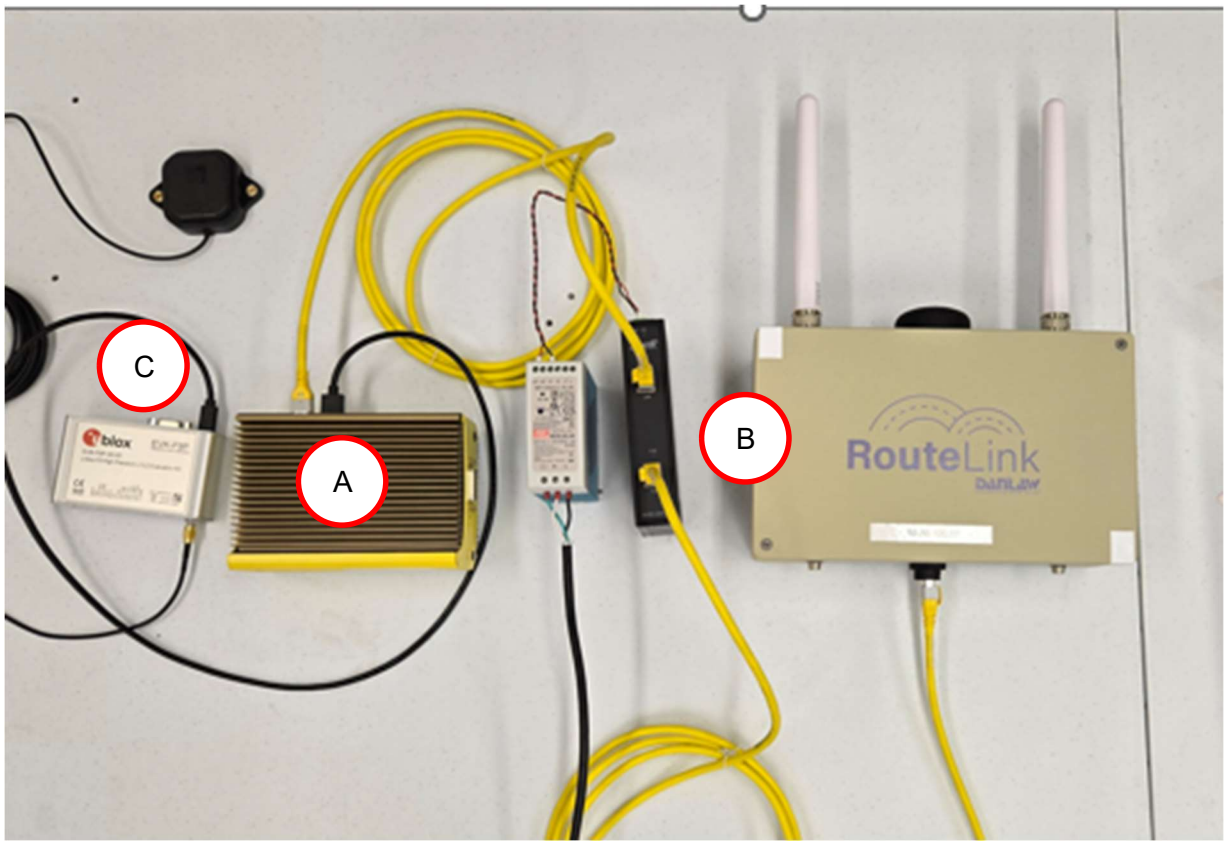


Figure 10: RBS Prototype Equipment

The RBS prototype consists of the following equipment (Figure 10):

A. Neousys Technology POC-451VTC Computing Platform
B. DanLaw Technology RouteLink RSU
C. u-Blox ZED-F9P Base Station (Optional)

3.1.1 Setup the Neousys Technology POC-451VTC

The RBS CP hardware was chosen to be the POC-451VTC device from Neousys Technology mainly due to having both TIA/EIA-422 and Digital I/O hardware interfaces. This facilitates the various type of ways that the RBS can be set up to receive the HRI preemption.

3.1.1.1 Connecting the Hardware

The RCVW basic capabilities are developed around the reception of the status of the HRI warning devices activations. This information is received via a Direct Current (DC) voltage level preemption signal directly from an HRI Controller, or by a digital signal coming from an Institute of Electrical and Electronics Engineers (IEEE) 1570 standard compliant device. Although IEEE 1570 is capable of providing a richer set of crossing information, which could be taken advantage of through the use of the CBS, the standard is not widely deployed. The RCVW RBS needs to be configured prior to deployment choosing one of the two signal reception methods..

When a preemption signal is used (from an HRI controller), a two-conductor DB-15 cable is required for connecting the pre-emption signals to the CP, which provides a 15-pin Digital I/O interface. The cable must contain one positive voltage wire and one negative voltage, or ground, wire. (See Figure 11)

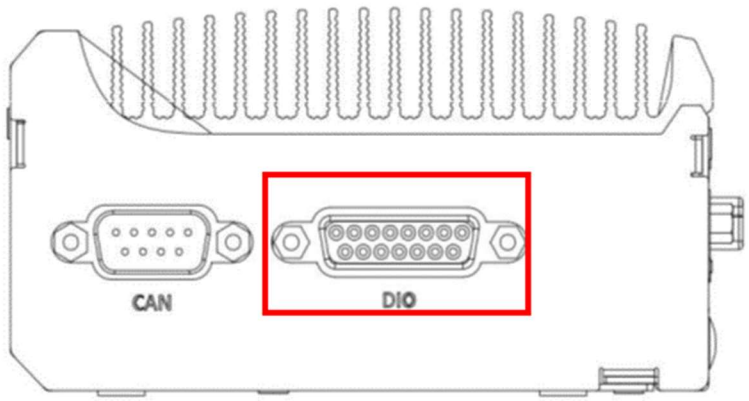


Figure 11: DIO Port of the Neousys Intel CPU

If an IEEE-1570 compliant device is used, the IEEE 1570 device includes a TIA/EIA-422 interface that connects to the COM1 port of the Neuosys Intel CPU (See Figure 12).

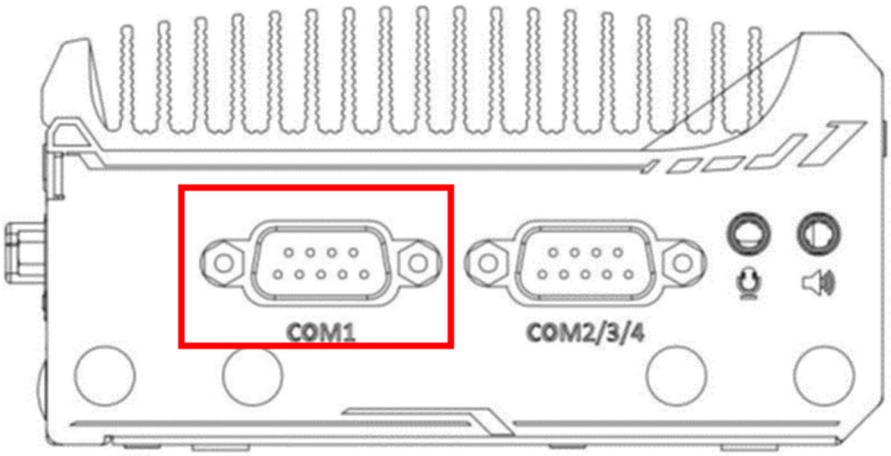


Figure 12. COM ports of the Neousys Intel CPU

Coordinate with the railroad to install the cable to the HRI Controller or IEEE 1570 device. Instructions on configuring the RBS for both methods is provided in Chapter 7 of this document.

3.1.1.2 Configuring the Basic Input/Output System (BIOS)

The CP's COM1 port supports RS-232 (full-duplex), RS-422 (full-duplex) and RS-485 (half-duplex) mode. However, the system is shipped with factory-default BIOS settings that are not compatible for the IEEE 1570 hookup to this COM port. Therefore, when the POC-451VTC is first booted, it is best to stop the boot process by pressing the **`F2`** key and enter the **`Advanced->Peripheral`** configuration menu. Select `RS-485`, the hit **`F10`** to **`Save and Exit`**.

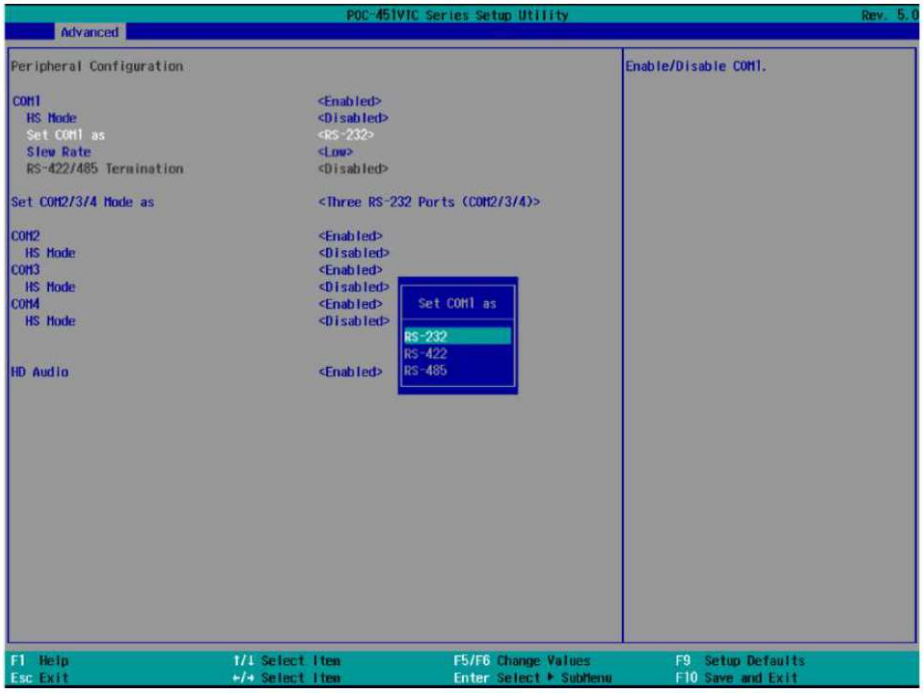


Figure 13: COM Port BIOS Configuration

3.1.1.3 Configuring the Kernel

Access to the POC-451VTC Digital I/O interface requires a special pre-built Operating System driver and library to access. These are all provided under the HRI Status plugin source code tree. The kernel module can be installed on the hardware by copying to the appropriate kernel module directory and loading it on boot:

```
$ sudo cp wdt_dio.ko /lib/modules/$(uname -r)/kernel/drivers/watchdog
$ sudo modprobe wdt_dio
$ echo wdt_dio | sudo tee -a /etc/modules
$ sudo reboot
$ lsmod | grep wdt_dio
```

There is a test program available to help check the status of the DIO pins

```
$ chmod +x test_dio
$ sudo ./test_dio
DIReadPort = 0xf
DIReadPort = 0xf
DIReadPort = 0xf
^C
```

3.1.2 Setup the RBS Application

The installed `rcvw-all/rbs/docker-compose.yaml` file specifies the services that make up the RBS deployment. Dependent services include the V2X Hub messaging broker, which is implemented on the RBS using Apache Kafka and its Apache Zookeeper dependency, both of which are defined in `rcvw-all/docker-compose.yaml` file. For convenience, the inter-communicating services all use the `network_mode` of **`host`**, which directly affects the Ethernet on the CP instead of an isolated virtual network managed by Docker.

The HRI Status plugin is set up to connect either to the COM1 port or the DIO interface to read in the current preemption state. This, along with some HRI specific information, is used to generate the SAE J2735 SPAT message which is passed through Apache Kafka to the other RBS plugins and directly to the optionally configured CBS Service Bus `J2735.SPAT` topic. The `rbs-hri` service defines the plugin operation with the highly customizable `rcvw-hri-config` Docker config. This service must be configured with `privileged` execution for the `root` user since it reads data from the serial COM or the DIO interface device files. Additional parameters can be used to set the correct lane identifier mapping.

The Map plugin is a simple application that forwards the pre-encoded SAE J2735 MAP message through the Apache Kafka core and to the CBS Service Bus `J2735.MAP` topic. The `rbs-map` service uses the `rcvw-map-config` config to specify unique VBS parameters to use in the algorithm.

The RTCM plugin is arguably the most complicated and configurable RCVW plugin. The `rbs-rtcm` service is responsible for receiving RTCM correction bytes and generating associated SAE J2735 RTCMcorrection messages, which are forwarded through the local Apache Kafka service in order for them to be sent to the radio, as well as to the CBS at the `J2735.RTCM` topic. Obtaining the RTCM bytes, however, can occur in one of three different ways. The associated `rcvw-rtcm-config` configuration is set up by default to receive messages from the CBS Service Bus `V2X.RTCM3` topic, which can be written to by the CBS RTCM Proxy service. However, there are customizable parameters in the configuration that allow for receiving directly from an Network Transport of RTCM via Internet Protocol (NTRIP) caster or from a local GNSS receiver acting as a base station.

The C-V2X Immediate Forward plugin, also known as the RSU 4.1 plugin, reads messages from the local Apache Kafka topics and forwards to the immediate forward port on the local RSU over the UDP protocol. As this plugin does not generate its own J2735 messages, it only sends the plugin status information to the CBS for monitoring, although this includes radio statistics that come from the RSU 4.1 standard MIB. This plugin is run from the `rbs-rsu` service using the `rcvw-rsu-config` configuration.

3.1.2.1 Configuring the Environment

Table 6 describes the available environment variables that can be used to configure the VBS. Each of these must be set in the environment file described in section 2.3.1 above. If the value is not set, then the default value is assumed, which may be empty. The additional required values from Table 3 above are also mostly used for the RBS configuration.

Table 6: RBS Environment File Configuration Checklist

Variable	Description	Default Value	Set?
RCVW_HRI_DESCRIPTION	A quick intersection description to use in the HRI SPaT message.	None	
RCVW_HRI_TRACKED_LANE	The MAP lane identifier in the intersection that denotes the tracked lane signal group.	`0`	
RCVW_HRI_VEHICLE_LANE	The MAP lane identifier in the intersection that denotes the vehicle ingress lane signal group.	`1`	
RCVW_CBS	A calculated environment variable for that depicts the connection context for the CBS Service Bus.	Based on provided CBS user and password	

RCVW_RSU	The IP address of the local RSU radio to broadcast J2735 messages from.	`local-rsu`
RCVW_RSU_IFM_PORT	The immediate forward port for the local RSU radio.	`1516`
RCVW_RSU_AUTHPROTO	The authentication protocol for an NTCIP (SNMP v3) connection to the local RSU radio.	`SHA`
RCVW_RSU_PRIVPROTO	The privacy protocol for an NTCIP (SNMP v3) connection to the local RSU radio.	`AES`
RCVW_IEEE_INPUT	The serial device file to use to read in the IEEE 1570 messages.	None[9]
RCVW_DIO_PIN	The pin from the local Digital I/O port to read for an active-low preemption signal.	`0`[10]
RCVW_GPSD_SOURCE	The GPS device file to configure as a local base station for RTCM.	`/dev/null`[11]
RCVW_HRI_LAT	The absolute latitude for the intersection in 10^7 degrees, if using a local base station for RTCM	None
RCVW_HRI_LON	The absolute longitude for the intersection in 10^7 degrees, if using a local base station for RTCM	None
RCVW_HRI_ALT	The absolute altitude for the intersection in cm above sea level, if using a local base station for RTCM	None

[9] This implies that the default preemption signal interface is the DIO on the Neousys POC-451VTC

[10] See section 7.1.1.1

[11] This implies that the default RTCM source is from the CBS and not from a local base station

RCVW_NTRIP_SOURCE	The NTRIP caster URL, e.g., `my.example.com:2101/mountpoint`	None
RCVW_RBS_MAP_DEBUG	Software debugging level for the MAP plugin	`DEBUG`
RCVW_RBS_HRI_DEBUG	Software debugging level for the HRI Status plugin	`DEBUG`
RCVW_RBS_RSUIFM_DEBUG	Software debugging level for the C-V2X Immediate Forward plugin	`DEBUG`
RCVW_RBS_RTCM_DEBUG	Software debugging level for the RTCM plugin	`DEBUG`

3.2 Roadside Unit Overview

3.2.1 Setup the Danlaw Technology RouteLink Roadside Unit

The RCVW application prototypes come with the Danlaw RouteLink RSU pre-configured to broadcast MAP, SPAT and RTCM. See the datasheet for more detailed product specifications.

The system comes pre-installed with a Configuration Tool that is convenient for the RSU setup. However, before the RSU Configuration Tool can be used, make sure the following three prerequisites are met:

- A Laptop/Tablet/Mobile with latest web browser (latest chrome is recommended).
- The laptop is configured with a static IPv4 and this IP address is set to the same network as the RSU.
- Supported software version for CV2X RSU is CV2X_v2.8.0 or greater.

The Configuration Tool can be accessed by connecting to port 3000 at the IPv4 address of the RSU using your web browser. For example, if the RSU is `10.30.100.30`, then enter:

```
http://10.30.100.30:3000
```

The login screen should appear:

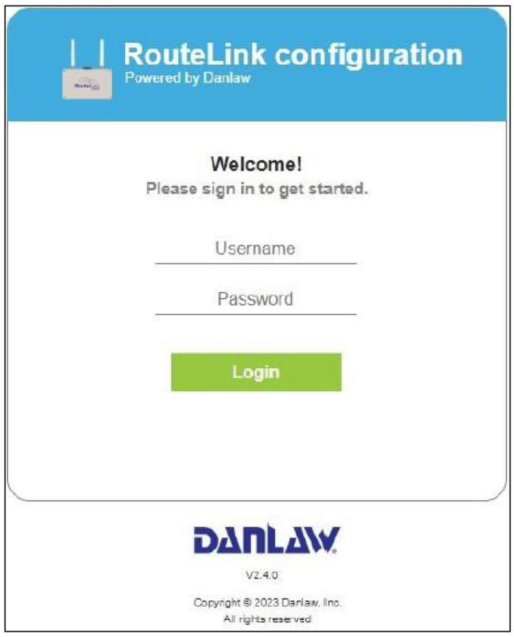


Figure 14: RouteLink Configuration Tool Login Screen

Enter `admin` for the **`Username`** and `danlaw` for the **`Password`** and then click ***`Login`***. The main dashboard appears:

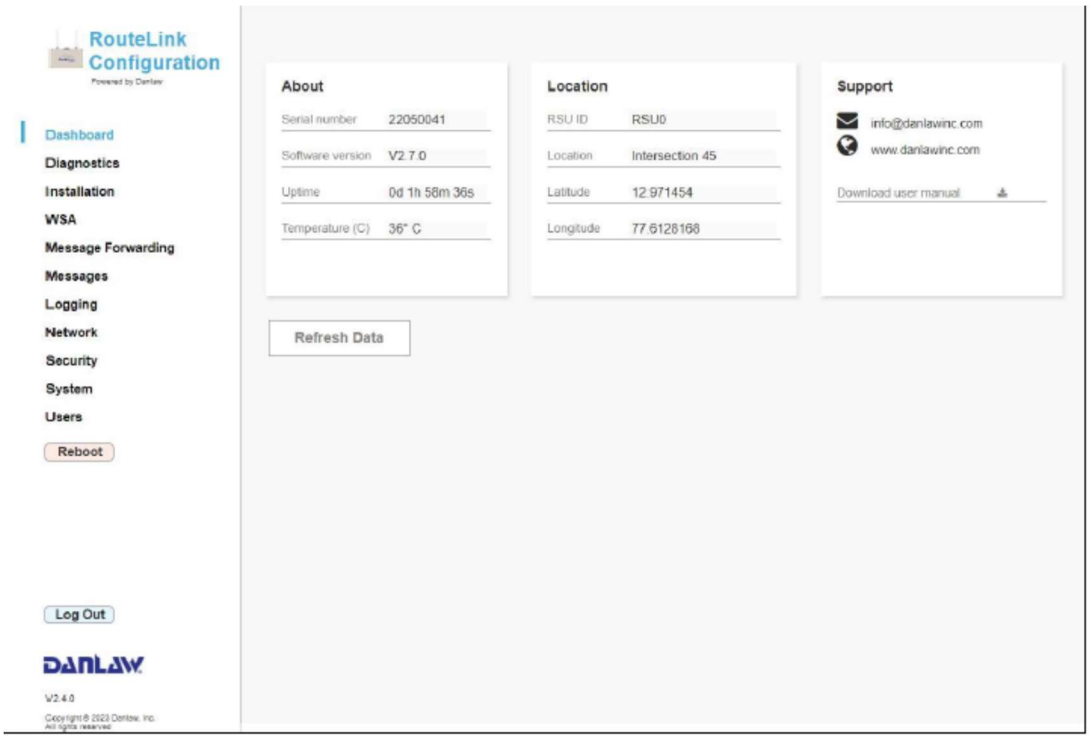


Figure 15: RouteLink Configuration Tool Main Dashboard

3.2.1.1 Connecting the Hardware

The RouteLink RSU is powered by Power over Ethernet (PoE) through the use of an injector that sends both power and data over the Ethernet connection. Install the PoE injector within the specified enclosure, in near proximity to the RBS CP. Connect the Ethernet cable to the `LAN` port of the PoE injector (see red rectangle in Figure 16) and connect the other end of the Ethernet cable to the same network as the RBS CP. At minimum, this can be done by connecting to the POC-451VTC Ethernet port (see blue rectangle in Figure 17).

Figure 16: PoE Injector

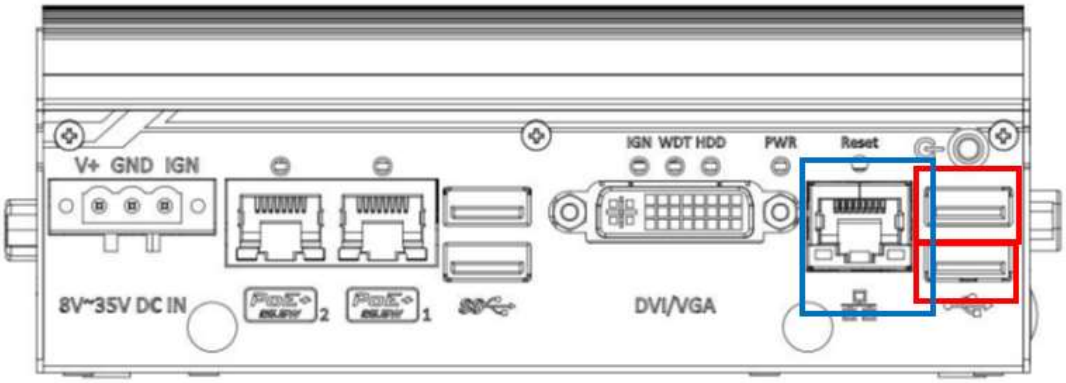


Figure 17: Neousys Computing Platform Universal Serial Bus (USB) and Ethernet Ports

Install the Danlaw RouteLink RSU radio to a fixed structure such as a pole and connect the C-V2X radio antennas and the GPS antenna to the unit (see Figure 18).

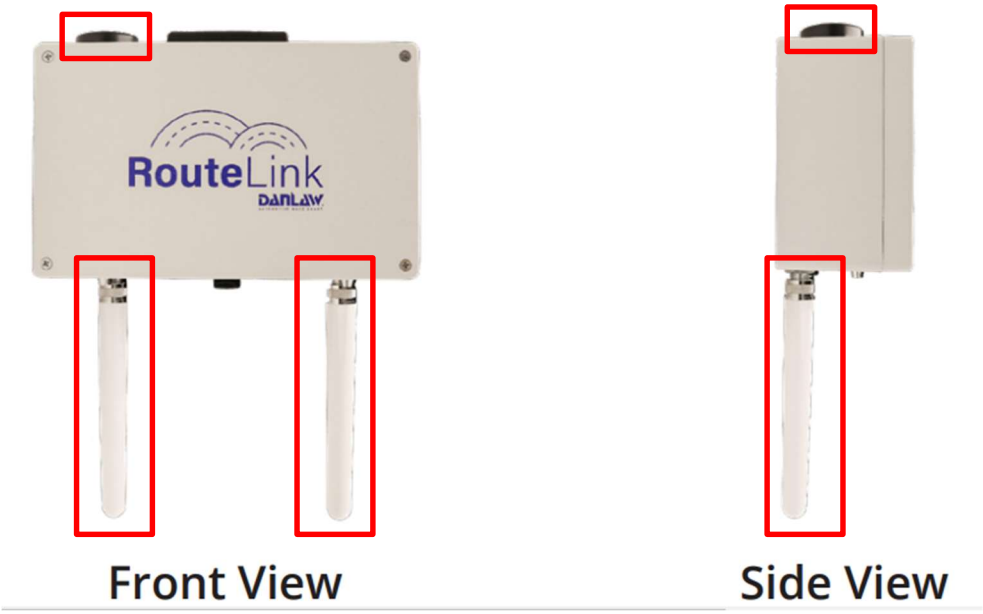


Figure 18: Danlaw Technology RouteLink C-V2X Radio and GPS Antennas

The VBS C-V2X radio/antenna must have a clear line of sight to the RBS C-V2X radio/antenna as it approaches the HRI. An optimal installation location for the RouteLink radio/antenna is 7 to 10 meters above ground. An alternative is to place a portable telescopic pole with a base alongside the HRI and mount the RBS C-V2X radio/antenna to it. Before finalizing the mounting of the RBS C-V2X radio/antenna, connect an Ethernet cable to its Ethernet port and connect the other end of the cable to the PoE injector's `POE` port (see yellow rectangle in Figure 16). Note that this Ethernet cable must be long enough to reach from the cabinet to the radio height.

3.2.1.2 Configuring the Network

To set a different IP address for the RSU, click on the ***`Network`*** menu in the Configuration Tool.

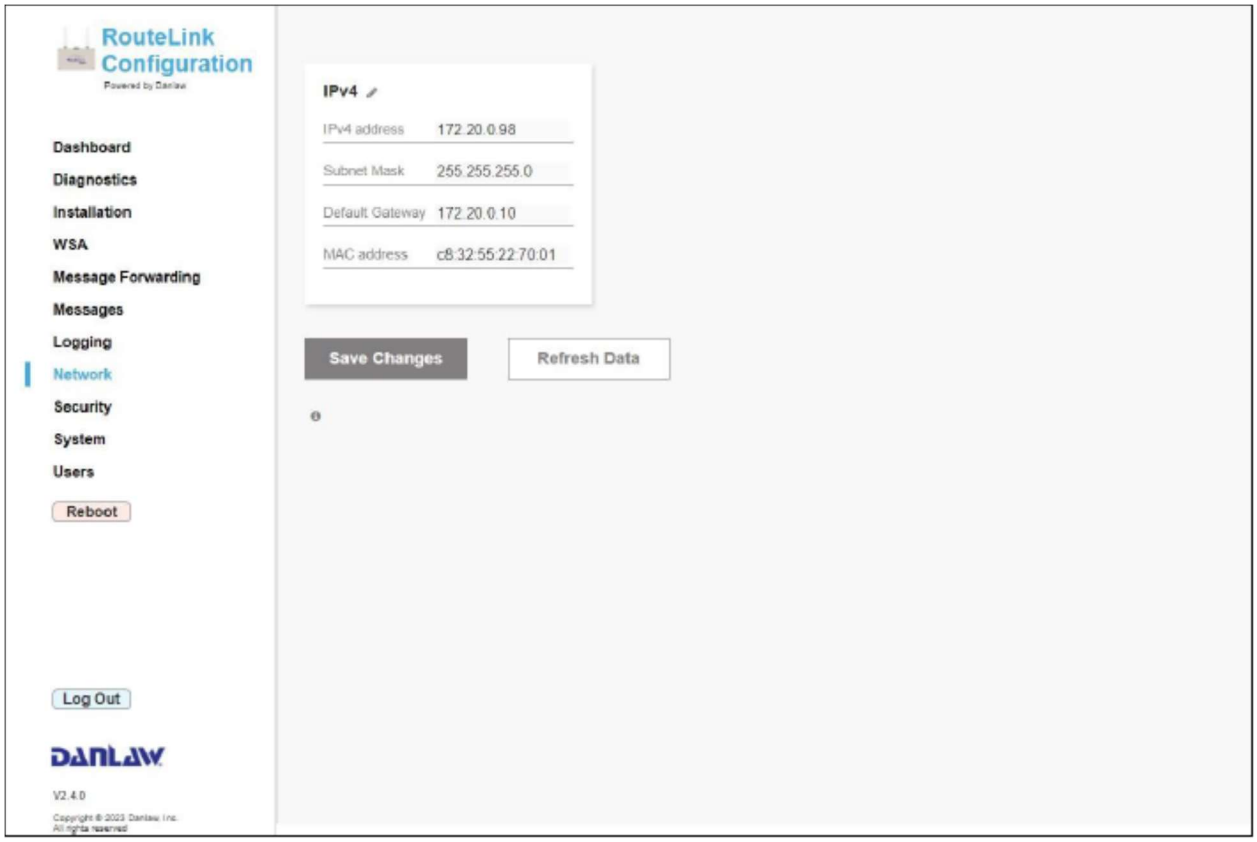


Figure 19: RouteLink Configuration Tool Network Screen

Click on the ***`Edit`*** (pen) icon in the **`IPv4`** window to enable editing. Modify the **`IPv4 address`**, **`Subnet Mask`** and **`Default Gateway`** and click on ***`Save Changes`***. Note that the **`MAC address`** field is read-only.

3.2.1.3 Configuring the Immediate Forward Messages

For this release of RCVW, the Immediate Forward messages table must be completed in order for the RCVW RSU plugin to be able to correctly use the radio. This can be updated by selecting the ***`Messages`*** menu.

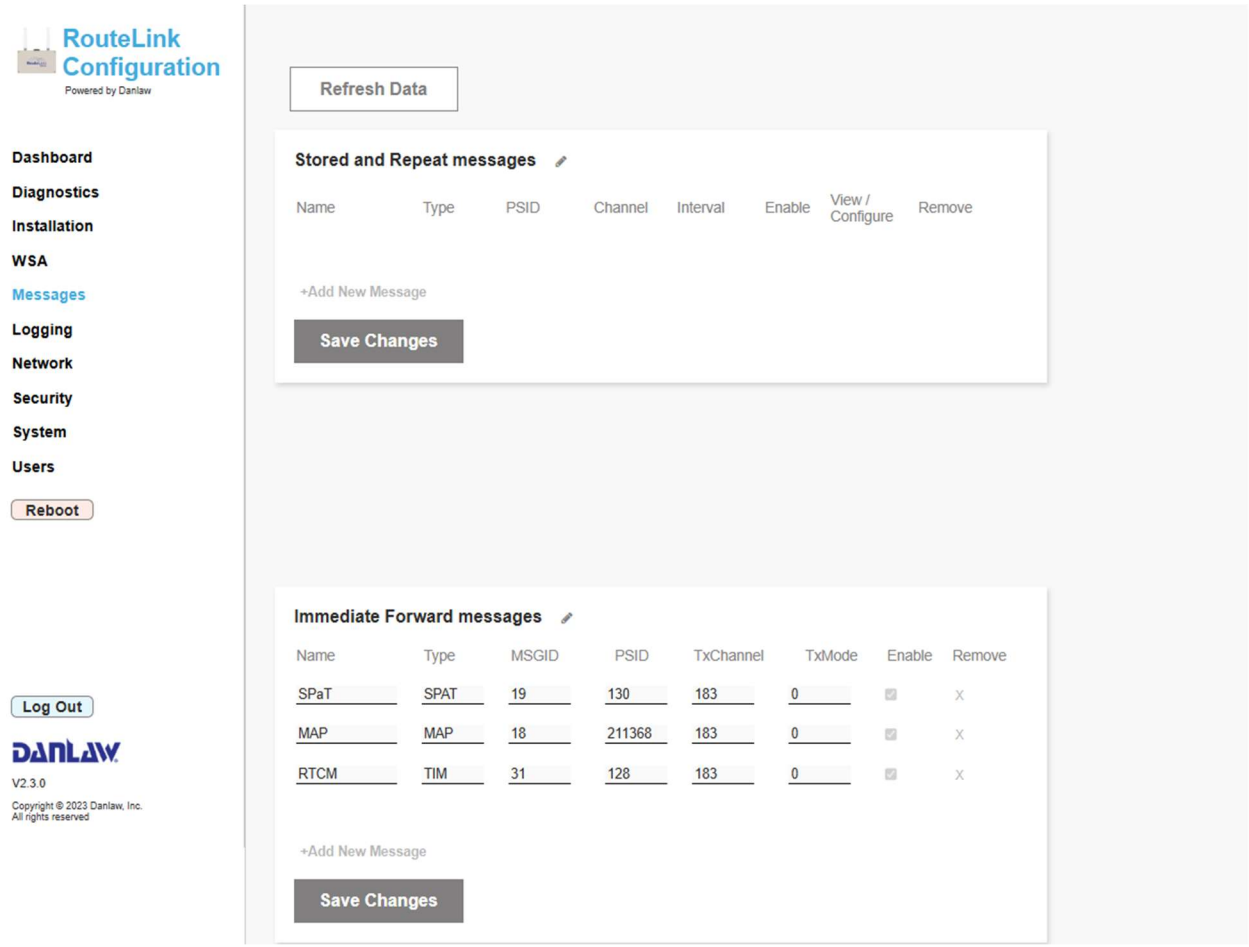


Figure 20: RouteLink Configuration Tool Messages Screen

Enable editing by clicking on the ***Edit*** (pen) icon next to the **Immediate Forward messages** and selecting ***Add New Message***. All fields in this window correspond to columns as defined in the `rsuDsrcForwardTable` object of the RSU-MIB. Please note that **PSID** field accepts integer values only. Therefore, the p-encoded[12] PSIDs should be converted to integers and then entered. For example, the p-encoded PSID for MAP is `E0-00-00-17`, and the integer equivalent is `2113687`, which is what should be entered for **PSID**. The rows depicted in Table 7 must be added to the table:

Table 7: RSU Immediate Forward Messages Checklist

Name	MSGID	PSID	TxChannel	TxMode	Enable	Set?
SPaT	19	130	183	0	Yes	

[12] As defined in IEEE 1609.12 (IEEE Standard for Wireless Access in Vehicular Environments (WAVE) - Identifier Allocations, 2016)

MAP	18	211368	183	0	Yes
RTCM	31	128	183	0	Yes

The entry for the RTCM messages must initially set the incorrect message identifier for a SAE J2735 Traveller Information Message (TIM) message because the DanLaw device only supports adding MAP or SPaT or TIM in the immediate forward table UI. This can then be corrected through an SNMP command run on the RBS. Note that the default Secure Shell (SSH) port for the RSU is `12345`, and the default password for the RSU root user is `Cv2xDePlS2`:

```
user@rcvw-rbs-N0602087:~$ ssh -p 12345 -l root local-rsu snmpset -v3 -l authPriv -a
SHA -A authandauth -x AES -X authandpriv -u rsuRwUser localhost .1.0.15628.4.1.99.0 i
2
root@local-rsu's password:
iso.0.15628.4.1.99.0 = INTEGER: 2
user@rcvw-rbs-N0602087:~$ ssh -p 12345 -l root local-rsu snmpset -v3 -l authPriv -a
SHA -A authandauth -x AES -X authandpriv -u rsuRwUser localhost .1.0.15628.4.1.5.1.3.3
i 28
root@local-rsu's password:
iso.0.15628.4.1.5.1.3.3 = INTEGER: 28
user@rcvw-rbs-N0602087:~$ ssh -p 12345 -l root local-rsu snmpset -v3 -l authPriv -a
SHA -A authandauth -x AES -X authandpriv -u rsuRwUser localhost .1.0.15628.4.1.99.0 i
4
root@local-rsu's password:
iso.0.15628.4.1.99.0 = INTEGER: 4
```

3.2.1.4 Configuring the Message Logging

If data transfer information is required, make sure that RSU (PCAP) Paquet Capture logging is enabled by clicking on the ***Logging*** menu.

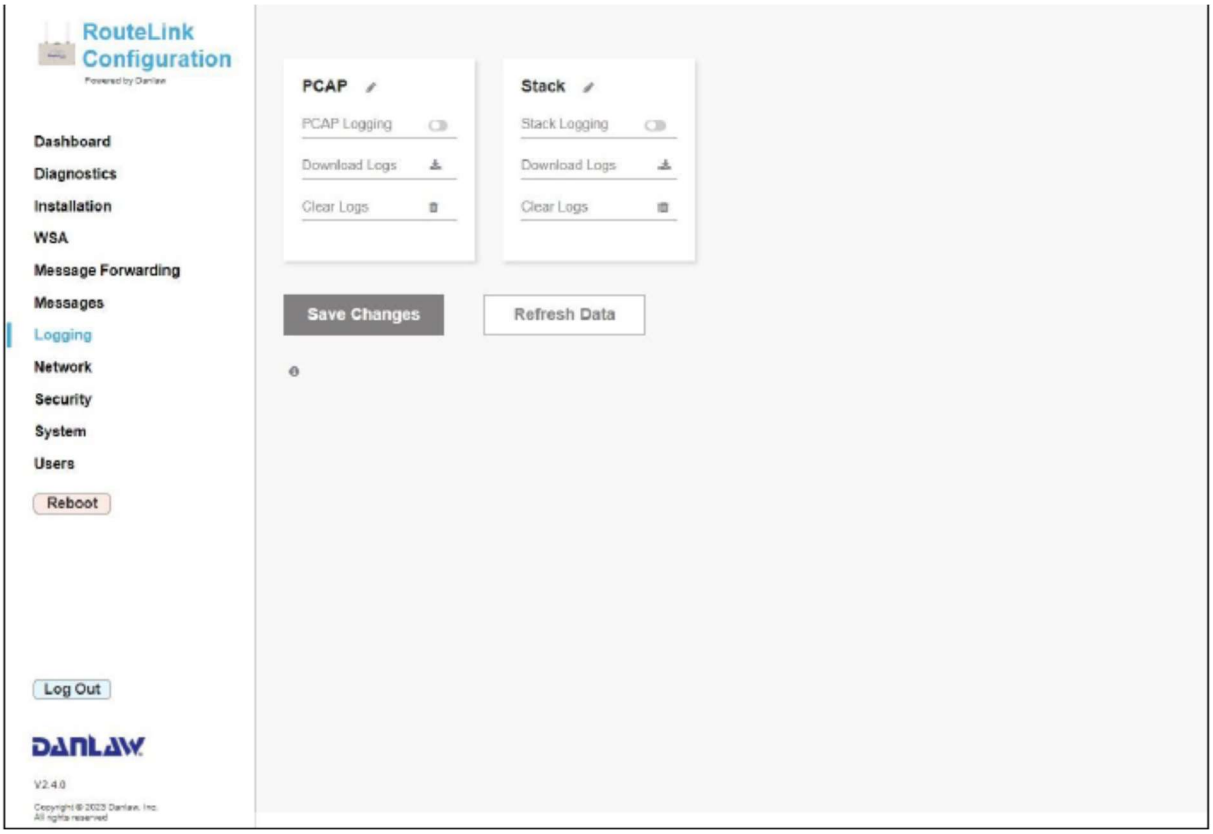


Figure 21: RouteLink Configuration Tool Logging Screen

Under the **PCAP** window, click the ***Edit*** (pen) icon to enable editing. Then, set **PCAP Logging** to on by clicking the toggle button and selecting ***Save Changes***.

3.2.1.5 Configuring the Certificates

Certificates for the RSU can be added under the ***Security*** menu.

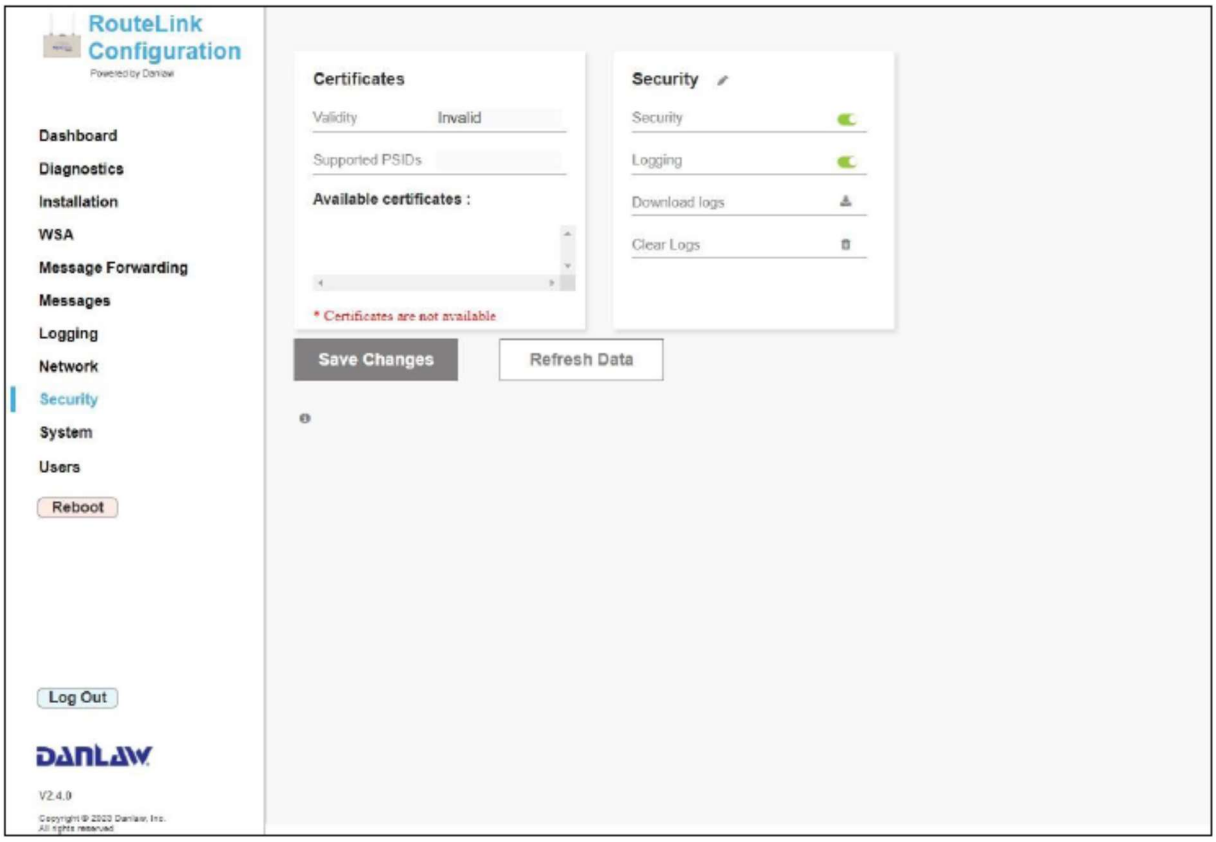


Figure 22: RouteLink Configuration Tool Security Screen

However, the RCVW prototypes do not secure the transmissions by default. Therefore, it is best to select the ***`Edit`*** (pen) icon under the **`Security`** window and disable the **`Security`** function. Click ***`Save Changes`*** to save.

3.3 Optional Base-Station Overview

The RTCM correction feed is set up by default to come from the CBS, which utilizes public sponsored base stations. However, these are currently limited in the CBS to a few State DOTs and there is no guarantee to the availability of a reference station nor for the accuracy of corrections. Therefore, in order to support a fully self-contained system, the RCVW prototypes provide the u-Blox ZED-F9P (not to be confused with the ZED-F9R attached to the VBS plate) as an optional GNSS receiver that can be used as a local base station. This section steps through the setup of a GNSS receiver as a local base station to support the RTCM corrections.

3.3.1 Setup the u-Blox ZED-F9P Base Station

The ZED-F9P is the main component of the C099-F9P application board that is included with the RCVW prototypes. However, any GNSS receiver that is capable of reporting RTCM version 3 messages for RTK resolution via GPSd may be used.

3.3.1.1 Connecting the Hardware

Install the C099-F9P application board within the RBS enclosure. Use the vendor provided USB cable and connect the micro-USB end to the C099-F9P (see blue rectangle on Figure 23) and the other end to one of the open USB ports on the RBS CP (see red rectangles on Figure 17).

Figure 23: u-Blox C099-F9P USB connection

The next step is to position and connect the GNSS base station antenna. The GNSS base station antenna needs to be positioned in a location where it will be able to have an obstruction free satellite view and stable enough to prevent excessive movement and vibrations. Camera tripods have been used in the past for this purpose. Ensure that when the antenna is mounted to the selected hardware, the antenna surface is not covered by tape or any other material as it will affect the satellite signal reception. The base station requires precise Longitude, Latitude, and Elevation of the antenna. Once the optimal location has been selected, use survey-grade equipment to acquire its longitude, latitude and elevation. Next, measure the height of the antenna in centimeters from its base to the ground and add this to the elevation recorded by the survey-grade system. Mark the location with a survey nail for future use. Once the antenna has been positioned, connect it to the SubMiniature version A (SMA) port of the u-Blox C099-F9P (see Figure 24). Note that the GNSS module is equipped with two SMA interfaces. Ensure that the GNSS antenna is connected to the interfaced marked ZED.

Figure 24: u-Blox C099-F9P antenna connection

3.3.1.2 Configuring the Environment

The RTCM plugin is pre-configured to use a GNSS base station if the `RCVW_GPSD_SOURCE` environment variable is set to the GPS device file. The environment must also include the variables `RCVW_HRI_LAT`, `RCVW_HRI_LON` and `RCVW_HRI_ALT` for the surveyed position information. These are programmed into the u-Box receiver by running a program:

```
$ docker compose run --rm cbs-basestation
```

After this is run, the connected ZED-F9P should now be producing RTCM.

```
$ docker compose up --force-recreate -d rbs-rtcm
$ docker compose exec rbs-rtcm cgps
```

Chapter 4. Vehicle Installation Standard Operating Procedures

4.1 Computing Platform Overview

As denoted in Figure 25, the VBS software architecture includes multiple components running on the computing platform, including:

- Message Receiver plugin
- Location plugin
- Differential GNSS plugin
- RCVW plugin
- DVI Web App

These software components operate as one or more Docker *services*, each running inside a separate *container*. The Location plugin was originally thought to be a separate service, as was the case in previous iterations of RCVW. However, the ultimate design is that the Location plugin functionality is built-in to the RCVW service as a TMX® messaging broker. See the RCVW Software Application Programming Interface (API) Description (Baumgardner G. M., 2025) for more information.

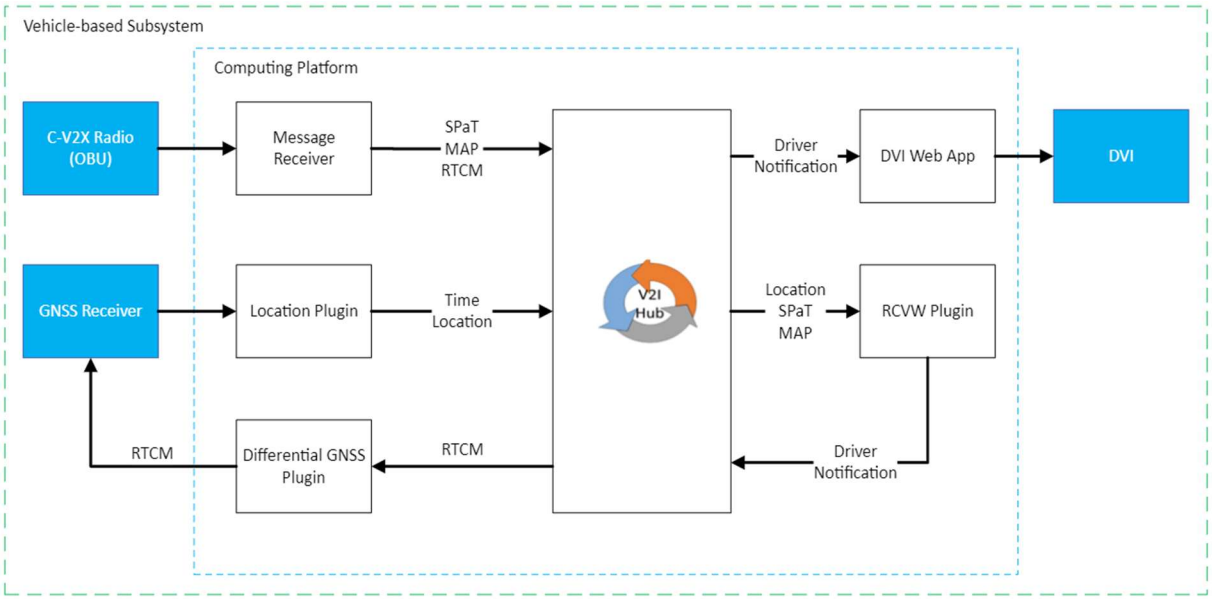


Figure 25: VBS Software Design[13]

[13] Taken directly from the RCVW Architecture and Design document (Sanchez-Badillo & Baumgardner, 2024)

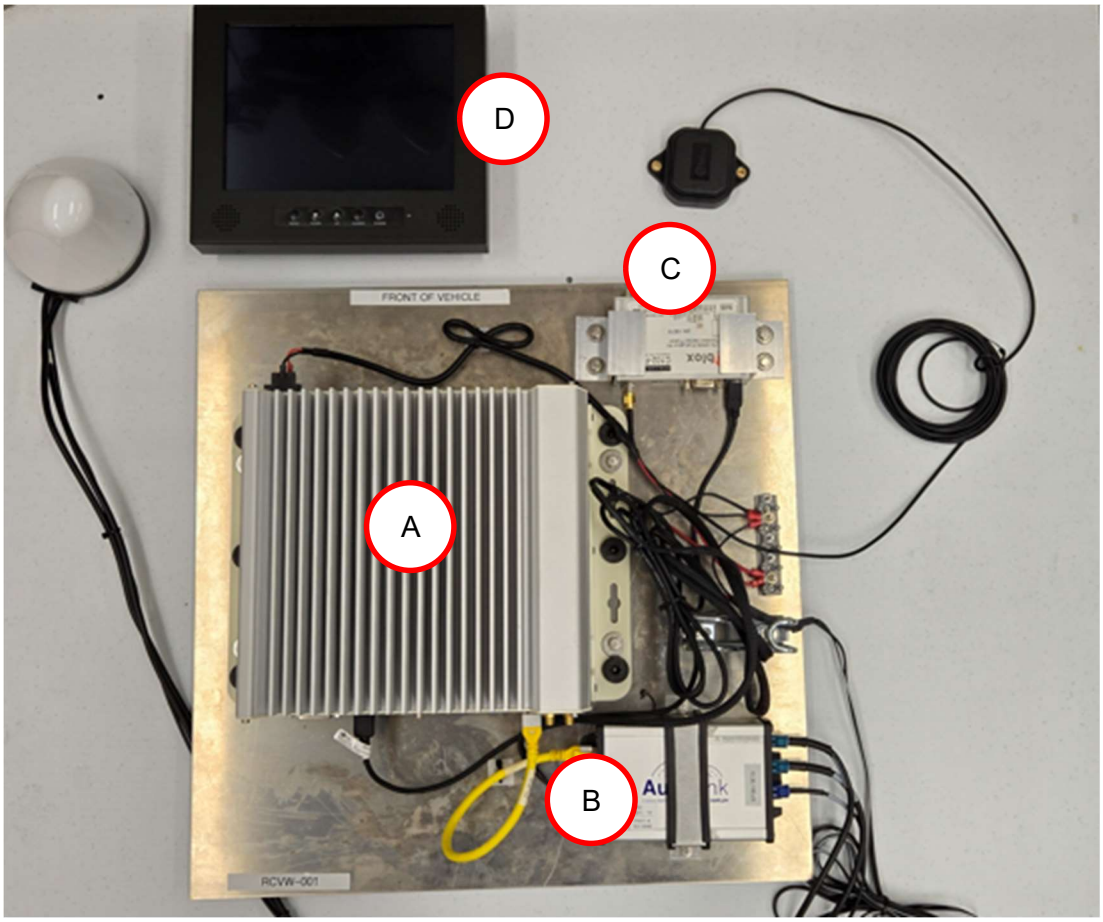


Figure 26: VBS Prototype Device-Mounting Plate

The VBS prototype (Figure 26) is a pre-mounted plate consisting of the following equipment:

A. Neousys Technology Nuvo-7200VTC Computing Platform
B. DanLaw Technology AutoLink Aftermarket Safety Device
C. u-Blox ZED-F9R GNSS Receiver
D. Tru-Vu HDMI Monitor for DVI

4.1.1 Setup the Neousys Technology Nuvo-7200VTC

The VBS computing platform contains a number of interfaces, many of which, for example CAN, are not used in the supplied prototypes. The device is bolted to the VBS prototype plate using custom mounting brackets. Additionally, an external terminal block is mounted to the plate in order to connect to an auxiliary 12V power adapter inside the vehicle.

4.1.1.1 Configuring for Ignition Power

The Nuvo-7200VTC is meant to operate using the DC power supplied by a vehicle. Therefore, the proper setup of this device is to turn off and on with the ignition. This is configurable through

the included module (see the blue highlighted section in Figure 27), which is accessible from the bottom of the device.

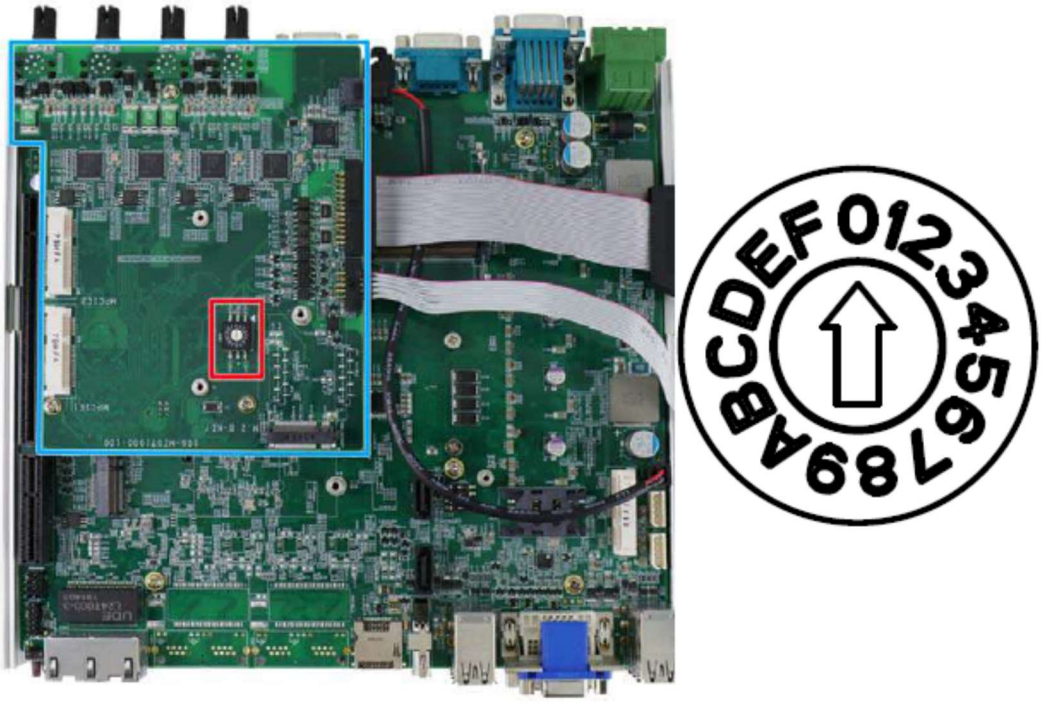


Figure 27: Nuvo-7200VTC Rotary Ignition Switch on the MezIO™ Module

The ignition power control module offers the following capabilities:

- **Low battery detection**: The ignition power control module continuously monitors the voltage of DC input when the system is operational. If input voltage is less than 9V (for 12VDC input) or less than 18V (for 24VDC input) over a 60-second duration, it will shut down the system automatically.
- **Guarded power-on/ power-off delay duration**: If ignition signal goes inactive during the power-on delay duration, the ignition power control module will cancel the power-on delay process and go back to idle status. Likewise, if ignition signal goes active during the power-off delay duration, the ignition power control module will cancel the power-off delay process and keep the system running.
- **System hard-off**: In some cases, system may fail to shutdown via a soft-off operation due to system/application halts. The ignition power control module offers a mechanism called “hard-off” to handle this unexpected condition. By detecting the system status, it can determine whether the system is shutting down normally. If not, the ignition power control module will force cut-off the system power 10 minutes after the power-off delay duration.
- **Smart off-delay**: The ignition power control module offers two modes (mode 6 & mode 7) which have very long power-off delay duration for applications require additional off-line time to process after the vehicle has stopped. In these two modes, the ignition power control module will automatically detect the system status during the power-off

delay duration. If the system has shutdown (by the application software) prior to power-off delay expiring, it will cut off the system power immediately to prevent further battery consumption.

The rotary switch (see red rectangle in Figure 27) is used to configure the operation mode. The system offers 16 (`0-15`) possible operation modes which are some combination of the different power-on/power-off delay configurations. The ignition control module is also BIOS-configurable. When the rotary switch is set to mode `15` (0xF), the ignition power control is executed according to the parameters configured in the BIOS setup menu, which allows a richer combination of power-on/power-off delay and more detailed control parameters. The VBS prototypes come pre-set to Mode 1, which is AT mode without power-on and power-off delay. The system automatically turns on when DC power is applied. A try mechanism is designed to repeat the power-on system if the system fails to boot up.

4.1.1.2 Connecting the Hardware

The power to the CP includes a 3-pin connector that is pre-wired to the terminal block. For deployment, the Ethernet cable should be connected from the right-most LAN port (see the red highlighted port in Figure 28) directly to the Ethernet port on the OBU. However, in order to configure the CP correctly, this port may be used temporarily for connectivity. The second LAN port (see the blue highlighted port in Figure 28) may also be used for configuration, but this requires additional configuration to set the IPv4 address of that port.

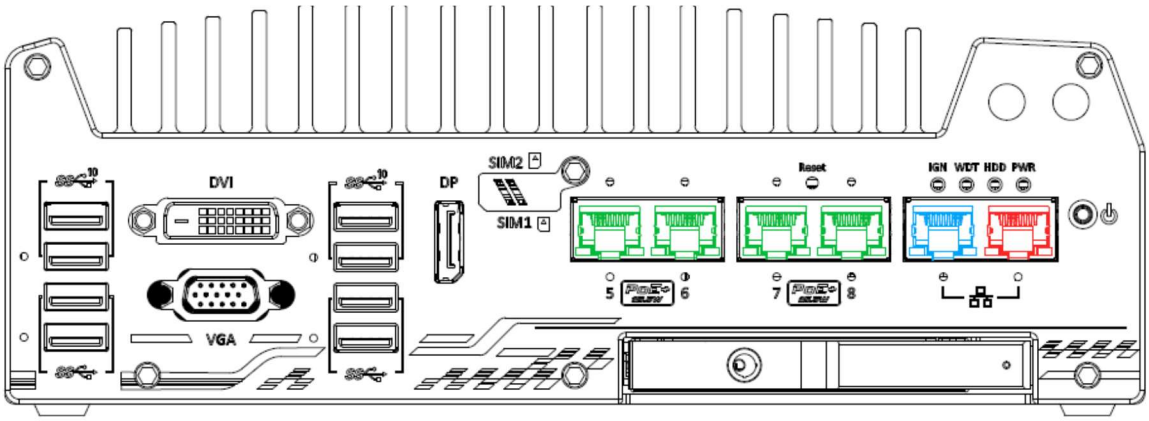


Figure 28: Nuvo-7200VTC Ethernet and PoE ports

4.1.2 Setup the VBS Application

The installed `rcvw-all/vbs/docker-compose.yaml` file specifies the services that make up the VBS deployment. Dependent services include the V2X Hub messaging broker, which is implemented on the VBS using Apache Kafka and its Apache Zookeeper dependency, both of which are defined in `rcvw-all/docker-compose.yaml` file. Additionally, the `vps-dgps` service is a dependency used to operate the GPSd server that ultimately creates Position, Velocity and Time (PVT) messages from the GNSS receiver. See section 4.3. For convenience, the inter-communicating services all use the `network_mode` of **`host`**, which directly affects the Ethernet on the CP instead of an isolated virtual network managed by Docker.

The Differential GNSS plugin specifically receives J2735 RTCM correction messages from Apache Kafka and applies them to the GNSS receiver in order to augment the satellite positioning to achieve Real-Time Kinematic (RTK) resolution. The `vbs-dgps` service defines the plugin operation with config `rcvw-dgps-config`. This service must be configured with `privileged` execution for the `root` user since it writes the data to the serial device file on the CP.

The RCVW plugin is designed to process many messages per second coming from Apache Kafka and apply the RCVW algorithm in order to determine faults and alerts that are passed to the driver via the DVI Web App. The `vbs-rcvw` service uses the highly customizable `rcvw-rcvw-config` config to specify unique VBS parameters to use in the algorithm.

4.1.2.1 Configuring the Environment

Table 8 describes the available environment variables that can be used to configure the VBS. Each of these must be set in the environment file described in section 2.3.1 above. If the value is not set, then the default value is assumed.

Table 8: RSU Immediate Forward Messages Checklist

Variable	Description	Default Value	Set?
RCVW_GPS	The device file for the local GNS receiver.	`/dev/ttyGPS`	
RCVW_OBU_FWD_PORT	The port to receive J2735 messages on.	`26789`	
RCVW_VBS_VEHICLE	An identifier for this vehicle to use in log messages. Note that the prototypes set this value to the serial number of the CP.	`0`	
RCVW_VBS_TYPE	The type of vehicle that is being used, either (`1`) for a car, (`2`) for a light truck, (`3`) for a heavy truck	`1`	
RCVW_VBS_LENGTH	The total length of the vehicle (in **meters**).	`4.51104`[14]	

[14] Taken from the reported average length of a mid-size sedan (`14.8 ft`)

RCVW_VBS_ANTENNA_HEIGHT	The position of the antenna (in **meters**) from the road surface.	`1.70688`[15]
RCVW_VBS_ANTENNA_X	The position of the antenna (in **meters**) from the driver side of the vehicle.	`0.9525`[16]
RCVW_VBS_ANTENNA_Y	The position of the antenna (in **meters**) from the front of the vehicle.	`2.25552`[17]
RCVW_VBS_DGPS_DEBUG	Software debugging level for the Differential GPS plugin	`DEBUG`
RCVW_VBS_RECV_DEBUG	Software debugging level for the Message Receiver plugin	`DEBUG`
RCVW_VBS_RCVW_DEBUG	Software debugging level for the RCVW plugin	`DEBUG`
RCVW_ERR_GNSS	The estimated error (in **meters**) produced by the GNSS device.	`1.5`[18]
RCVW_ERR_RT	The estimated reaction time (in **seconds**) for a driver.	`2.5`[19]
RCVW_FAULT_HRI_DISTANCE	The distance (in **meters**) from a known HRI in which missing critical messages will result in a fault.	`1000.0`

[15] Taken from the reported average height of a mid-size sedan (`5.6 ft`)

[16] Taken as the midpoint from the reported average width of a mid-size sedan (`6.25 ft`)

[17] Taken as the midpoint from the reported average length of a mid-size sedan (`14.8 ft`)

[18] Taken from the estimated worst case position error for RTK resolution (`1.5 m`)

[1919] Taken from (AASHTO, 2018)

RCVW_FAULT_PVT	Flag to determine if the frequency of position message should be verified.	`true`
RCVW_FAULT_PVT_FREQ	The minimum number of position messages (**per second**) expected.	`4`
RCVW_FAULT_PVT_COUNT	The number of sample position messages to verify the frequency.	`30`
RCVW_FAULT_RTK	Flag to determine if RTK position fix should be verified.	`true`
RCVW_LATENCY_APP	The estimated application latency time (in **seconds**).	`0.085`
RCVW_LATENCY_COMM	The estimated communications latency time (in **seconds**)	`0.3`
RCVW_LATENCY_MSG	The latency time (in **milliseconds**) before incoming messages should be considered expired.	`500`
RCVW_KNOWN_HRIS	A comma separated list of known HRIs[20]. This list is added to the pre-configured of Battelle demonstration sites.	None

There are a few other parameters set in the RCVW plugin manifest file that are generally fixed for nominal operations. If any require update, it must manually be done in Docker `compose` file.

After the environment file is updated, the VBS application plugin containers must be recreated and the processes restarted. Note that these commands assume the `user` home directory contains the linked Docker `compose` project.

```
$ cd ~user
$ docker compose up -d --force-recreate --no-deps vbs-dgps vbs-recv vbs-rcvw
```

[20] Should be in JSON form such as: `{"Latitide": ##.###, "Longitude": ##.###, "HRIName": "Some Name"}`

4.2 On-Board Unit Overview

On February 22, 2023, Battelle release Request for Proposal (RFP) No. 23-0569 on behalf of FRA on a pricing order for four C-V2X capable OBUs with commercial procurement under government contract terms and conditions. The request included a three to five-year maintenance plan to ensure the prototype hardware is under support beyond the scope of the project. The critical technical requirement for the OBU was that any J2735 received messages could be forwarded via User Datagram Protocol (UDP) to the connected VBS CP.

Of the received bids on this RFP, the Danlaw Technology AutoLink OBU was selected. However, the architecture of RCVW ensures that other capable radios may be substituted for additional prototypes.

4.2.1 Setup Danlaw Technology AutoLink Aftermarket Safety Device (ASD)

The Danlaw AutoLink Aftermarket Safety Device (ASD) is an aftermarket C-V2X OBU that natively supports a set built-in V2X applications, as well as the key feature of UDP message forwarding. Further product specifications can be found within the datasheet.

Figure 29: Danlaw Technology AutoLink C-V2X ASD

4.2.1.1 Connecting the Hardware

The Danlaw OBU comes with a mounting bracket that is already screwed into the VBS prototype plate. See Figure 30.

Figure 30: OBU Mounting Bracket

The VBS prototype kit comes a combined GNSS/C-V2X radio antenna in the form of a mag-mount fin. See Figure 31. This antenna must be installed in a position where there are no obstructions all the way around and a clear view to the sky as much as possible. For ideal performance, the antenna should be mounted on a flat, metal service. The position of the radio antennas should also not interfere with the separate GNSS receiver antenna for the u-Blox device. See Figure 32 for an example positioning. Connect the GPS wire of the antenna to the GPS port on the device (see the red rectangle in Figure 29) and the two C-V2X wires from the antenna to the two C-V2X ports (labeled 1 and 2) on the ASD (see the blue rectangles in Figure 29). It does not matter which wire goes into which port.

Figure 31: GNSS/C-V2X Radio Antenna

Figure 32: Example vehicle antenna positioning

4.2.1.2 Configuring the Network

In order to enable the forwarding of the IEEE 1609 Wireless Access in Vehicular Environments (WAVE) Short Message (WSM) contents to the VBS CP, first login to the OBU as root via Secure Shell (SSH). Note that the AutoLink device comes with a built-in IPv4 address set to `192.168.XX.YY` in which ***XX.YY*** are the last **4 digits** of the serial number written on the AutoLink box, and uses the unique SSH port of `12345`. For example, use the following commands to login the AutoLink device whose serial number is `23160009` using the default password:

```
$ ssh -p 12345 root@192.168.0.9
root@192.168.100.23's password: danlaw
```

The IPv4 address is set in the /config/environment file. Therefore, if the address of the OBU is supposed to be 192.168.100.27, then this can be modified by the following commands:

```
root@danlawv2x-imx6dl:~# cp /config/environment /config/environment.bak
root@danlawv2x-imx6dl:~# sed -i -e 's/^UE_IPADDRESS=.*/UE_IPADDRESS=192.168.100.27/'
/config/environment
root@danlawv2x-imx6dl:~# reboot
```

Make sure the device network is set up correctly so it can communicate the RSU, which in this example is `192.168.100.24`.

```
root@danlawv2x-imx6dl:~# ping -c 5 192.168.100.24
PING 10.30.100.24 (10.30.100.24): 56 data bytes
64 bytes from 192.168.100.24: seq=0 ttl=64 time=0.370 ms
```

```
64 bytes from 192.168.100.24: seq=1 ttl=64 time=0.336 ms
64 bytes from 192.168.100.24: seq=2 ttl=64 time=0.336 ms
64 bytes from 192.168.100.24: seq=3 ttl=64 time=0.350 ms
64 bytes from 192.168.100.24: seq=4 ttl=64 time=0.329 ms

--- 10.30.100.24 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
```

4.2.1.3 Configuring the Messaging

The OBU configuration is store in a single file on the device. Back up this configuration file and then add the appropriate lines as described below.

```
root@danlawv2x-imx6dl:~# cp /config/danlaw.conf /config/danlaw.conf.bak
root@danlawv2x-imx6dl:~# cat /config/danlaw.conf
###Vehicle classifier,
### Provide Integer value as per DE_BasicVehicleClass definition of J2735 v2016
vehicle_class = 10

###Channel access hack for some RSUs
wsa_channel_access_hack = 1

###Enable WSA Selector
enable_wsa_forwarder = 1

###Enable diagnostic lights on the board
diagnostics_enabled = 1

###Below are the configuration for the Security - Comment the below line to turn off
Security
onboardsecurity_configuration =

### Provide Integer value as per DE_BasicVehicleRole definition in J2735
vehicle_role = 0

# wsm forwarding, both received and sent for testing
wsm_rx_udp_dest_port = 26789
wsm_rx_udp_dest_ip = 192.168.100.24
```

```
#wsm_tx_udp_dest_port =
#wsm_tx_udp_dest_ip =

#####################
#######################################################
############ RECOMMENDED TO NOT MODIFY ANYTHING BELOW THIS LINE ############
#########################################################################

###List of PSIDs supported w.r.t Customer Requirements###
# PSID for Service 1
#PVD Messages
psid_service_used = 84

# PSID for Service 2
# MAP messages
psid_service_used = 2113687

# PSID for Service 3
# SPAT message
psid_service_used = 130

# PSID for Service 4
psid_service_used = 204096

# PSID for Service 5
#psid_service_used =

###Enable BSM congestion control
enable_congestion_control = 1

###VAD configuration file
#vehicle_vad_conf_file = vad_vehicle.conf
vehicle_vad_conf_file = /config/ModelDeploymentRemovable.conf

###Below are for the PVD setup
#Probe data interval in msec
vehicle_pvd_interval = 1000
```

```
###Enable PVD on probe message service available
vehicle_pvd_on_probe_service = 1

###Use u-blox GNSS interface
#use_ublox_gnss = 1

###Below configurations are for the Radio Configurations
#radio_0_access_type= ALTERNATING
#radio_1_access_type = CONTINUOUS
radio_0_default_channel=183
radio_1_default_channel=183
#radio_0_service_channel=180
radio_bsmuper_txchannel = 183
radio_eva_txchannel = 183
radio_srmuper_txchannel = 183

###Below configuration is for the setting the BSM Tx Pwr - TxPower to use for BSM
UPERs, 0 uses device default
radio_bsmuper_txpower = 20
radio_srmuper_txpower = 20

###The location of application configuration files
#config_dir

###Communication Logs details
log_pcap = false

#Transmit only if New GPS Data is available
vehicle_tx_new_gps_only = 1

###Use J2945 minimum transmission requirements - set to 1 and device will not transmit
any BSM when no GPS fix
vehicle_use_minimum_tx_info = 1

#For getting last known heading value
vehicle_heading_persist_file = /opt/danlaw/bin/vehicle.heading
```

```
#For getting last known path history data
path_history_storage_location = /opt/danlaw/bin/pathhistorydata.txt

#For path history and Path Prediction logging
#path_history_log_directory = /v2x_logs
#path_prediction_log_directory = /v2x_logs

#For path history stored previous location distance
path_history_reset_distance = 20.0

### udp forwarding details
#udp_forward_address =
#udp_forward_port =
#udp_rx_forward_port =
##########################################################################
```

Add the highlighted section to set the parameters that are used for WSM forwarding. The combination of **`wsm_rx_udp_dest_ip`** and **`wsm_rx_udp_dest_port`** is used to determine the host machine and port number to send incoming WSM messages via UDP. These include any J2735 message coming from the RSU. Likewise, the combination of **`wsm_tx_udp_dest_ip`** and **`wsm_tx_udp_dest_port`** is used to determine the host machine and port number to sending outgoing WSM messages via UDP. These include the Basic Safety Message (BSM) being generated by the OBU. The RCVW requires the use of the Rx forwarding from the AutoLink, but no the Tx forwarding. This file could be updated using a text editor like vim, or using the following command, which again assumes the VBS is at IP address `192.168.100.24` and the **`RCVW_OBU_FWD_PORT`** is set to the default `26789`. After a reboot, the OBU should be configured.

```
root@danlawv2x-imx6dl:~# reboot
```

4.3 Global Navigation Satellite System Overview

The GNSS receiver is a key component for the VBS as it needs to provide a highly accurate assessment of the current PVT of the vehicle at any given moment. Because the accuracy of the position estimate is difficult to quantify and often altogether indeterminate, the RCVW VBS application uses a qualitative measurement based on the quality of the signal fix. By default, the RCVW requires PVT measurements with a Real-time Kinematics (RTK) resolution, either floating point or fixed. This is shown to be accurate to 1.5 m for 95% of measurements, based on performance testing done in previous iterations of RCVW (Baumgardner G. M., 2020). Because RTK is designed to correct errors in the carrier wave signals coming from the satellite, instead of the position measurements themselves, the augmented signal is nearly identical to the one sent from the satellite, resulting in highly precise estimates due to the lack of signal

error. Therefore, the VBS by default assumes that an RTK resolution is close enough to reality to produce alerts that are neither early nor late.

Table 9 describes the minimum capabilities required for the VBS GNSS receiver.

Table 9: VBS GNSS Minimum Hardware Specification Checklist

Component	Measure	Minimum	Applies To	Check?
Speed	Rate	10 Hz	VBS	
	Baud	115200 bps		
Satellite Support	GPS	Yes	VBS	
	GLONASS	Yes		
Fix Quality	Max Resolution	Fixed RTK (3)	VBS	
Accuracy	Horizontal Position w/RTK	1 m	VBS	
	Vertical Position w/RTK	1 m		
	Velocity	0.5 mph		

The prototype systems come equipped with the ZED-F9R receiver from u-Blox, which is a economical professional grade receiver that contains RTK augmentation fused with data from inertial measurement unit (IMU) sensors and dead-reckoning (DR) calculations, allowing for a very reliable positioning system for a small cost. Refer to the device datasheet for more information.

4.3.1 Setup u-Blox ZED-F9R

The prototype VBS plates come with the C102-F9R application board from u-Blox since it makes mounting and connecting easy. The RF antenna must support multi-band GNSS signals in order to achieve RTK resolutions.

Figure 33: u-Blox C102-F9R

4.3.1.1 Connecting the Hardware

First the VBS device-mounting plate must be mounted on a flat surface inside the vehicle. Possible locations could be the back seat, back seat floor or the vehicle's trunk. The terminal block needs to be within reach of an auxiliary 12V power outlet to receive power.

Once a location has been selected for the device-mounting plate, the GNSS antenna needs to be mounted. Place the GNSS antenna (see Figure 34) on the roof of the vehicle. Ensure that the antenna is located in the center of the vehicle as much as possible and away from any obstruction. The antenna needs to have a clear hemispheric view of the sky. The antenna provided with the RCVW prototype has a magnetic base which eliminates the need of additional mounting hardware. Once the antenna is mounted, X and Y coordinates for the location of the antenna need to be measured and recorded (these values need to be recorded in meters since the RCVW system uses the metric system for its calculations). Assume that the left front corner

of the vehicle is coordinate (0,0), positive values for the X coordinates are left to right and positive values for Y coordinates are front to back of the vehicle. See Figure 35

Figure 34: u-Blox mult-channel GNSS antenna

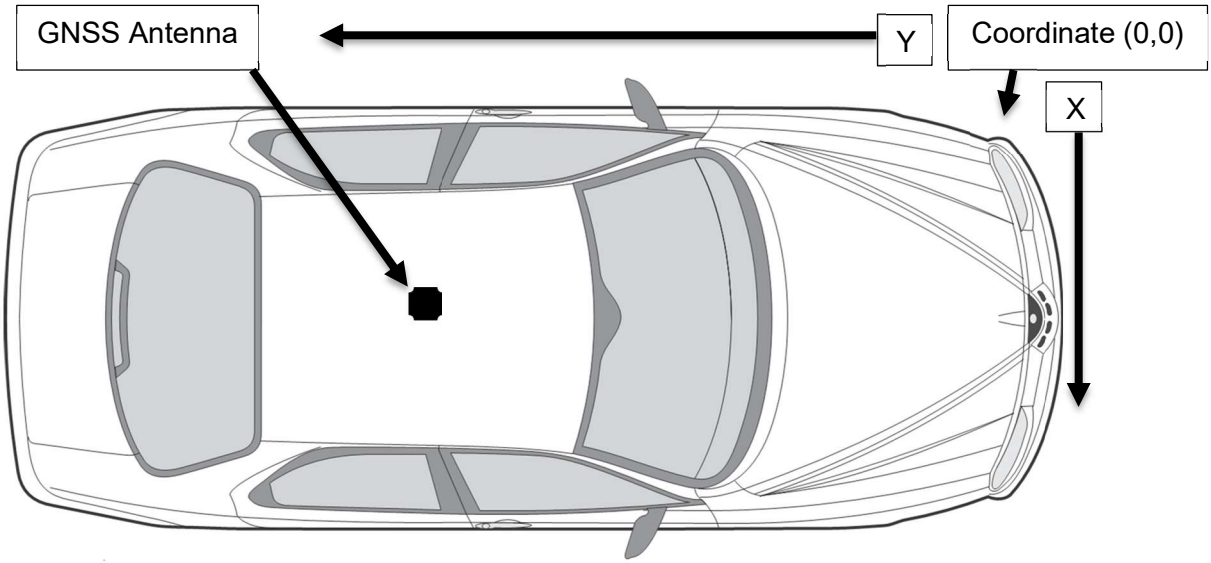


Figure 35: GNSS antenna location

The height of the antenna, in meters, with reference to the ground needs to be recorded. Route the antenna cable through a vehicle window and connect it to the SMA connector labeled **`RF-IN`** of the GNSS Module (See the red rectangle in Figure 33). The length of the vehicle in meters from front bumper to rear bumper should be recorded as well.

4.3.2 Setup the VBS Application

The GNSS receiver is managed by the GPSd server process, which also is a server that provides interpreted PVT data to the client. The RCVW plugin application is pre-configured to read location information from the default GPSd port (`2947`) on the `localhost`, meaning that the GNSS receiver must be directly connected to an available USB port on the VBS CP.

4.3.2.1 Configuring the Environment

When first testing the VBS prototype in the field, the team discovered that RTK resolution was never being reached. It was later determined that the problem was the GPSd server software used on Ubuntu 22.04 was older and thus incapable of reading the u-Blox hardware interface messages that report the quality of the fix. The GPSd was always interpreting the value as differential GPS (DGPS) regardless if the augmentation was RTK quality. Therefore, the deployment was modified to use a Docker service called `vbs-dgps` that uses a more recent version of the server software.

```
$ docker compose run --entrypoint 'gpsd --version' --rm vbs-gpsd
gpsd: 3.25 (revision 3.25)
```

This version of the software correctly reports the status of the RTK fix quality, either fixed or float.

In order for the service to run correctly, it must know the serial device file of the connected RCVW. This can be determined by the following loop command:

```
$ for sysdevpath in $(find /sys/bus/usb/devices/usb*/ -name dev); do
    (
        syspath="${sysdevpath%/dev}"
        devname="$(udevadm info -q name -p $syspath)"
        [[ "$devname" == "bus/"* ]] && exit
        eval "$(udevadm info -q property --export -p $syspath)"
        [[ -z "$ID_SERIAL" ]] && exit
        echo "/dev/$devname - $ID_SERIAL"
    )
done
/dev/ttyUSB0 - FTDI_FT230X_Basic_UART_DO01LPJD
/dev/input/event1 - CHICONY_HP_Basic_USB_Keyboard
/dev/ttyACM0 - u-blox_AG_-_www.u-blox.com_u-blox_GNSS_receiver
```

However, it is probably easiest just to set up the `udev` system to automatically mount the device to the same file name every time[21]. The prototype systems mount to `/dev/ttyGPS`.

Once the environment is configured, the GPSd process can be started and the status of the positioning signal verified.

[21] See https://gist.github.com/edro15/1c6cd63894836ed982a7d88bef26e4af for details on how to configure your udev to link the local GPS device to `/dev/ttyGPS`.

```
$ docker compose up -d vbs-gpsd
$ docker compose exec vbs-gpsd cgps
```

By default, the GNSS receiver may be misconfigured as likely indicated by a 1 message per second rate. In order to reset the device back to the default and then re-configure, executed the following steps:

```
$ docker compose run --entrypoint 'ubxtool -p RESET' --rm vbs-autogps
$ docker compose run --entrypoint 'ubxtool -p COLDBOOT' --rm vbs-autogps
```

Wait about 2 minutes for the GPS to reboot before proceeding.

```
$ docker compose restart vbs-gpsd
$ docker compose run -rm vbs-autogps
$ docker compose exec vbs-gpsd cgps
```

4.4 Driver-Vehicle Interface Overview

The RCVW prototype driver-vehicle interface is simply a web browser displayed through the Ubuntu desktop on a small monitor that is mounted in visible range of the driver. The basic monitor requirements are listed in Table 10. The prototypes come with a Tru-Vu High Definition Multimedia Interface (HDMI) monitor.

Table 10: DVI Monitor Minimum Hardware Specification Checklist

Component	Measure	Minimum	Applies To	Check?
Monitor	Size Resolution	7” 1280x800	VBS	
Power	DC	12V	VBS	
Brightness	Nits (*cd/m2*)	500	VBS	
Video Input Interface	N/A	HDMI or DVI-D	VBS	
Audio Output	External Speaker	75 dB[22]	VBS	

4.4.1 Setup the Tru-Vu HDMI Monitor

The VBS prototype comes with a customized monitor from Tru-Vu for use in RCVW.

4.4.1.1 Connecting the Hardware

The DVI is equipped with a RAM ® Mount base with ball (see Figure 36) which attaches to a Round-A-Mount (RAM) Pod HD vehicle mount (see Figure 37).

[22] Taken from the typical car speaker volume level

Figure 36: RAM Ball mount attached to DVI

Figure 37. RAM Pod HD vehicle mount

The RAM Pod vehicle mount is a no drill mount that attaches to an existing seat rail bolt or seat track. No drilling or modifications to the vehicle are required. The RAM vehicle mount should be installed using the seat rail bolt or seat track of the front passenger seat. Instructions on how to install the RAM Pod HD vehicle mount can be found in the following link: RAM Pod HD Installation Guide

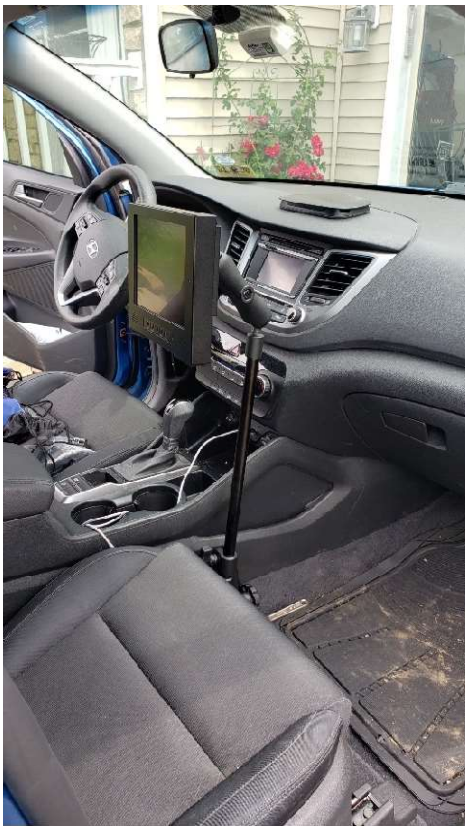


Figure 38. RAM Pod vehicle mount and DVI installation

Once the mount and DVI are installed, locate the DVI power connector and attach it to the DVI port labeled DC 12-24V. Locate the HDMI cable and attach it to the DVI port labeled HDMI IN (See Figure 39 for reference).

Figure 39. DVI connectors

Connect the other end of the HDMI cable to the CPs DVI/ Video Graphics Array (VGA) port. See red rectangle on Figure 40.

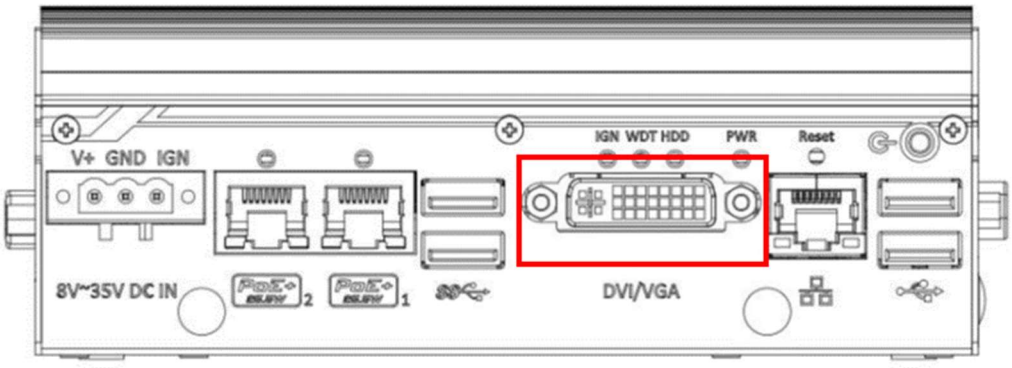


Figure 40. Neousys Intel CPU ports

Finally, connect the DVI and the Terminal Block power connector to auxiliary power outlets.

4.4.2 Setup the VBS Application

The DVI web application is implemented as two separate VBS Docker services. The first is a simple Apache web server meant to serve the RCVW human-machine interface (HMI) pages. The `vbs-hmi` service manually starts up the web server with pages included with the RCVW VBS Docker image. The `httpd-config` Docker config sets up the VBS **`VirtualHost`** through an RCVW-specific config file stored in the `/usr/local/apache2/conf/` extension directory. Note that since the HMI files are mounted from the RCVW plugin Docker service container, the `vbs-rcvw` service has to be a dependency for the HMI, and updates to that plugin container will affect `vbs-hmi` as well.

The HMI pages, however, are loaded dynamically with the JavaScript code through the receipt of RCVW **`Application`** messages via the TMX® core messaging broker. The JavaScript code was originally written to receive messages from a HTTPS web socket server included within TMX®. However, the new version of RCVW utilizes Apache Kafka as the VBS messaging broker, and there is no included web socket server. Therefore, the `vbs-kafka2ws` service operates external software called `kafka2websocket` that allow messages written to Kafka topics be exposed through a web socket. This was done in order to restrict modifications to the HMI.

4.4.2.1 Configuring the Web Browser

The latest version of Mozilla Firefox works best for viewing the RCVW HMI. Other web browsers such as Google Chrome were attempted but fail to properly connect to the `vbs-kafka2ws`. Therefore, Firefox must be installed on the VBS CP:

```
$ sudo add-apt-repository -y -n ppa:mozillateam/ppa
$ printf 'Package: firefox*\nPin: release o=LP-PPA-mozillateam\nPin-Priority: 501\n' |
sudo tee /etc/apt/preferences.d/mozilla
Package: firefox*
```

```
Pin: release o=LP-PPA-mozillateam
Pin-Priority: 501
$ sudo apt update
$ sudo apt install -y firefox
$ firefox --version
Mozilla Firefox 134.0.2
```

4.4.2.2 Configuring the Environment

Start the HMI Apache service and test the HMI by opening up Firefox:

```
$ docker compose up -d vbs-hmi
$ firefox -new-tab "http://localhost/rcvw/InVehicle/index.html"
```

In order for the VBS HMI to come up at reboot, a new Linux system service must be created. This service requires additional software:

```
$ sudo apt install -y xorg openbox
```

A startup script `~user/rcvw-startx.sh` should be created under the `user` account. This script should have the following contents:

```
$ cat ~user/rcvw-startx.sh
#!/bin/bash

xset -dpms
xset s off

openbox-session &

# Wait for the Web Server to come on-line
while ! curl -f -s http://localhost >/dev/null
do
  sleep 1
done

/usr/bin/firefox --kiosk http://localhost/rcvw/InVehicle/index.html
```

Finally, the service file should be added under `/etc/systemd/system/rcvw.service`.

```
$ cat /etc/systemd/system/rcvw.service
[Unit]
Description=Launch Kuba Kiosk app
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=startx /etc/X11/Xsession /home/user/rcvw-startx.sh

[Install]
WantedBy=multi-user.target
```

Enable the service and test the bootup:

```
$ chmod 755 ~user/rcvw-startx.sh
$ sudo systemctl daemon-reload
$ sudo systemctl enable rcvw.service
$ sudo reboot
```

Now, when the computer boots up, the RCVW application screen should appear. Using Alt-F4 will close the browser window and allow a new login to the system.

Figure 41. RCVW VBS Application Screen

The monitor also must not go to sleep during operation. This can be done by logging into the Ubuntu UI and opening the Settings window. Select the Power menu, and make sure the Power Mode is Performance and Screen Blank is set to Never. See Figure 42.

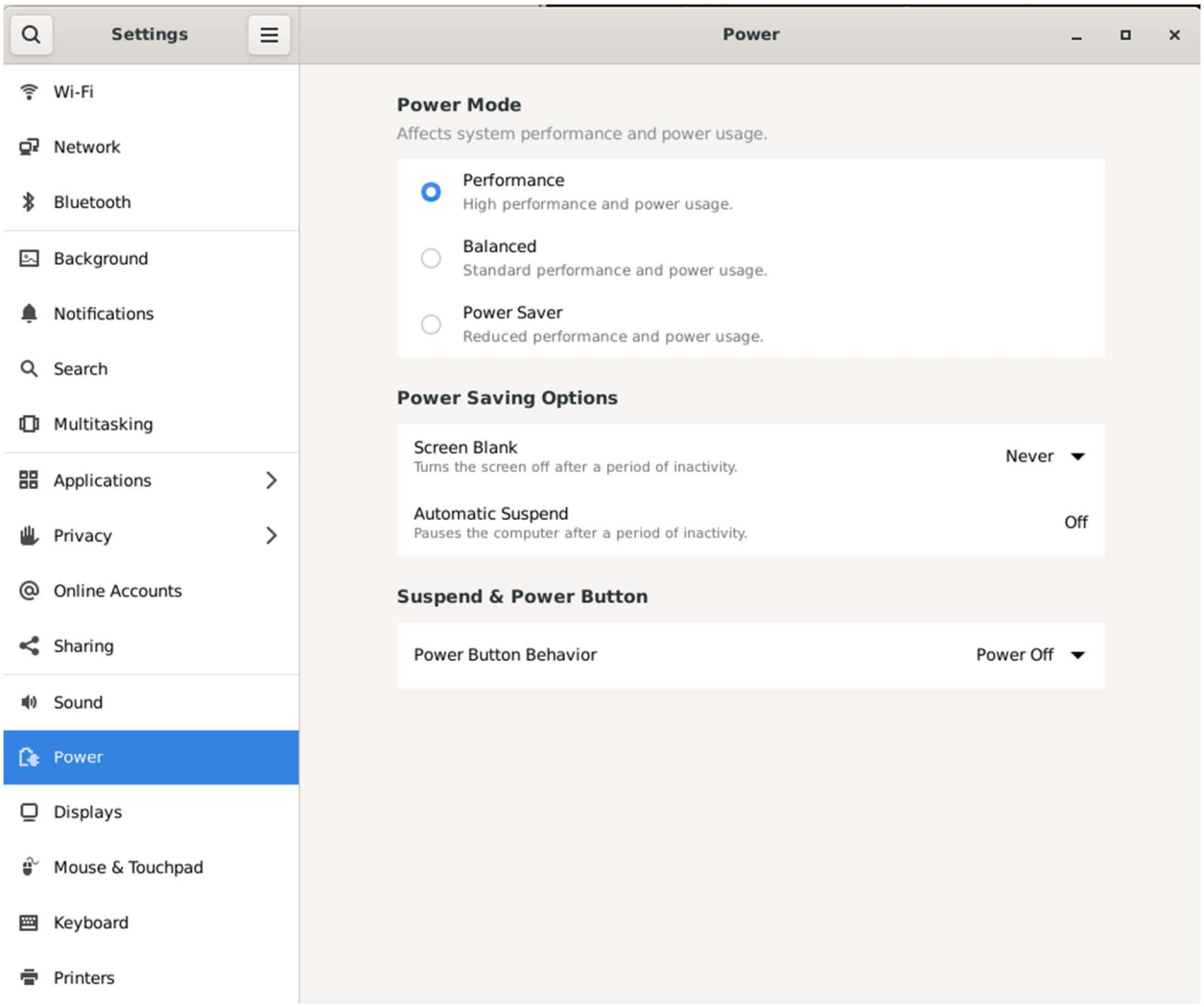


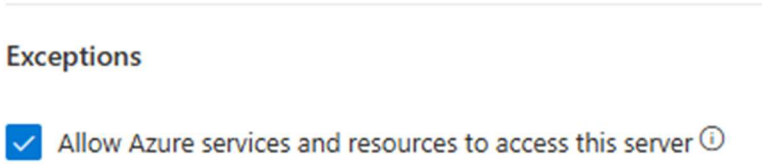
Figure 42. Ubuntu Power Settings Menu

Chapter 5. Microsoft Azure Cloud Installation Standard Operating Procedures

5.1 Azure Structured Query Language (SQL) Overview

The primary RCVW CBS data log consists of a SQL database running in the cloud. The Microsoft Azure SQL Server resource is created to manage the actual database. This resource is generally used to define CPU resources and manage access to the server. This includes configuring the Microsoft Entra administrator ID and appropriate firewall security so that only trusted resources can access the DB server.

Note that the RCVW requirements designate that only subscribed users should be able to access the CBS resources, but the backend DB is ***not*** meant to be public. Therefore, only approved administrators on approved networks should be allowed to directly reach the cloud resources. However, the CBS software itself must be able to connect to this database to work properly. Therefore, for deployment within the same cloud subscription, the following exception should be checked under the Azure SQL Server **`Networking`** configuration:



5.1.1 Setup the CBS Database

The database resource should be added to the Azure SQL Server instance. This database will grow to a very large size depending on the number of equipped HRIs in the deployment and the retention policies of the organization. Each HRI instance may generate up to 20 messages every second, each of which should be under 0.5 kB in size, and the MAP and SPaT will each be logged twice. Therefore, a quick estimate of the data usage *for each HRI* would be `12 kB` per second, or around `1 GB` per day. Make sure that the `rcvw-db` database instance is scaled appropriately.

5.1.1.1 Creating the Database Schema

The deployment source code comes with a SQL script to help create the database schema. This include the tables that match the FRA data dictionary on the static HRI information as well as the actively updated tables, including `HRI_INCOMING_MSG`, `HRI_INCOMING_SPAT`, `HRI_INCOMING_MAP` and `HRI_ACTIVATION_STATUS`. See the RCVW Software API Description for more details about the DB schema. (Baumgardner G. M., 2025)

5.2 Azure Service Bus Overview

Messages being transmitted between the CBS and the RBS utilize an Azure Service Bus Namespace resource with topics specifically created to facilitate that communication.

5.2.1.1 Configuring the Topics and Subscriptions

Each topic and subscription described in Appendix A of the RCVW Software API Description (Baumgardner G. M., 2025) is configured according to the policies laid out by an organization, and by the Software API Description. For example, a message to a `rsu-incoming` subscription should be held only long enough for it to be processed a single time. If multiple instances of the receiver function run, there may be duplicate entries in the data log, and if too many retries exist for failed inserts, the process may become backed up and unable to keep current.

5.2.1.2 Configuring the Queues

There are also two queues that are needed for the CBS Event Manager scheduling to operate correctly. These are queues because there should never be any attempts to retry since a new schedule will automatically occur.

5.3 Azure Container Environment Overview

Once the CBS has an accessible database and the Advanced Message Queuing Protocol (AMQP) queues and topics ready, a deployment for each of the compiled Docker images can be created. Note that these can all be tested locally on the RBS utilizing the environment provided. This section, however, describes how to correctly configure the necessary Function Apps and Container Apps environments within Microsoft Azure. This includes the environment variables that come from the Docker `compose` file. See the RCVW Software API Description (Baumgardner G. M., 2025) for more information about the CBS construction of functions vs. containers.

Containers in Azure can only be retrieved from a public Docker hub or an internal Azure Container Registry (ACR). Therefore, the RCVW CBS images either need to be pushed to a public repository or to an ACR within the same subscription.

Table 11 describes the minimal environment required in the deployment configuration to correctly operate any CBS container.

Table 11: CBS Container Environment Configuration Checklist

Variable	Description	Default Value	Set?
AZURE_SQL_CONNECTIONSTRING	A SQL Server connection string to the Azure DB.	Based on: RCVW_DB_HOST RCVW_DB_USER RCVW_DB_PASS	
rcvwcbssb_CBSReceiver_SERVICEBUS	A service bus end-point connection string for the CBS Service Bus instance	Based on: RCVW_CBS_HOST RCVW_CBS_USER RCVW_CBS_PASS	

5.3.1 Setup the Function Apps Environment

The Function Apps environment was originally constructed in order to directly deploy the Azure Python function code. However, since Docker image deployment is generally more portable to unit test systems and other clouds, this plan was altered to make the Function Apps Environment a logically separate Azure Container Apps Environment for the two Python trigger functions: the CBS Ingest and the CBS Event Manager. Aside from the DB and Service Bus operation, there should never be data ingress or egress outside of this environment.

5.3.1.1 Configure the CBS Ingest Container App

The CBS Ingest container application is designed to trigger upon incoming messages to the Service Bus in order to log those messages into the database. Because of this design, the CBS Ingest container app is the only CBS component meant to run multiple replicas. For example, if the CBS is monitoring 5 HRIs, then 5 or more replicas of this trigger should be running in the Azure Container Environment in order to process the load effectively.

5.3.1.2 Configure the CBS Event Manager Container App

The CBS Event Manager container application is triggered by the scheduler built into the Service Bus queues `cbs.scheduler.hri-check` and `cbs.schedule.rbs-check`. While these checks can technically happen simultaneously, it is generally not recommended because the RBS check routine first requires that the HRI check routine passes, thus making the separate operations effectively sequential. Both operations manipulate the DB directly, thus making this container dependent on the CBS Ingest process.

5.3.2 Setup the Container Apps Environment

The major difference between this Container Apps Environment and the Function Apps Environment is that both of these containers require ingress to and/or egress from the Azure network. Therefore, the logical separation may also be used to design the cloud deployment efficiently so not to allocate money to unnecessary features.

5.3.2.1 Configure the CBS API Container App

The CBS API container application is built on the Azure Data API Builder (DAB) feature that only requires a connection to the database, as configured in **Error! Reference source not found.**. The **`Ingress`** section of the app deployment configuration should be set to allow incoming Hyper-Text Transport Protocol (HTTP) traffic to port 5000. Insecure connections were allowed for test deployments in lieu of producing a certificate for the HTTP server.

5.3.2.2 Configure the CBS RTCM Proxy Container App

The RTCM Proxy container application exists singularly to push RTCM correction messages for a specific HRI. Therefore, a unique deployment must exist for every HRI that is being monitored. The function will post messages directly to the `v2x.rtcm3` Service Bus topic with the AMQP address set to the HRI identifier. In order for the RBS RTCM plugin to receive the message, however, a subscription specific to that HRI is needed. For example, `rtcm-14815` would be used for HRI 14815.

In addition to the specified connection parameters, the RTCM Proxy application requires additional environment variables as denoted in Table 12.

The Python script for determines the position of the HRI by connecting to the CBS API. Then, the script will retrieve the corrections from the NTRIP caster and will forward the returned connections through the Azure Service Bus topic.

Table 12: CBS RTCM Proxy Container Environment Configuration Checklist

Variable	Description	Default Value	Set?
RCVW_HRI	The identifier for the HRI for which the RTCM corrections will be attained	None	
RCVW_CBS_API	The hostname for the CBS API service, e.g., Azure Data API Builder (DAB), which is used to obtain latitude and longitude of the HRI	`cbs-api`	

Chapter 6. MAP Messaging Standard Operating Procedures

A SAE J2735 MapData message provides the appropriate method for constructing and distributing the intersection geometry to the vehicle but consigns most implementation details to the programmer of the RCVW application. For RCVW to function properly, lanes are created within this MAP that represent vehicle entry lanes approaching the HRI, as well as exit lanes leaving the HRI. Additionally, a separate lane or set of lanes are required to represent the railway track itself. The vehicle and rail lanes, however, must be in separate signal groups since the accompanying SPaT messages will signal a stop (red-light) condition for the vehicle approach lane while a train is present and a proceed (green-light) condition when there is no train present. This section describes the procedure for building the geometric representation of the HRI and communicating it over the DSRC safety channel. This requires the Map Plugin and RSU Immediate Forward Plugin (see Figure 9) on the RBS, as well as an external MAP creator tool such as the Intersection Situation Data (ISD) Builder.

6.1 HRI MAP Messaging Overview

The key for proper RCVW operation is to build the geometry of the HRI such that alerts occur at recommended distances. RCVW alerting occurs in two phases. The Inform alert is issued at any point in which the vehicle is in an approach lane and a train is present. A second, more intrusive, Warn alert occurs at any point within the approach lane at which the Decision Sight Distance as defined by the American Association of State Highway Officials (AASHTO) Green Book (AASHTO, 2018) at the approaching vehicle speed exceeds the total distance from the stop bar at the end of the lane. Therefore, by design, the optimal length of a vehicle approach lane in the MAP should be representative of that Decision Sight Distance from the traditional Warning Sign Placement for the highway-rail grade crossing signage as defined in the Manual on Uniformed Traffic Control Devices (MUTCD) (Federal Highway Administration, 2009) for the posted speed limit or 85th percentile (typical) speed on the highway. For example, for a 55-mph rural road with a Decision Sight Distance of 535 feet and a traditional Warning Sign Placement of 990 feet (MUTCD Table 2C-4 Condition A in this example), an approach lane for the vehicle should begin at 535 + 990 = 1525 feet from the stop bar. This ensures that if a train is present, an inform shall occur upon entering the MAP approach lane, and the warn may occur at least as far away as where the fixed sign placement would have been. Note, however, that unlike the traditional signage, the active RCVW warn alert may also occur (or re-occur) at any position following, up to the stop bar, thus providing an effective notice to a distracted driver that might commonly miss the stationary rail-crossing sign.

The J2735 MAP message standard identifies the intersections it represents under a sequence of `IntersectionGeometry` objects, that includes among other things a unique identifier (`IntersectionReferenceID`), the latitude / longitude coordinates (`refPoint`), and `LaneList`, which is a sequence of lane information. To capture the presence of a train, the `LaneList` must include vehicle lanes entering and exiting the HRI, plus a non-traditional lane that represents the railroad tracks. The key component within each lane structure includes an identifier unique to this intersection (`LaneID`), some attributes identifying the direction of the lane (`LaneDirection`), and the type of vehicle(s) using the lane (`LaneTypeAttributes`),

which should be set to **`vehicle`** for the vehicle approach lanes and **`trackedVehicle`** for the rail lane (SAE International, 2016).

Each node in the lanes is encoded using coordinate (x, y) offsets from the previous node. This limits the size of the MAP file being broadcast. See Appendix B.

Finally, the available sequence of `Connection` attributes for each lane in the MAP message can be used to identify several important relationships in the traffic flow. For example, the `ConnectingLane` object identifies both an adjacent lane that this one connects to (`LaneID`) and the allowed vehicle maneuver (`AllowedManeuvers`) between them, such as **`maneuverStraightAllowed`**, **`maneuverLeftAllowed`**, **`maneuverRightAllowed`** and/or **`maneuverUTurnAllowed`**. Additionally, the connecting lane is mapped to a "signal group" identifier (`SignalGroupID`) that indicates the traffic signal controller (TSC) signal phase or overlap that controls the maneuver for this connection (SAE International, 2016).

6.1.1 Building a MAP Message

The MAP message Extensible Markup Language (XML) or JavaScript Object Notation (JSON) format may be built by hand, but it takes specialized software in order to convert that over to the Abstract Syntax Notation One (ASN.1) encoded byte structure, which for the J2735 2006 release uses the unaligned-packed encoding rules (UPER) of a ubiquitous MessageFrame envelope that specifies the specific message enclosed in order to decode it. Therefore, to avoid extra work in the encoding of the message, the ISD Builder Tool (ISD Builder Tool for J2735 3/2016, n.d.) is a web application designed to assist in building the MAP message contents and encoding them. This section details instructions to assist in this process, as a supplement to the on-line help on the ISD page.

6.1.1.1 Creating a Parent Map

The ISD tool conceptually has a *parent* map file that identifies the global configuration of the HRI, which primarily consists of the `refPoint`, and a *child* map that constructs the `LaneList` and `Connection` objects. The parent map can be built under the **`File->New Parent Map`** option. The two tools available under the parent map builder are the **`Reference Point Marker`** and the **`Verified Point Marker`**. Add the **`Reference Point Marker`** to the point that will represent the center of the HRI. Whereas the Latitude, Longitude and Elevation options should remain the same, the Intersection Name, Intersection ID and Master Lane Width fields may be customized. Next, place a **`Verified Point Marker`** on the specific location on the MAP which ideally, should be a GNSS point that was professionally surveyed. This time replace the **`Verified Latitude`**, **`Verified Longitude`** and **`Verified Elevation`** fields with the resultant surveyed values.

The parent map is complete and can be saved using **`File->Save`**. A revision number is added to the file name in order to keep track of future updates, such as to the verified points. The resulting JSON file can be reloaded into the editor using **`File->Open Parent Map`**.

6.1.1.2 Creating a Child Map

A new child map is created with the **`File->New Child Map`** option. The child map in the ISD tool is always associated with some parent map, thus a dialog opens up to select the parent map to use. In particular, the reference GNSS points built into the parent map are used to

calculate the node locations for the lanes. Changes to that parent map are not automatically reflected in the child map. Instead, they must be manually updated under the child map using the **`File->Update Child Markers`** option.

Both markers from the parent map can now be seen in the child map. The **`Reference Point`** should be used as the relative position for constructing the lanes in the map. The toolbox for the child map editor consists of Approach, Lane, and Measure. An approach is a container for lane, and in general there is a one-to-one mapping of approach to lane. The initial step to constructing the lanes for the MAP in the ISD is to build the Approach. See the on-line help for specific drawing instructions. The actual Approach can be a small box big enough to hold a single lane node near the HRI reference. For example, Figure 43, shows two approach boxes, one for an ingress lane and one for an egress lane in the opposite direction.

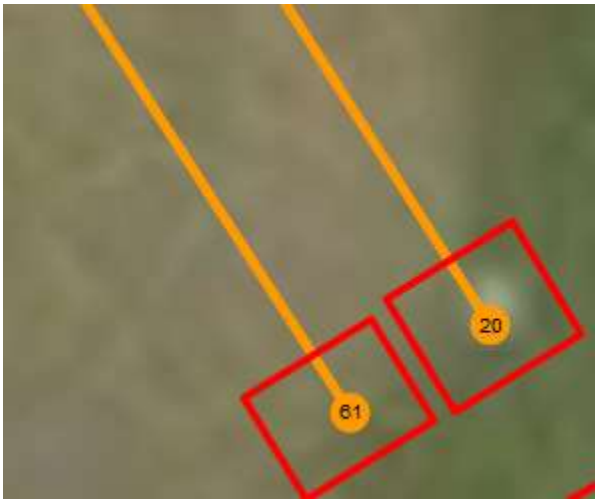


Figure 43. Two approaches for two lane nodes

Every Approach has two attributes: the type and the ID. An approach type can be Ingress (incoming), Egress (outgoing), Both or None (the approach type None means that it does not support travel such as medians, curbs, etc.). Obviously, the lanes used by the vehicle should be exclusively Ingress or Egress. However, the railroad track lane can either be directional if there are directional tracks, or they can be combined into a single approach of type Both.

An approach ID can really be anything, but traffic engineers tend to follow simple numbering schemes while configuring traffic controller phases that denote the directionality of movement. Left turn movements are assigned odd numbers, while through movements are assigned even numbers. Phase 2 is usually assigned to the major streets, and the rest are numbered clockwise by twos around the intersection. More generally, this can be applied to the directions on a compass as seen in Figure 44.

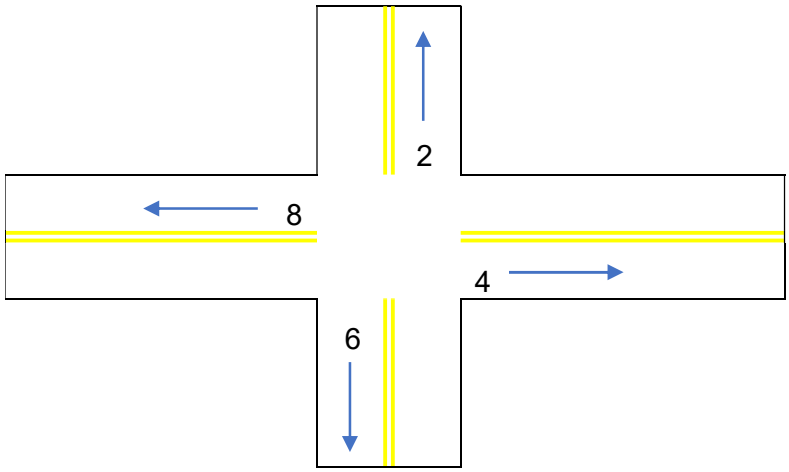


Figure 44. Basic Approach Numbering

A complementary approach ID used for turning lanes should be set so that the sum of the two are either 7 or 11 (for numbers 7 and above). Thus, a 5 also applies N approach, since 2 (N) + 5 = 7. Likewise, 7 is an E approach since 4 (E) + 7 = 11. Thus, the odd number approaches are (N, S, E, W): 5, 1, 7, 3. Variations of this numbering scheme has been used in previous software applications to simplify calculations, but currently the directionality for RCVW enabled vehicles is determined by the heading of the vehicle in relation to the HRI location. Therefore, this numbering is optional, although still highly recommended.

After the approach is drawn, the lane nodes are created using the available Lane tool. The first node should be closest to the HRI, and each subsequent node should move further away. Node positions are generally only set to build a smoother geometry for the lane, such as adding a dense population of nodes along the curve the road. The Lane Type and Lane Number are key attributes that should be set on the first node of the lane. The type should be ***Vehicle*** for every type of automobile lane, including the tracks, but the MAP may also define crosswalk or bicycle lanes. The numbering of the lane is recommended to be a two-digit value with the first digit being the same as the approach. For example, the lane 61 from Figure 45 is the first south-bound lane. In that same picture, lane 20 represents the egress lane that is immediately across from lane 21, the ingress.

Train tracks are a special case of a vehicle lane that requires a Shared With attribute setting to ***(8) Tracked Vehicle***. As stated previously, and generally speaking, there is one lane in an approach. However, since the RCVW software is built upon the V2X Hub intersection concept which requires separate ingress and egress lanes for a thru maneuver, the train tracks must be split on either side of the HRI center. Therefore, the example below as seen in Figure 45 includes two separate sections of the tracks (lanes 40 and 41) within the same approach. The software simply assumes the tracks span directly through the HRI.

Figure 45. A simple HRI with approach lanes and tracks

Thus far, only the static position and geometry has been defined for the child map. There is additional information that must be configured into the lanes that help define the SPaT relationships. Under the Lane Configuration of the first lane node, there is a SPaT tab and a Connections tab. The former is for static phase information such as a stop sign or a permanent flashing light. The Connections tab is more useful in a dynamic situation. Select the ingress approach lane (e.g. lanes 21 and 61), then add a new connection under that tab. The connection ID can be sequential, but the "To" Lane Number must be the lane number of the direct egress lane on the other side of the track (e.g. lanes 20 and 60). Additionally, a signal group ID needs to be set for the connection. It is important that the approach lanes are assigned a distinct signal group ID from the track lanes. For example, the signal group 1 can be for the tracks and signal group 2 for the approach lanes. The builder tool on the editor contains a set of lane feature signs that can be dragged over to the allowed maneuvers. For the example in Figure 45, the connection from lane 21 to 20 should be a straight-thru maneuver. See Figure 46 for a completed connection.

Figure 46. Approach lane connection over the tracks

After adding the approach lane maneuvers, the same process needs to be applied to the tracks, only with a different signal group, as shown in Figure 47.

Figure 47. Railroad track lane connection

After the connections are built, the child map should be saved with revision number by selecting **`File->Save`**.

Finally, the resultant MAP must be encoded by the ISD tool for use in V2X Hub. This is done with the **`Tools->Encoder`** option. When used with the MAP Plugin, the Message Type must be set to **`Frame+Map`** and the Node Offsets should be **`Tight`**. If the crossing is significantly graded, the **`Enable Elevation?`** option must be marked as true. When the **`Encode`** button is pressed, the UPER Hex field will have the exact J2735 MapData message output that is to be broadcast. This value must be copied from the browser window and saved to the environment file on the RBS manually.

6.1.2 Configuring the MAP Plugin

The `manifest.json` configuration file for the MAP plugin allows one to set the parameters for running the plugin. This is automatically generated in the RBS prototype using a Docker config. Each MAP file relates to an "action" on the TSC. The default action is `0`.

```
plugin@rcvw-rbs-14815:/var/tmp$ cat /var/lib/plugin/MAP/manifest.json
{
  "name": "RCVW-RBS-MAP",
  "description": "Plugin that reads intersection geometry from a configuration file
and publishes a J2735 MAP message.",
  "version": "4.0.0",
  "exe": "bin/MapPlugin",
  "channels": [
    {
      "comment": "TMX RBS core connection. Must use unique IDs for Kafka.",
      "context": "kafka://localhost:29092",
      "config": {
        "auto-subscribe": false,
        "write-only": true,
        "topics": "^(J2735|tmx)/",
        "thread-count": 2,
        "thread-assignment": "ShortestQueue"
      }
  ],
  "Frequency": 1000,
  "MapFiles": [
    {
      "Action": 0,
      "Bytes":
"001280EC380530203073BE154D75DE4439AB3522198C02DC051F702B2009707870E5BF0E3D1330EECA280
00000604C8253482964405050455377105 1B2F90CCC6D488211B1F4435E1A6D21BB2D1A08F2D7CC2221272
```

```
8B996040A0A19A958103253AE58632D56486C2E00073BE0202480668000000107AE5D9277993CDAF50B31F
02828389A8E86C3600073BE0202201764000000107F5B651713CCB0B88A6983EDE838806E90000000411B1
0B5213972014141342C9C67E66F998011130080080015A79BC85D17C3E680888C020020287119A40E4E2A6
1D325343B48EB03865DC78D7881133C98918360500039DF0201"
    }
  ],
  "loglevel": "DEBUG"
}
```

Chapter 7. SPaT Messaging Standard Operating Procedures

7.1 HRI SPaT Messaging Overview

For a SPaT message, the `IntersectionState` object is used to pass the dynamic state information within an intersection. The `IntersectionReferenceID` field specifies the identifier of the intersection as defined in the MAP message. Likewise, each `MovementState` object contains the signal group identifier (`SignalGroupID`) defined in the MAP and a `MovementEventList` for the current state. The project team manipulated the `MovementPhaseState` to signify the state of the virtual light for the signal group maneuver. For example, **`stop-And-Remain`** represents a normal red light and **`protected-Movement-Allowed`** a normal green. Whereas a conventional SPaT application uses state data for each signal group directly from the TSC, the HRI status software running on RCVW RBS transitions each signal group to its appropriate `MovementPhaseState` based on the status of the HRI infrastructure equipment. This represents whether or not a train is approaching based on the phase state of the signal group.

7.1.1 Configuring the Preemption Signal

The phase state is automatically injected into the SPaT message by the HRI Status Plugin. There are two possible configurations that can be used to determine the HRI pre-emption signal state. The first is a digital interface developed by the IEEE, which employs the TIA/EIA-422 serial communication electrical interconnection interface standard. The second utilizes the existing discrete voltage common to the railway train detection systems.

7.1.1.1 Voltage-based Signaling

The Neousys POC-451VTC also has a discrete Digital Input/Output (DIO) feature. Cabling interconnects the HRI status signal, and its return, to the DIO via a DB-15 connector. Input pins are (either 1, 2, 10 or 11 for the signal), and (either pin 3 or 9 for the return). The signal is active low, so a drop in voltage from high to low will indicate that a train is present. Thus, the input should be grounded when the signal is disconnected or lost, indicating the safe state of presumed train presence. Note that pin chosen for the voltage must be configured in the HRI Status Plugin.

7.1.1.2 IEEE 1570 Signaling

The IEEE 1570 messaging code is built into the HRI Status Plugin. Cabling interconnects the IEEE 1570 device to the computing platform of the RBS. The POC-451VTC has multiple COM serial ports, each having a DB-9 male connector, that can be configured in the BIOS as a TIA/EIA-422 terminus. The HRI pre-emption messages received by the application are decoded and translated and the resultant status set inside the outgoing SPaT. If no messages are received by the plugin, the HRI is assumed to be active, i.e. a train present.

7.1.2 Configuring the HRI Status Plugin

The HRI Status plugin `manifest.json` file sets how to interpret the MAP signal groups as well as providing intersection information to use in the SPaT message.

7.1.2.1 Voltage-based Signaling

When using the voltage-based signaling, no COM port should be specified, and the DIO **`RailPinNumber`** must be configured in the manifest.

```
root@rcvw-rbs-21223:/var/tmp# cat /var/lib/plugin/HRIStatus/manifest.json
{
  "name": "RCVW-RBS-HRI",
  "description":"Sends HRI Status as SPAT Message",
  "version":"4.0.0",
  "exe": "bin/HRIStatusPlugin",
  "channels":[
    {
      "comment": "TMX RBS core connection. Must use unique IDs for Kafka.",
      "context": "kafka://localhost:29092",
      "config": {
        "auto-subscribe": true,
        "write-only": true,
        "topics": "^(J2735|tmx)/",
        "thread-count": 2,
        "thread-assignment": "ShortestQueue"
      }
    }
  ],
  "Frequency": 100,
  "RailPinNumber": 0,
  "LaneMap": [
    { "tracked": true, "signalGroup": 1 },
    { "vehicle": true, "signalGroup": 2 }
  ],
  "IntersectionID": "21223",
  "IntersectionName": "San Juan Ave & Roosevelt Blvd",
  "AlwaysSend": true,
  "PortName": "",
  "SerialDataTimeout": 1500,
```

```
  "loglevel": "DEBUG"
}
```

7.1.2.2 IEEE 1570 Signaling

The **`PortName`** specifies the COM port on the POC-451VTC to use for reading the IEEE 1570 device.

```
root@rcvw-rbs-14815:/var/tmp# cat /var/lib/plugin/HRIStatus/manifest.json
{
  "name": "RCVW-RBS-HRI",
  "description":"Sends HRI Status as SPAT Message",
  "version":"4.0.0",
  "exe": "bin/HRIStatusPlugin",
  "channels":[
    {
      "comment": "TMX RBS core connection. Must use unique IDs for Kafka.",
      "context": "kafka://localhost:29092",
      "config": {
        "auto-subscribe": true,
        "write-only": true,
        "topics": "^(J2735|tmx)/",
        "thread-count": 2,
        "thread-assignment": "ShortestQueue"
      }
  ],
  "Frequency": 100,
  "RailPinNumber": 0,
  "LaneMap": [
    { "tracked": true, "signalGroup": 2 },
    { "vehicle": true, "signalGroup": 1 }
  ],
  "IntersectionID": "14815",
  "IntersectionName": "Battelle West Jefferson",
  "AlwaysSend": true,
  "PortName": "/dev/ttyUSB0",
  "SerialDataTimeout": 1500,
  "loglevel": "DEBUG"
```

```
}
```

Chapter 8. RTCM Correction Messaging Standard Operating Procedures

8.1 HRI RTCM Messaging Overview

The RTCM Correction message is one of the more straight-forward structures within the J2735 message set. This is primarily because the RTCM messages themselves have been well defined for some time, and thus their use over DSRC broadcast effectively became a pass-through. There is a field that indicates what version of RTCM is used for the enclosed messages (`rev`), plus a sequence of 1kB octet strings (`msgs`). The `msgs` are inserted verbatim from the RTCM correction message generated by the NTRIP caster or some other means, subjected only to the J2735 encoding process.

8.1.1 Configuring the RTCM Plugin

The RTCM plugin can be configured one of three ways to receive corrections. The RBS prototype systems utilize the environment variables from Table 6 in order to generate a `manifest.json` file that will select the appropriate source.

8.1.1.1 CBS RTCM Proxy Source

The plugin must contain a connection to the CBS Service Bus for the subscription setup as described in section 5.3.2.2.

```
plugin@rcvw-rbs-21223:/var/tmp$ cat /var/lib/plugin/RTCM/manifest.json
{
  "name": "RCVW-RBS-RTCM",
  "description": "Plugin to listen for RTCM messages from an NTRIP caster and route
those messages over C-V2X",
  "version":"4.0.0",
  "exe": "bin/RtcmPlugin",
  "channels": [
    {
      "comment": "TMX RBS core connection. Must use unique IDs for Kafka.",
      "context": "kafka://localhost:29092",
      "config": {
        "auto-subscribe": false,
        "write-only": true,
        "topics": "^(J2735|tmx)/",
        "thread-count": 2,
```

```
        "thread-assignment": "ShortestQueue"
      }
    },
    {
      "comment": "Attempt to automatically subscribe to the RTCM proxy channel in the
CBS for this HRI.",
      "id": "rtcm-proxy",
      "context": "sb://${RCVW_CBS_USER}:${RCVW_CBS_PASS}@${RCVW_CBS_HOST}",
      "config": {
        "auto-subscribe": true,
        "auto-publish": false,
        "subscription": "rsu-21223",
        "topics": "^V2X/RTCM[23]",
        "thread-count": 2,
        "thread-assignmet": "ShortestQueue"
      }
    }
  ],
  "latitude": "0.0",
  "longitude": "0.0",
  "loglevel": "DEBUG"
}
```

8.1.1.2 RBS Base Station Source

If a local GNSS receiver is used as a base station, a GPSd connection can be configured:

```
plugin@rcvw-rbs-14815:/var/tmp$ cat /var/lib/plugin/RTCM/manifest.json
{
  "name": "RCVW-RBS-RTCM",
  "description": "Plugin to listen for RTCM messages from an NTRIP caster and route
those messages over C-V2X",
  "version":"4.0.0",
  "exe": "bin/RtcmPlugin",
  "channels": [
    {
      "comment": "TMX RBS core connection. Must use unique IDs for Kafka.",
      "context": "kafka://localhost:29092",
```

```
      "config": {
        "auto-subscribe": false,
        "write-only": true,
        "topics": "^(J2735|tmx)/",
        "thread-count": 2,
        "thread-assignment": "ShortestQueue"
      }
    },
    {
      "comment": "The RTCM source from a GPSd basestation.",
      "id": "rtcm-gpsd",
      "context": "gpsd://localhost",
      "config": {
        "auto-subscribe": true,
        "auto-publish": false,
        "topics": ".*RTCM[23]*.*",
        "thread-count": 2,
        "thread-assignmet": "ShortestQueue"
      }
    }
  ],
  "latitude": "0",
  "longitude": "0",
  "loglevel": "DEBUG"
}
```

8.1.1.3 NTRIP Caster

It is possible to directly configure an NTRIP caster as a connection to the plugin, also setting the latitude and longitude of the HRI in order to generate the required NMEA input string:

```
plugin@rcvw-rbs-14815:/var/tmp$ cat /var/lib/plugin/RTCM/manifest.json
{
  "name": "RCVW-RBS-RTCM",
  "description": "Plugin to listen for RTCM messages from an NTRIP caster and route
those messages over C-V2X",
  "version":"4.0.0",
  "exe": "bin/RtcmPlugin",
```

```
  "channels": [
    {
      "comment": "TMX RBS core connection. Must use unique IDs for Kafka.",
      "context": "kafka://localhost:29092",
      "config": {
        "auto-subscribe": false,
        "write-only": true,
        "topics": "^(J2735|tmx)/",
        "thread-count": 2,
        "thread-assignment": "ShortestQueue"
      }
    },
    {
      "comment": "The RTCM source from an NTRIP castor.",
      "id": "rtcm-ntrip",
      "context": "ntrip://example.ntrip.com:2101/rtcm3",
      "config": {
        "comment": "NTRIP sources must wait to subscribe because they require a GGA
string.",
        "auto-subscribe": false,
        "auto-publish": false,
        "topics": ".*RTCM[23]*.*",
        "thread-count": 2,
        "thread-assignmet": "ShortestQueue"
      }
    }
  ],
  "latitude": "39.9570248",
  "longitude": "-83.2478194",
  "loglevel": "DEBUG"
}
```

Works Cited

AASHTO. (2018). *A Policy on Geometric Design of Highways and Streets* (7th ed.). Atlanta, GA: AASHTO.

Baumgardner, G. M. (2020). *Position Solution Comparative Analysis Technical Memorandum.*

Baumgardner, G. M. (2025). *Rail-Crossing Violation Warning (RCVW) Software API Description.*

Baumgardner, G., Paselsky, B., & Sanchez-Badillo, A. (2021). *RCVW Standard Operating Procedure - Hardware and Software Configuration.* Washington, D.C: Federal Railroad Administration.

Federal Highway Administration. (2009). *Manual on Uniform Traffic Control Devices for Streets and Highways.* Washington, D.C.: Federal Highway Administration.

IEEE Standard for Wireless Access in Vehicular Environments (WAVE) - Identifier Allocations. (2016). *IEEE Std 1609.12-2016 (Revision of IEEE Std 1609.12-2012)*, 1-21.

ISD Builder Tool for J2735 3/2016. (n.d.). Retrieved from connectedvcs.com: https://webapp.connectedvcs.com/isd/

SAE International. (2016). *J2735: Dedicated Short Range Communications (DSRC) Message Set Directory.* Warrendale, PA: SAE International.

Sanchez-Badillo, A., & Baumgardner, G. (2024). *Rail Crossing Violation Warning Architecture and Design Specifications.*

Appendix A. List of Acronyms and Abbreviations

AASHTO	American Association of State Highway Officials
ACR	Azure Container Registry
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
ASN.1	Abstract Syntax Notation One
BIOS	Basic Input/Output System
CBS	Cloud-based Subsystem (an RCVW subsystem)
CMD	Command Prompt
CP	Computing Platform
CPU	Computer Processing Unit
CV	Connected Vehicle
C-V2X	Cellular Vehicle to Everything
DAB	Data API Builder
DC	Direct Current
DGPS	Differential GPS
DIO	Digital Input/Output
DSRC	Dedicated Short-Range Communication
DVI	Driver-Vehicle Interface
FRA	Federal Railroad Administration
GNSS	Global Navigation Satellite Systems
GPS	Global Positioning System
HDMI	High-Definition Multimedia Interface
HRI	Highway-Rail Intersection
HTTP	Hyper-Text Transport Protocol
IEEE	Institute of Electrical and Electronics Engineers
IMU	Inertial Measurement Unit
ISD	Intersection Situation Data
JSON	JavaScript Object Notation
LAN	Local Area Network
LTE	Long Term Evolution
LTS	Long-term Support

MAP	Roadway Geometry and Attribute Data (a.k.a., GID)
MUTCD	Manual on Uniformed Traffic Control Devices
NMEA	National Electrical Manufacturers Association
NTP	Network Time Protocol
NTRIP	Network Transport of RTCM via Internet Protocol
OBU	On-board Unit (DSRC radio in VBS)
PCAP	Paquet Capture
PoE	Power Over Ethernet
PPS	Pulse Per Second
PVT	Position Velocity and Time
RAM	Round-A-Mount (ball and socket mounting design)
RFP	Request for Proposal
RBS	Roadside-based Subsystem (an RCVW subsystem)
RCVW	Rail Crossing Violation Warning (warning message)
RSU	Roadside Unit (DSRC radio in RBS)
RTCM	Radio Technical Commission for Maritime Services
RTK	Real-Time Kinematic
SMA	SubMiniature version A
SOP	Standard Operating Procedure
SPaT	Signal Phase and Timing
SQL	Structured Query Language
SSH	Secure Shell
TSC	Traffic Signal Controller
TMC	Traffic Management Center
UDP	User Datagram Protocol
UPER	Unaligned-Packed Encoding Rules
USB	Universal Serial Bus
US DOT	US Department of Transportation
VAC	Volts Alternating Current
VBS	Vehicle-based Subsystem (an RCVW subsystem)
VGA	Video Graphics Array
V2X	Vehicle to Everything
WAVE	Wireless Access in Vehicular Environments

WSM	WAVE Short Message
XML	Extensible Markup Language

Appendix B. Example MAP Message

Note that decoding the message from hexadecimal currently requires a special tool built from the SAE J2735 ASN.1 files.

```
$ echo
001280FC3801300020D45C0547A5FB763AA9E4AE0F2402DC16682A44C02002000954F35C89F965FE193D80
03517009A1EB33008008001BEE3F0B2F10B37619158003517009210 9A000000041E41450B543D049208020
005DF098654E000D45C04484068000000007C3E4E2DA8241D50654A000D45C04483E68000000007DA7D02D
A6841AE86546000D45C0420548400000000B0D595CB466B0438205284000000007F0B582D1A7C12E1223BA
0000000011E4C78A7D68F9941929800351701122 43A0000000012CC68CA7BF2FA54192A8003517010828C1
0000000022F4B3620D42F4F882941000000002 3689F420CCEF5C0829C10000000010891B0AA97EFBC8 |
xxd -r -p | ~/j2735-dump -iper -
<MessageFrame>
    <messageId>18</messageId>
    <value>
        <MapData>
            <msgIssueRevision>1</msgIssueRevision>
            <layerType><intersectionData/></layerType>
            <layerID>0</layerID>
            <intersections>
                <IntersectionGeometry>
                    <id>
                        <id>27182</id>
                    </id>
                    <revision>1</revision>
                    <refPoint>
                        <lat>302060150</lat>
                        <long>-815787345</long>
                        <elevation>-220</elevation>
                    </refPoint>
                    <laneWidth>366</laneWidth>
                    <laneSet>
                        <GenericLane>
                            <laneID>21</laneID>
                            <ingressApproach>2</ingressApproach>
```

```
<egressApproach>2</egressApproach>
<laneAttributes>
    <directionalUse>
        11
    </directionalUse>
    <sharedWith>
        0000000010
    </sharedWith>
    <laneType>
        <vehicle>
            00000100
        </vehicle>
    </laneType>
</laneAttributes>
<nodeList>
    <nodes>
        <NodeXY>
            <delta>
                <node-XY3>
                    <x>-685</x>
                    <y>1239</y>
                </node-XY3>
            </delta>
        </NodeXY>
        <NodeXY>
            <delta>
                <node-XY5>
                    <x>-3086</x>
                    <y>4863</y>
                </node-XY5>
            </delta>
        </NodeXY>
    </nodes>
</nodeList>
<connectsTo>
    <Connection>
        <connectingLane>
```

```
                        <lane>61</lane>
                        <maneuver>
                            100000000000
                        </maneuver>
                    </connectingLane>
                    <remoteIntersection>
                        <id>27182</id>
                    </remoteIntersection>
                    <signalGroup>1</signalGroup>
                </Connection>
            </connectsTo>
        </GenericLane>
        <GenericLane>
            <laneID>61</laneID>
            <ingressApproach>6</ingressApproach>
            <egressApproach>6</egressApproach>
            <laneAttributes>
                <directionalUse>
                    11
                </directionalUse>
                <sharedWith>
                    0000000010
                </sharedWith>
                <laneType>
                    <vehicle>
                        00000100
                    </vehicle>
                </laneType>
            </laneAttributes>
            <nodeList>
                <nodes>
                    <NodeXY>
                        <delta>
                            <node-XY2>
                                <x>503</x>
                                <y>-772</y>
                            </node-XY2>
```

```
                        </delta>
                    </NodeXY>
                    <NodeXY>
                        <delta>
                            <node-XY6>
                                <x>6024</x>
                                <y>-9797</y>
                            </node-XY6>
                        </delta>
                    </NodeXY>
                </nodes>
            </nodeList>
            <connectsTo>
                <Connection>
                    <connectingLane>
                        <lane>21</lane>
                        <maneuver>
                            100000000000
                        </maneuver>
                    </connectingLane>
                    <remoteIntersection>
                        <id>27182</id>
                    </remoteIntersection>
                    <signalGroup>1</signalGroup>
                </Connection>
            </connectsTo>
        </GenericLane>
        <GenericLane>
            <laneID>33</laneID>
            <ingressApproach>3</ingressApproach>
            <laneAttributes>
                <directionalUse>
                    10
                </directionalUse>
                <sharedWith>
                    0000000000
                </sharedWith>
```

```
                <laneType>
                    <vehicle>
                        00000000
                    </vehicle>
                </laneType>
            </laneAttributes>
            <nodeList>
                <nodes>
                    <NodeXY>
                        <delta>
                            <node-XY2>
                                <x>800</x>
                                <y>276</y>
                            </node-XY2>
                        </delta>
                    </NodeXY>
                    <NodeXY>
                        <delta>
                            <node-XY6>
                                <x>10782</x>
                                <y>585</y>
                            </node-XY6>
                        </delta>
                    </NodeXY>
                    <NodeXY>
                        <delta>
                            <node-XY1>
                                <x>0</x>
                                <y>0</y>
                            </node-XY1>
                        </delta>
                    </NodeXY>
                    <NodeXY>
                        <delta>
                            <node-XY1>
                                <x>239</x>
                                <y>19</y>
```

```
                                </node-XY1>
                            </delta>
                        </NodeXY>
                    </nodes>
                </nodeList>
                <connectsTo>
                    <Connection>
                        <connectingLane>
                            <lane>83</lane>
                            <maneuver>
                                100000000000
                            </maneuver>
                        </connectingLane>
                        <remoteIntersection>
                            <id>27182</id>
                        </remoteIntersection>
                        <signalGroup>2</signalGroup>
                    </Connection>
                </connectsTo>
            </GenericLane>
            <GenericLane>
                <laneID>32</laneID>
                <ingressApproach>3</ingressApproach>
                <laneAttributes>
                    <directionalUse>
                        10
                    </directionalUse>
                    <sharedWith>
                        0000000000
                    </sharedWith>
                    <laneType>
                        <vehicle>
                            00000000
                        </vehicle>
                    </laneType>
                </laneAttributes>
                <nodeList>
```

```
                    <nodes>
                        <NodeXY>
                            <delta>
                                <node-XY2>
                                    <x>903</x>
                                    <y>590</y>
                                </node-XY2>
                            </delta>
                        </NodeXY>
                        <NodeXY>
                            <delta>
                                <node-XY6>
                                    <x>13572</x>
                                    <y>938</y>
                                </node-XY6>
                            </delta>
                        </NodeXY>
                    </nodes>
                </nodeList>
                <connectsTo>
                    <Connection>
                        <connectingLane>
                            <lane>82</lane>
                            <maneuver>
                                100000000000
                            </maneuver>
                        </connectingLane>
                        <remoteIntersection>
                            <id>27182</id>
                        </remoteIntersection>
                        <signalGroup>2</signalGroup>
                    </Connection>
                </connectsTo>
            </GenericLane>
            <GenericLane>
                <laneID>31</laneID>
                <ingressApproach>3</ingressApproach>
```

```
                    <laneAttributes>
                        <directionalUse>
                            10
                        </directionalUse>
                        <sharedWith>
                            0000000000
                        </sharedWith>
                        <laneType>
                            <vehicle>
                                00000000
                            </vehicle>
                        </laneType>
                    </laneAttributes>
                    <nodeList>
                        <nodes>
                            <NodeXY>
                                <delta>
                                    <node-XY2>
                                        <x>948</x>
                                        <y>976</y>
                                    </node-XY2>
                                </delta>
                            </NodeXY>
                            <NodeXY>
                                <delta>
                                    <node-XY6>
                                        <x>13520</x>
                                        <y>861</y>
                                    </node-XY6>
                                </delta>
                            </NodeXY>
                        </nodes>
                    </nodeList>
                    <connectsTo>
                        <Connection>
                            <connectingLane>
                                <lane>81</lane>
```

```
                        <maneuver>
                            100000000000
                        </maneuver>
                    </connectingLane>
                    <remoteIntersection>
                        <id>27182</id>
                    </remoteIntersection>
                    <signalGroup>2</signalGroup>
                </Connection>
            </connectsTo>
        </GenericLane>
        <GenericLane>
            <laneID>42</laneID>
            <egressApproach>4</egressApproach>
            <laneAttributes>
                <directionalUse>
                    01
                </directionalUse>
                <sharedWith>
                    0000000000
                </sharedWith>
                <laneType>
                    <vehicle>
                        00000000
                    </vehicle>
                </laneType>
            </laneAttributes>
            <nodeList>
                <nodes>
                    <NodeXY>
                        <delta>
                            <node-XY3>
                                <x>1077</x>
                                <y>-425</y>
                            </node-XY3>
                        </delta>
                    </NodeXY>
```

```
                    <NodeXY>
                        <delta>
                            <node-XY6>
                                <x>9013</x>
                                <y>540</y>
                            </node-XY6>
                        </delta>
                    </NodeXY>
                </nodes>
            </nodeList>
        </GenericLane>
        <GenericLane>
            <laneID>41</laneID>
            <egressApproach>4</egressApproach>
            <laneAttributes>
                <directionalUse>
                    01
                </directionalUse>
                <sharedWith>
                    0000000000
                </sharedWith>
                <laneType>
                    <vehicle>
                        00000000
                    </vehicle>
                </laneType>
            </laneAttributes>
            <nodeList>
                <nodes>
                    <NodeXY>
                        <delta>
                            <node-XY2>
                                <x>993</x>
                                <y>-168</y>
                            </node-XY2>
                        </delta>
                    </NodeXY>
```

```
                    <NodeXY>
                        <delta>
                            <node-XY6>
                                <x>9039</x>
                                <y>604</y>
                            </node-XY6>
                        </delta>
                    </NodeXY>
                </nodes>
            </nodeList>
        </GenericLane>
        <GenericLane>
            <laneID>71</laneID>
            <ingressApproach>7</ingressApproach>
            <laneAttributes>
                <directionalUse>
                    10
                </directionalUse>
                <sharedWith>
                    0000000000
                </sharedWith>
                <laneType>
                    <vehicle>
                        00000000
                    </vehicle>
                </laneType>
            </laneAttributes>
            <nodeList>
                <nodes>
                    <NodeXY>
                        <delta>
                            <node-XY2>
                                <x>-782</x>
                                <y>-226</y>
                            </node-XY2>
                        </delta>
                    </NodeXY>
```

```
                    <NodeXY>
                        <delta>
                            <node-XY6>
                                <x>-16716</x>
                                <y>-822</y>
                            </node-XY6>
                        </delta>
                    </NodeXY>
                </nodes>
            </nodeList>
            <connectsTo>
                <Connection>
                    <connectingLane>
                        <lane>41</lane>
                        <maneuver>
                            100000000000
                        </maneuver>
                    </connectingLane>
                    <remoteIntersection>
                        <id>27182</id>
                    </remoteIntersection>
                    <signalGroup>2</signalGroup>
                </Connection>
            </connectsTo>
        </GenericLane>
        <GenericLane>
            <laneID>72</laneID>
            <ingressApproach>7</ingressApproach>
            <laneAttributes>
                <directionalUse>
                    10
                </directionalUse>
                <sharedWith>
                    0000000000
                </sharedWith>
                <laneType>
                    <vehicle>
```

```
                    00000000
                </vehicle>
            </laneType>
        </laneAttributes>
        <nodeList>
            <nodes>
                <NodeXY>
                    <delta>
                        <node-XY2>
                            <x>-666</x>
                            <y>-605</y>
                        </node-XY2>
                    </delta>
                </NodeXY>
                <NodeXY>
                    <delta>
                        <node-XY6>
                            <x>-16903</x>
                            <y>-726</y>
                        </node-XY6>
                    </delta>
                </NodeXY>
            </nodes>
        </nodeList>
        <connectsTo>
            <Connection>
                <connectingLane>
                    <lane>42</lane>
                    <maneuver>
                        100000000000
                    </maneuver>
                </connectingLane>
                <remoteIntersection>
                    <id>27182</id>
                </remoteIntersection>
                <signalGroup>2</signalGroup>
            </Connection>
```

```
        </connectsTo>
    </GenericLane>
    <GenericLane>
        <laneID>81</laneID>
        <egressApproach>8</egressApproach>
        <laneAttributes>
            <directionalUse>
                01
            </directionalUse>
            <sharedWith>
                0000000000
            </sharedWith>
            <laneType>
                <vehicle>
                    00000000
                </vehicle>
            </laneType>
        </laneAttributes>
        <nodeList>
            <nodes>
                <NodeXY>
                    <delta>
                        <node-XY3>
                            <x>-1292</x>
                            <y>822</y>
                        </node-XY3>
                    </delta>
                </NodeXY>
                <NodeXY>
                    <delta>
                        <node-XY5>
                            <x>-6495</x>
                            <y>-353</y>
                        </node-XY5>
                    </delta>
                </NodeXY>
            </nodes>
```

```
        </nodeList>
    </GenericLane>
    <GenericLane>
        <laneID>82</laneID>
        <egressApproach>8</egressApproach>
        <laneAttributes>
            <directionalUse>
                01
            </directionalUse>
            <sharedWith>
                0000000000
            </sharedWith>
            <laneType>
                <vehicle>
                    00000000
                </vehicle>
            </laneType>
        </laneAttributes>
        <nodeList>
            <nodes>
                <NodeXY>
                    <delta>
                        <node-XY3>
                            <x>-1176</x>
                            <y>500</y>
                        </node-XY3>
                    </delta>
                </NodeXY>
                <NodeXY>
                    <delta>
                        <node-XY5>
                            <x>-6553</x>
                            <y>-328</y>
                        </node-XY5>
                    </delta>
                </NodeXY>
            </nodes>
```

```
        </nodeList>
    </GenericLane>
    <GenericLane>
        <laneID>83</laneID>
        <egressApproach>8</egressApproach>
        <laneAttributes>
            <directionalUse>
                01
            </directionalUse>
            <sharedWith>
                0000000000
            </sharedWith>
            <laneType>
                <vehicle>
                    00000000
                </vehicle>
            </laneType>
        </laneAttributes>
        <nodeList>
            <nodes>
                <NodeXY>
                    <delta>
                        <node-XY2>
                            <x>-956</x>
                            <y>108</y>
                        </node-XY2>
                    </delta>
                </NodeXY>
                <NodeXY>
                    <delta>
                        <node-XY6>
                            <x>-11073</x>
                            <y>-540</y>
                        </node-XY6>
                    </delta>
                </NodeXY>
            </nodes>
```

```
                            </nodeList>
                        </GenericLane>
                    </laneSet>
                </IntersectionGeometry>
            </intersections>
        </MapData>
    </value>
</MessageFrame>
```

Appendix C. Example HRI Active SPaT Message

Note that decoding the message from hexadecimal currently requires a special tool built from the SAE J2735 ASN.1 files.

```
$ echo
001335003 86A9F5DD8B2E1DA829644099048E7D104BD3BB3F3414B220A66A2E020010E4EADD00E0200208C
8805280520020434402940290 0 | xxd -r -p | ~/j2735-dump -iper -
<MessageFrame>
    <messageId>19</messageId>
    <value>
        <SPAT>
            <intersections>
                <IntersectionState>
                    <name>Sunbeam Rd &amp;amp; Hst Kings Rd S</name>
                    <id>
                        <id>27182</id>
                    </id>
                    <revision>1</revision>
                    <status>
                        0000000000001000
                    </status>
                    <moy>468822</moy>
                    <timeStamp>59399</timeStamp>
                    <states>
                        <MovementState>
                            <signalGroup>1</signalGroup>
                            <state-time-speed>
                                <MovementEvent>
                                    <eventState><protected-Movement-
Allowed/></eventState>
                                    <timing>
                                        <minEndTime>32850</minEndTime>
                                        <maxEndTime>32850</maxEndTime>
                                    </timing>
                                </MovementEvent>
                            </state-time-speed>
```

```
                        </MovementState>
                        <MovementState>
                            <signalGroup>2</signalGroup>
                            <state-time-speed>
                                <MovementEvent>
                                    <eventState><stop-And-Remain/></eventState>
                                    <timing>
                                        <minEndTime>32850</minEndTime>
                                        <maxEndTime>32850</maxEndTime>
                                    </timing>
                                </MovementEvent>
                            </state-time-speed>
                        </MovementState>
                    </states>
                </IntersectionState>
            </intersections>
        </SPAT>
    </value>
</MessageFrame>
```

Appendix D. Example HRI Inactive SPaT Message

Note that decoding the message from hexadecimal currently requires a special tool built from the SAE J2735 ASN.1 files.

```
$ echo
001335003 86A9F5DD8B2E1DA829644099048E7D104BD3BB3F3414B220A66A2E020000E4EB0B56E02002086
8805280520020454402940290 0 | xxd -r -p | ~/j2735-dump -iper -
<MessageFrame>
    <messageId>19</messageId>
    <value>
        <SPAT>
            <intersections>
                <IntersectionState>
                    <name>Sunbeam Rd &amp;amp; Hst Kings Rd S</name>
                    <id>
                        <id>27182</id>
                    </id>
                    <revision>1</revision>
                    <status>
                        0000000000000000
                    </status>
                    <moy>468824</moy>
                    <timeStamp>23223</timeStamp>
                    <states>
                        <MovementState>
                            <signalGroup>1</signalGroup>
                            <state-time-speed>
                                <MovementEvent>
                                    <eventState><stop-And-Remain/></eventState>
                                    <timing>
                                        <minEndTime>32850</minEndTime>
                                        <maxEndTime>32850</maxEndTime>
                                    </timing>
                                </MovementEvent>
                            </state-time-speed>
                        </MovementState>
```

```
                        <MovementState>
                            <signalGroup>2</signalGroup>
                            <state-time-speed>
                                <MovementEvent>
                                    <eventState><permissive-Movement-
Allowed/></eventState>
                                    <timing>
                                        <minEndTime>32850</minEndTime>
                                        <maxEndTime>32850</maxEndTime>
                                    </timing>
                                </MovementEvent>
                            </state-time-speed>
                        </MovementState>
                    </states>
                </IntersectionState>
            </intersections>
        </SPAT>
    </value>
</MessageFrame>
```

Appendix E. Example RTCM Message

Note that decoding the message from hexadecimal currently requires a special tool built from the SAE J2735 ASN.1 files.

```
$ echo
001C80A142740422C09AD300953EC6FA26C6E142900940C91200581451B2302A4010028D60E7CF640FFE64
A28F81827BFFA3347108122A3D7FF7F514EA4C230FFDE2A3386A6242C40150A8A6D460DD002AED16439A49
78A0077F4516F304AC017DE8D4222A14A20056FA2918982B400D8D471133894C98012ED1519CC23B00376A
2C8A8A0FF10000F68A45A607F7FFEFD19478F68F07FFC894513230613FF64E8EA05D7391 | xxd -r -p |
~/j2735-dump -iper -
<MessageFrame>
    <messageId>28</messageId>
    <value>
        <RTCMcorrections>
            <msgCnt>39</msgCnt>
            <rev><rtcmRev3/></rev>
            <timeStamp>8470</timeStamp>
            <msgs>
                <RTCMmessage>
                    D3 00 95 3E C6 FA 26 C6 E1 42 90 09 40 C9 12 00
                    58 14 51 B2 30 2A 40 10 02 8D 60 E7 CF 64 0F FE
                    64 A2 8F 81 82 7B FF A3 34 71 08 12 2A 3D 7F F7
                    F5 14 EA 4C 23 0F FD E2 A3 38 6A 62 42 C4 01 50
                    A8 A6 D4 60 DD 00 2A ED 16 43 9A 49 78 A0 07 7F
                    45 16 F3 04 AC 01 7D E8 D4 22 2A 14 A2 00 56 FA
                    29 18 98 2B 40 0D 8D 47 11 33 89 4C 98 01 2E D1
                    51 9C C2 3B 00 37 6A 2C 8A 8A 0F F1 00 00 F6 8A
                    45 A6 07 F7 FF EF D1 94 78 F6 8F 07 FF C8 94 51
                    32 30 61 3F F6 4E 8E A0 5D 73 91
                </RTCMmessage>
            </msgs>
        </RTCMcorrections>
    </value>
</MessageFrame>
$ echo
D300953EC6FA26C6E142900940C91200581451B2302A4010028D60E7CF640FFE64A28F81827BFFA3347108122
```